# Compiler intrinsics and assembly language

Learn how Visual Studio can create low-level, processor-specific code for maximum performance and control.

## Use compiler intrinsics

OVERVIEW

Compiler intrinsics overview

REFERENCE

ARM intrinsics

ARM64 intrinsics

x86 intrinsics list

x64 (amd64) intrinsics list

Intrinsics available on all architectures

Alphabetical intrinsics list

## Use inline assembly for x86 in Microsoft C/C++

OVERVIEW

Inline assembler overview

CONCEPT

The __asm keyword

HOW-TO GUIDE

Use assembly language in __asm blocks

Use C or C++ in __asm blocks

Use and preserve registers

Call C++ functions in inline assembly

Optimize inline assembly

## Use the ARM and ARM64 assembler

REFERENCE

ARM and ARM64 assembler reference

ARM and ARM64 assembler command-line reference

ARM and ARM64 assembler diagnostic messages

ARM and ARM64 assembler directives

## Use the x86 and x64 assembler

REFERENCE

Microsoft Macro Assembler for x86 and x64 reference

MASM for x64 (ml64.exe)

ML and ML64 command-line reference

Directives reference

Symbols reference

Operators reference

MASM BNF grammar

ML and ML64 error messages

Processor manufacturer programming manuals

# Compiler intrinsics

Most functions are contained in libraries, but some functions are built in (that is, intrinsic) to the compiler. These are referred to as intrinsic functions or intrinsics.

## Remarks

If a function is an intrinsic, the code for that function is usually inserted inline, avoiding the overhead of a function call and allowing highly efficient machine instructions to be emitted for that function. An intrinsic is often faster than the equivalent inline assembly, because the optimizer has a built-in knowledge of how many intrinsics behave, so some optimizations can be available that are not available when inline assembly is used. Also, the optimizer can expand the intrinsic differently, align buffers differently, or make other adjustments depending on the context and arguments of the call.

The use of intrinsics affects the portability of code, because intrinsics that are available in Microsoft C++ might not be available if the code is compiled with other compilers and some intrinsics that might be available for some target architectures are not available for all architectures. However, intrinsics are usually more portable than inline assembly. The intrinsics are required on 64-bit architectures where inline assembly is not supported.

Some intrinsics, such as `__assume` and `__ReadWriteBarrier`, provide information to the compiler, which affects the behavior of the optimizer.

Some intrinsics are available only as intrinsics, and some are available both in function and intrinsic implementations. You can instruct the compiler to use the intrinsic implementation in one of two ways, depending on whether you want to enable only specific functions or you want to enable all intrinsics. The first way is to use `#pragma intrinsic(`*intrinsic-function-name-list*`)`. The pragma can be used to specify a single intrinsic or multiple intrinsics separated by commas. The second is to use the /Oi (Generate intrinsic functions) compiler option, which makes all intrinsics on a given platform available. Under **/Oi**, use `#pragma function(`*intrinsic-function-name-list*`)` to force a function call to be used instead of an intrinsic. If the documentation for a specific intrinsic notes that the routine is only available as an intrinsic, then the intrinsic implementation is used regardless of whether **/Oi** or `#pragma intrinsic` is specified. In all cases, **/Oi** or `#pragma intrinsic` allows, but does not force, the optimizer to use the intrinsic. The optimizer can still call the function.

Some standard C/C++ library functions are available in intrinsic implementations on some architectures. When calling a CRT function, the intrinsic implementation is used if **/Oi** is specified on the command line.

A header file, <intrin.h>, is available that declares prototypes for the common intrinsic functions. Manufacturer-specific intrinsics are available in the <immintrin.h> and <ammintrin.h> header files. Additionally, certain Windows headers declare functions that map onto a compiler intrinsic.

The following sections list all intrinsics that are available on various architectures. For more information on how the intrinsics work on your particular target processor, refer to the manufacturer's reference documentation.

- ARM intrinsics
- ARM64 intrinsics
- x86 intrinsics list
- x64 (amd64) Intrinsics List
- Intrinsics available on all architectures
- Alphabetical listing of intrinsic functions

## See also

ARM assembler reference
Microsoft Macro Assembler reference
Keywords
C run-time library reference

---

Last updated on 11/05/2025

# Inline Assembler

Article • 08/03/2021

**Microsoft Specific**

Assembly language serves many purposes, such as improving program speed, reducing memory needs, and controlling hardware. You can use the inline assembler to embed assembly-language instructions directly in your C and C++ source programs without extra assembly and link steps. The inline assembler is built into the compiler, so you don't need a separate assembler such as the Microsoft Macro Assembler (MASM).

> **Note**
>
> Programs with inline assembler code are not fully portable to other hardware platforms. If you are designing for portability, avoid using inline assembler.

Inline assembly is not supported on the ARM and x64 processors. The following topics explain how to use the Visual C/C++ inline assembler with x86 processors:

- Inline Assembler Overview
- Advantages of Inline Assembly
- __asm
- Using Assembly Language in __asm Blocks
- Using C or C++ in __asm Blocks
- Using and Preserving Registers in Inline Assembly
- Jumping to Labels in Inline Assembly
- Calling C Functions in Inline Assembly
- Calling C++ Functions in Inline Assembly
- Defining __asm Blocks as C Macros
- Optimizing Inline Assembly

**END Microsoft Specific**

# See also

Compiler Intrinsics and Assembly Language

C++ Language Reference

# Inline Assembler Overview

**Microsoft Specific**

The inline assembler lets you embed assembly-language instructions in your C and C++ source programs without extra assembly and link steps. The inline assembler is built into the compiler — you don't need a separate assembler such as the Microsoft Macro Assembler (MASM).

Because the inline assembler doesn't require separate assembly and link steps, it is more convenient than a separate assembler. Inline assembly code can use any C or C++ variable or function name that is in scope, so it is easy to integrate it with your program's C and C++ code. And because the assembly code can be mixed with C and C++ statements, it can do tasks that are cumbersome or impossible in C or C++ alone.

The __asm keyword invokes the inline assembler and can appear wherever a C or C++ statement is legal. It cannot appear by itself. It must be followed by an assembly instruction, a group of instructions enclosed in braces, or, at the very least, an empty pair of braces. The term "`__asm` block" here refers to any instruction or group of instructions, whether or not in braces.

The following code is a simple `__asm` block enclosed in braces. (The code is a custom function prolog sequence.)

C++

```
// asm_overview.cpp
// processor: x86
void __declspec(naked) main()
{
    // Naked functions must provide their own prolog...
    __asm {
        push ebp
        mov ebp, esp
        sub esp, __LOCAL_SIZE
    }

    // ... and epilog
    __asm {
        pop ebp
        ret
    }
}
```

Alternatively, you can put `__asm` in front of each assembly instruction:

C++

```
__asm push ebp
__asm mov  ebp, esp
```

```
__asm sub  esp, __LOCAL_SIZE
```

Since the `__asm` keyword is a statement separator, you can also put assembly instructions on the same line:

C++

```
__asm push ebp   __asm mov  ebp, esp   __asm sub  esp, __LOCAL_SIZE
```

**END Microsoft Specific**

# See also

Inline Assembler

---

Last updated on 06/25/2025

# Advantages of Inline Assembly

Article • 08/03/2021

**Microsoft Specific**

Because the inline assembler doesn't require separate assembly and link steps, it is more convenient than a separate assembler. Inline assembly code can use any C variable or function name that is in scope, so it is easy to integrate it with your program's C code. Because the assembly code can be mixed inline with C or C++ statements, it can do tasks that are cumbersome or impossible in C or C++.

The uses of inline assembly include:

- Writing functions in assembly language.
- Spot-optimizing speed-critical sections of code.
- Making direct hardware access for device drivers.
- Writing prolog and epilog code for "naked" calls.

Inline assembly is a special-purpose tool. If you plan to port an application to different machines, you'll probably want to place machine-specific code in a separate module. Because the inline assembler doesn't support all of Microsoft Macro Assembler's (MASM) macro and data directives, you may find it more convenient to use MASM for such modules.

**END Microsoft Specific**

## See also

Inline Assembler

# `__asm`

**Microsoft Specific**

The `__asm` keyword invokes the inline assembler and can appear wherever a C or C++ statement is legal. It can't appear by itself. It must be followed by an assembly instruction, a group of instructions enclosed in braces, or, at minimum, an empty pair of braces. The term "`__asm` block" here refers to any instruction or group of instructions, whether or not in braces.

> **Note**
>
> MSVC support for the Standard C++ `asm` keyword is limited to the fact that the compiler will not generate an error on the keyword. However, an `asm` block will not generate any meaningful code. Use `__asm` instead of `asm`.

## Grammar

*asm-block*:
  `__asm` *assembly-instruction* `;`$_{opt}$
  `__asm {` *assembly-instruction-list* `}` `;`$_{opt}$

*assembly-instruction-list*:
  *assembly-instruction* `;`$_{opt}$
  *assembly-instruction* `;` *assembly-instruction-list* `;`$_{opt}$

## Remarks

If used without braces, the `__asm` keyword means that the rest of the line is an assembly-language statement. If used with braces, it means that each line between the braces is an assembly-language statement. For compatibility with previous versions, `_asm` is a synonym for `__asm` unless compiler option /Za (Disable language extensions) is specified.

Since the `__asm` keyword is a statement separator, you can put assembly instructions on the same line.

Before Visual Studio 2005, the instruction

C++

```
__asm int 3
```

didn't cause native code to be generated when compiled with **/clr**; the compiler translated the instruction to a CLR break instruction.

`__asm int 3` now results in native code generation for the function. If you want a function to cause a break point in your code and if you want that function compiled to MSIL, use __debugbreak.

# Example

The following code fragment is a simple `__asm` block enclosed in braces:

C++

```
__asm {
   mov al, 2
   mov dx, 0xD007
   out dx, al
}
```

Alternatively, you can put `__asm` in front of each assembly instruction:

C++

```
__asm mov al, 2
__asm mov dx, 0xD007
__asm out dx, al
```

Because the `__asm` keyword is a statement separator, you can also put assembly instructions on the same line:

C++

```
__asm mov al, 2   __asm mov dx, 0xD007   __asm out dx, al
```

All three examples generate the same code, but the first style (enclosing the `__asm` block in braces) has some advantages. The braces clearly separate assembly code from C or C++ code and avoid needless repetition of the `__asm` keyword. Braces can also prevent ambiguities. If you want to put a C or C++ statement on the same line as an `__asm` block, you must enclose the block in braces. Without the braces, the compiler can't tell where assembly code stops and C or C++ statements begin. Finally, because the text in braces has the same format as ordinary MASM text, you can easily cut and paste text from existing MASM source files.

Unlike braces in C and C++, the braces enclosing an `__asm` block don't affect variable scope. You can also nest `__asm` blocks; nesting doesn't affect variable scope.

END Microsoft Specific

## See also

Keywords
Inline Assembler

---

Last updated on 11/05/2025

# Using Assembly Language in __asm Blocks

**Microsoft Specific**

The inline assembler has much in common with other assemblers. For example, it accepts any expression that is legal in MASM. This section describes the use of assembly-language features in `__asm` blocks.

## What do you want to know more about?

- Instruction Set for Inline Assembly
- MASM Expressions in Inline Assembly
- Data Directives and Operators in Inline Assembly
- EVEN and ALIGN Directives
- MASM Macro Directives in Inline Assembly
- Segment References in Inline Assembly
- Type and Variable Sizes in Inline Assembly
- Assembly-Language Comments
- The _emit Pseudoinstruction
- Debugging and Listings for Inline Assembly
- Intel's MMX Instruction Set

**END Microsoft Specific**

## See also

Inline Assembler

---

Last updated on 06/25/2025

# Instruction Set for Inline Assembly

**Microsoft Specific**

The MSVC compiler supports all opcodes through the Pentium 4 and AMD Athlon. Additional instructions supported by the target processor can be created with the _emit Pseudoinstruction.

**END Microsoft Specific**

## See also

Using Assembly Language in __asm Blocks

---

Last updated on 11/05/2025

# MASM Expressions in Inline Assembly

Article • 08/03/2021

**Microsoft Specific**

Inline assembly code can use any MASM expression, which is any combination of operands and operators that evaluates to a single value or address.

**END Microsoft Specific**

## See also

Using Assembly Language in __asm Blocks

# Data directives and operators in inline assembly

Article • 06/06/2022

**Microsoft Specific**

The inline assembler doesn't support all the data directives and operators available in MASM.

## Remarks

Although an `__asm` block can reference C or C++ data types and objects, it can't define data objects with MASM directives or operators. Specifically, you can't use the definition directives `DB`, `DW`, `DD`, `DQ`, `DT`, and `DF`, or the operators `DUP` or `THIS`. MASM structures and records are also unavailable. The inline assembler doesn't accept the directives `STRUC`, `RECORD`, `WIDTH`, or `MASK`.

**END Microsoft Specific**

## See also

Using Assembly Language in __asm blocks

# `EVEN` and `ALIGN` Directives

**Microsoft Specific**

Although the inline assembler doesn't support most MASM directives, it does support EVEN and ALIGN. These directives put `NOP` (no operation) instructions in the assembly code as needed to align labels to specific boundaries. This makes instruction-fetch operations more efficient for some processors.

**END Microsoft Specific**

## See also

Using Assembly Language in __asm Blocks

---

Last updated on 08/22/2025

# MASM Macro Directives in Inline Assembly

Article • 03/02/2022

**Microsoft Specific**

The inline assembler isn't a macro assembler. You can't use MASM macro directives (`MACRO`, `REPT`, `IRC`, `IRP`, and `ENDM`) or macro operators (`<>`, `!`, `&`, `%`, and `.TYPE`). An `__asm` block can use C preprocessor directives, however. For more information, see Using C or C++ in __asm blocks.

**END Microsoft Specific**

## See also

Using assembly language in __asm blocks

# Segment References in Inline Assembly

Article • 08/03/2021

**Microsoft Specific**

You must refer to segments by register rather than by name (the segment name `_TEXT` is invalid, for instance). Segment overrides must use the register explicitly, as in ES:[BX].

**END Microsoft Specific**

## See also

Using Assembly Language in __asm Blocks

# Type and Variable Sizes in Inline Assembly

Article • 08/03/2021

**Microsoft Specific**

The **LENGTH**, **SIZE**, and **TYPE** operators have a limited meaning in inline assembly. They cannot be used at all with the `DUP` operator (because you cannot define data with MASM directives or operators). But you can use them to find the size of C or C++ variables or types:

- The **LENGTH** operator can return the number of elements in an array. It returns the value 1 for non-array variables.
- The **SIZE** operator can return the size of a C or C++ variable. A variable's size is the product of its **LENGTH** and **TYPE**.
- The **TYPE** operator can return the size of a C or C++ type or variable. If the variable is an array, **TYPE** returns the size of a single element of the array.

For example, if your program has an 8-element `int` array,

```
C++

int arr[8];
```

the following C and assembly expressions yield the size of `arr` and its elements.

| __asm | C | Size |
|---|---|---|
| **LENGTH** arr | `sizeof(arr)/sizeof(arr[0])` | 8 |
| **SIZE** arr | `sizeof(arr)` | 32 |
| **TYPE** arr | `sizeof(arr[0])` | 4 |

**END Microsoft Specific**

## See also

Using Assembly Language in __asm Blocks

# Assembly-Language Comments

Article • 08/03/2021

**Microsoft Specific**

Instructions in an `__asm` block can use assembly-language comments:

```
C++

__asm mov ax, offset buff ; Load address of buff
```

Because C macros expand into a single logical line, avoid using assembly-language comments in macros. (See Defining __asm Blocks as C Macros.) An `__asm` block can also contain C-style comments; for more information, see Using C or C++ in __asm Blocks.

**END Microsoft Specific**

## See also

Using Assembly Language in __asm Blocks

# _emit Pseudoinstruction

**Microsoft Specific**

The **_emit** pseudoinstruction defines one byte at the current location in the current text segment. The **_emit** pseudoinstruction resembles the DB directive of MASM.

The following fragment places the bytes 0x4A, 0x43, and 0x4B into the code:

C++

```
#define randasm __asm _emit 0x4A __asm _emit 0x43 __asm _emit 0x4B
.
.
.
__asm {
    randasm
    }
```

> ⊗ **Caution**
>
> If `_emit` generates instructions that modify registers, and you compile the application with optimizations, the compiler cannot determine what registers are affected. For example, if `_emit` generates an instruction that modifies the **rax** register, the compiler does not know that **rax** has changed. The compiler might then make an incorrect assumption about the value in that register after the inline assembler code executes. Consequently, your application might exhibit unpredictable behavior when it runs.

**END Microsoft Specific**

## See also

Using Assembly Language in __asm Blocks

---

Last updated on 06/25/2025

# Debugging and Listings for Inline Assembly

**Microsoft Specific**

Programs containing inline assembly code can be debugged with a source-level debugger if you compile with the /Zi option.

Within the debugger, you can set breakpoints on both C or C++ and assembly-language lines. If you enable mixed assembly and source mode, you can display both the source and disassembled form of the assembly code.

Note that putting multiple assembly instructions or source language statements on one line can hamper debugging. In source mode, you can use the debugger to set breakpoints on a single line but not on individual statements on the same line. The same principle applies to an `__asm` block defined as a C macro, which expands to a single logical line.

If you create a mixed source and assembly listing with the /FAs compiler option, the listing contains both the source and assembly forms of each assembly-language line. Macros are not expanded in listings, but they are expanded during compilation.

**END Microsoft Specific**

## See also

Using Assembly Language in __asm Blocks

---

Last updated on 06/25/2025

# Intel's MMX Instruction Set

**Microsoft Specific**

The MSVC compiler allows you to use Intel's MMX (multimedia extension) instruction set in the inline assembler. The MMX instructions are also supported by the debugger disassembly. The compiler generates a warning message if the function contains MMX instructions but does not contain an EMMS instruction to empty the multimedia state. For more information, see the Intel Web site.

**END Microsoft Specific**

## See also

Using Assembly Language in __asm Blocks

---

Last updated on 11/05/2025

# Using C or C++ in __asm Blocks

**Microsoft Specific**

Because inline assembly instructions can be mixed with C or C++ statements, they can refer to C or C++ variables by name and use many other elements of those languages.

An `__asm` block can use the following language elements:

- Symbols, including labels and variable and function names
- Constants, including symbolic constants and `enum` members
- Macros and preprocessor directives
- Comments (both **/* */** and **//** )
- Type names (wherever a MASM type would be legal)
- `typedef` names, generally used with operators such as **PTR** and **TYPE** or to specify structure or union members

Within an `__asm` block, you can specify integer constants with either C notation or assembler radix notation (0x100 and 100h are equivalent, for example). This allows you to define (using `#define`) a constant in C and then use it in both C or C++ and assembly portions of the program. You can also specify constants in octal by preceding them with a 0. For example, 0777 specifies an octal constant.

## What do you want to know more about?

- Using Operators in __asm Blocks
- Using C or C++ Symbols_in __asm Blocks
- Accessing C or C++ Data in __asm Blocks
- Writing Functions with Inline Assembly

**END Microsoft Specific**

## See also

Inline Assembler

Last updated on 06/25/2025

# Using Operators in __asm Blocks

**Microsoft Specific**

An `__asm` block cannot use C or C++ specific operators, such as the << operator. However, operators shared by C and MASM, such as the * operator, are interpreted as assembly-language operators. For instance, outside an `__asm` block, square brackets ([ ]) are interpreted as enclosing array subscripts, which C automatically scales to the size of an element in the array. Inside an `__asm` block, they are seen as the MASM index operator, which yields an unscaled byte offset from any data object or label (not just an array). The following code illustrates the difference:

```
C++
int array[10];

__asm mov array[6], bx ;  Store BX at array+6 (not scaled)

array[6] = 0;         /* Store 0 at array+24 (scaled) */
```

The first reference to `array` is not scaled, but the second is. Note that you can use the **TYPE** operator to achieve scaling based on a constant. For example, the following statements are equivalent:

```
C++
__asm mov array[6 * TYPE int], 0 ; Store 0 at array + 24

array[6] = 0;                   /* Store 0 at array + 24 */
```

**END Microsoft Specific**

## See also

Using C or C++ in __asm Blocks

---

Last updated on 06/25/2025

# Using C or C++ Symbols in __asm Blocks

**Microsoft Specific**

An `__asm` block can refer to any C or C++ symbol in scope where the block appears. (C and C++ symbols are variable names, function names, and labels; that is, names that aren't symbolic constants or `enum` members. You cannot call C++ member functions.)

A few restrictions apply to the use of C and C++ symbols:

- Each assembly-language statement can contain only one C or C++ symbol. Multiple symbols can appear in the same assembly instruction only with **LENGTH**, **TYPE**, and **SIZE** expressions.
- Functions referenced in an `__asm` block must be declared (prototyped) earlier in the program. Otherwise, the compiler cannot distinguish between function names and labels in the `__asm` block.
- An `__asm` block cannot use any C or C++ symbols with the same spelling as MASM reserved words (regardless of case). MASM reserved words include instruction names such as **PUSH** and register names such as SI.
- Structure and union tags are not recognized in `__asm` blocks.

**END Microsoft Specific**

## See also

Using C or C++ in __asm Blocks

---

Last updated on 06/25/2025

# Accessing C or C++ Data in __asm Blocks

**Microsoft Specific**

A great convenience of inline assembly is the ability to refer to C or C++ variables by name. An `__asm` block can refer to any symbols, including variable names, that are in scope where the block appears. For instance, if the C variable `var` is in scope, the instruction

C++

```
__asm mov eax, var
```

stores the value of `var` in EAX.

If a class, structure, or union member has a unique name, an `__asm` block can refer to it using only the member name, without specifying the variable or `typedef` name before the period (.) operator. If the member name is not unique, however, you must place a variable or `typedef` name immediately before the period operator. For example, the structure types in the following sample share `same_name` as their member name:.

If you declare variables with the types

C++

```
struct first_type hal;
struct second_type oat;
```

all references to the member `same_name` must use the variable name because `same_name` is not unique. But the member `weasel` has a unique name, so you can refer to it using only its member name:

C++

```
// InlineAssembler_Accessing_C_asm_Blocks.cpp
// processor: x86
#include <stdio.h>
struct first_type
{
   char *weasel;
   int same_name;
};

struct second_type
{
   int wonton;
   long same_name;
};
```

```
int main()
{
   struct first_type hal;
   struct second_type oat;

   __asm
   {
      lea ebx, hal
      mov ecx, [ebx]hal.same_name ; Must use 'hal'
      mov esi, [ebx].weasel       ; Can omit 'hal'
   }
   return 0;
}
```

Note that omitting the variable name is merely a coding convenience. The same assembly instructions are generated whether or not the variable name is present.

You can access data members in C++ without regard to access restrictions. However, you cannot call member functions.

**END Microsoft Specific**

## See also

Using C or C++ in __asm Blocks

---

Last updated on 06/25/2025

# Writing functions with inline assembly

**Microsoft Specific**

> **Note**
>
> Inline assembly is only available for x86 targets. For similar functionality in x64 or ARM64 code, use compiler intrinsics.

If you write a function with inline assembly code, it's easy to pass arguments to the function and return a value from it. The following examples compare a function first written for a separate assembler and then rewritten for the inline assembler. The function, called `power2`, receives two parameters, multiplying the first parameter by 2 to the power of the second parameter. As a separate assembler file, the function might look like this:

asm

```
; power2.asm
; x86 code for C interop
; Command line: ml /c /Cx /W3 /WX power2.asm
        .686P
        .XMM
        .MODEL  flat

PUBLIC  _power2
; int power2(int num, int power);
; computes num x 2^power
_TEXT   SEGMENT
_power2 PROC
        push    ebp             ; save EBP
        mov     ebp, esp        ; Move ESP into EBP so we can refer
                                ;   to arguments on the stack
        mov     eax, [ebp+8]    ; load first argument
        mov     ecx, [ebp+12]   ; load second argument
        shl     eax, cl         ; compute result in EAX
        pop     ebp             ; restore EBP
        ret
_power2 ENDP
_TEXT   ENDS
END
```

Since it's written as a separate assembler file, the function requires separate assembly and link steps. C and C++ function arguments are usually passed on the stack, so this version of the `power2` function accesses its arguments by their positions on the stack. (The `MODEL` directive, available in MASM and some other assemblers, also allows you to access stack arguments and local stack variables by name.)

# Example

This program writes the `power2` function with inline assembly code:

C++

```
// Power2_inline_asm.c
// compile with: /EHsc
// processor: x86

#include <stdio.h>

int power2( int num, int power );

int main( void )
{
    printf_s( "3 times 2 to the power of 5 is %d\n", \
              power2( 3, 5) );
}
int power2( int num, int power )
{
   __asm
   {
      mov eax, num    ; Get first argument
      mov ecx, power  ; Get second argument
      shl eax, cl     ; EAX = EAX * ( 2 to the power of CL )
   }
   // Return with result in EAX
}
```

The inline version of the `power2` function refers to its arguments by name and appears in the same source file as the rest of the program. This version also requires fewer assembly instructions.

Because the inline version of `power2` doesn't execute a C `return` statement, it causes a harmless warning if you compile at warning level 2 or higher. The function does return a value, but the compiler can't tell it does in the absence of a `return` statement. You can use #pragma warning to disable the generation of this warning.

**END Microsoft Specific**

# See also

Using C or C++ in __asm Blocks

---

Last updated on 06/25/2025

# Using and Preserving Registers in Inline Assembly

**Microsoft Specific**

In general, you should not assume that a register will have a given value when an `__asm` block begins. Register values are not guaranteed to be preserved across separate `__asm` blocks. If you end a block of inline code and begin another, you cannot rely on the registers in the second block to retain their values from the first block. An `__asm` block inherits whatever register values result from the normal flow of control.

If you use the `__fastcall` calling convention, the compiler passes function arguments in registers instead of on the stack. This can create problems in functions with `__asm` blocks because a function has no way to tell which parameter is in which register. If the function happens to receive a parameter in EAX and immediately stores something else in EAX, the original parameter is lost. In addition, you must preserve the ECX register in any function declared with `__fastcall`.

To avoid such register conflicts, don't use the `__fastcall` convention for functions that contain an `__asm` block. If you specify the `__fastcall` convention globally with the /Gr compiler option, declare every function containing an `__asm` block with `__cdecl` or `__stdcall`. (The `__cdecl` attribute tells the compiler to use the C calling convention for that function.) If you are not compiling with /Gr, avoid declaring the function with the `__fastcall` attribute.

When using `__asm` to write assembly language in C/C++ functions, you don't need to preserve the EAX, EBX, ECX, EDX, ESI, or EDI registers. For example, in the POWER2.C example in Writing Functions with Inline Assembly, the `power2` function doesn't preserve the value in the EAX register. However, using these registers will affect code quality because the register allocator cannot use them to store values across `__asm` blocks. In addition, by using EBX, ESI or EDI in inline assembly code, you force the compiler to save and restore those registers in the function prologue and epilogue.

You should preserve other registers you use (such as DS, SS, SP, BP, and flags registers) for the scope of the `__asm` block. You should preserve the ESP and EBP registers unless you have some reason to change them (to switch stacks, for example). Also see Optimizing Inline Assembly.

Some SSE types require eight-byte stack alignment, forcing the compiler to emit dynamic stack-alignment code. To be able to access both the local variables and the function parameters after the alignment, the compiler maintains two frame pointers. If the compiler performs frame pointer omission (FPO), it will use EBP and ESP. If the compiler does not perform FPO, it will use EBX and EBP. To ensure code runs correctly, do not modify EBX in asm

code if the function requires dynamic stack alignment as it could modify the frame pointer. Either move the eight-byte aligned types out of the function, or avoid using EBX.

> **Note**
>
> If your inline assembly code changes the direction flag using the STD or CLD instructions, you must restore the flag to its original value.

**END Microsoft Specific**

## See also

Inline Assembler

---

Last updated on 06/25/2025

# Jumping to Labels in Inline Assembly

**Microsoft Specific**

Like an ordinary C or C++ label, a label in an `__asm` block has scope throughout the function in which it is defined (not only in the block). Both assembly instructions and `goto` statements can jump to labels inside or outside the `__asm` block.

Labels defined in `__asm` blocks are not case sensitive; both `goto` statements and assembly instructions can refer to those labels without regard to case. C and C++ labels are case sensitive only when used by `goto` statements. Assembly instructions can jump to a C or C++ label without regard to case.

The following code shows all the permutations:

C++

```
void func( void )
{
   goto C_Dest;  /* Legal: correct case   */
   goto c_dest;  /* Error: incorrect case */

   goto A_Dest;  /* Legal: correct case   */
   goto a_dest;  /* Legal: incorrect case */

   __asm
   {
      jmp C_Dest ; Legal: correct case
      jmp c_dest ; Legal: incorrect case

      jmp A_Dest ; Legal: correct case
      jmp a_dest ; Legal: incorrect case

      a_dest:    ; __asm label
   }

   C_Dest:       /* C label */
   return;
}
int main()
{
}
```

Don't use C library function names as labels in `__asm` blocks. For instance, you might be tempted to use `exit` as a label, as follows:

C++

```
; BAD TECHNIQUE: using library function name as label
   jne exit
   .
   .
   .
exit:
   ; More __asm code follows
```

Because **exit** is the name of a C library function, this code might cause a jump to the **exit** function instead of to the desired location.

As in MASM programs, the dollar symbol (`$`) serves as the current location counter. It is a label for the instruction currently being assembled. In `__asm` blocks, its main use is to make long conditional jumps:

C++

```
   jne $+5 ; next instruction is 5 bytes long
   jmp farlabel ; $+5
   .
   .
   .
farlabel:
```

**END Microsoft Specific**

# See also

Inline Assembler

---

Last updated on 06/25/2025

# Calling C Functions in Inline Assembly

**Microsoft Specific**

An `__asm` block can call C functions, including C library routines. The following example calls the `printf` library routine:

C++

```
// InlineAssembler_Calling_C_Functions_in_Inline_Assembly.cpp
// processor: x86
#include <stdio.h>

char format[] = "%s %s\n";
char hello[] = "Hello";
char world[] = "world";
int main( void )
{
   __asm
   {
      mov  eax, offset world
      push eax
      mov  eax, offset hello
      push eax
      mov  eax, offset format
      push eax
      call printf
      //clean up the stack so that main can exit cleanly
      //use the unused register ebx to do the cleanup
      pop  ebx
      pop  ebx
      pop  ebx
   }
}
```

Because function arguments are passed on the stack, you simply push the needed arguments — string pointers, in the previous example — before calling the function. The arguments are pushed in reverse order, so they come off the stack in the desired order. To emulate the C statement

C++

```
printf( format, hello, world );
```

the example pushes pointers to `world`, `hello`, and `format`, in that order, and then calls `printf`.

**END Microsoft Specific**

# See also

Inline Assembler

---

Last updated on 06/25/2025

# Calling C++ Functions in Inline Assembly

**Microsoft Specific**

An `__asm` block can call only global C++ functions that are not overloaded. If you call an overloaded global C++ function or a C++ member function, the compiler issues an error.

You can also call any functions declared with **extern "C"** linkage. This allows an `__asm` block within a C++ program to call the C library functions, because all the standard header files declare the library functions to have **extern "C"** linkage.

**END Microsoft Specific**

## See also

Inline Assembler

---

Last updated on 06/25/2025

# Defining __asm Blocks as C Macros

**Microsoft Specific**

C macros offer a convenient way to insert assembly code into your source code, but they demand extra care because a macro expands into a single logical line. To create trouble-free macros, follow these rules:

- Enclose the `__asm` block in braces.
- Put the `__asm` keyword in front of each assembly instruction.
- Use old-style C comments (`/* comment */`) instead of assembly-style comments (`; comment`) or single-line C comments (`// comment`).

To illustrate, the following example defines a simple macro:

C++

```
#define PORTIO __asm      \
/* Port output */         \
{                         \
   __asm mov al, 2        \
   __asm mov dx, 0xD007   \
   __asm out dx, al       \
}
```

At first glance, the last three `__asm` keywords seem superfluous. They are needed, however, because the macro expands into a single line:

C++

```
__asm /* Port output */ { __asm mov al, 2  __asm mov dx, 0xD007 __asm out dx, al }
```

The third and fourth `__asm` keywords are needed as statement separators. The only statement separators recognized in `__asm` blocks are the newline character and `__asm` keyword. Because a block defined as a macro is one logical line, you must separate each instruction with `__asm`.

The braces are essential as well. If you omit them, the compiler can be confused by C or C++ statements on the same line to the right of the macro invocation. Without the closing brace, the compiler cannot tell where assembly code stops, and it sees C or C++ statements after the `__asm` block as assembly instructions.

Assembly-style comments that start with a semicolon (;) continue to the end of the line. This causes problems in macros because the compiler ignores everything after the comment, all the

way to the end of the logical line. The same is true of single-line C or C++ comments (`// comment`). To prevent errors, use old-style C comments (`/* comment */`) in `__asm` blocks defined as macros.

An `__asm` block written as a C macro can take arguments. Unlike an ordinary C macro, however, an `__asm` macro cannot return a value. So you cannot use such macros in C or C++ expressions.

Be careful not to invoke macros of this type indiscriminately. For instance, invoking an assembly-language macro in a function declared with the `__fastcall` convention may cause unexpected results. (See Using and Preserving Registers in Inline Assembly.)

**END Microsoft Specific**

## See also

Inline Assembler

---

Last updated on 06/25/2025

# Optimizing Inline Assembly

**Microsoft Specific**

The presence of an `__asm` block in a function affects optimization in several ways. First, the compiler doesn't try to optimize the `__asm` block itself. What you write in assembly language is exactly what you get. Second, the presence of an `__asm` block affects register variable storage. The compiler avoids enregistering variables across an `__asm` block if the register's contents would be changed by the `__asm` block. Finally, some other function-wide optimizations will be affected by the inclusion of assembly language in a function.

**END Microsoft Specific**

## See also

Inline Assembler

---

Last updated on 06/25/2025

# ARM Assembler reference

Article • 05/10/2022

The articles in this section of the documentation provide reference material for the Microsoft ARM assembler (**armasm** or **armasm64**) and related tools.

## Related articles

| Title | Description |
|---|---|
| ARM Assembler command-line reference | Describes the Microsoft armasm and armasm64 command-line options. |
| ARM Assembler diagnostic messages | Describes commonly seen armasm and armasm64 warning and error messages. |
| ARM Assembler directives | Describes the ARM directives that are different in Microsoft armasm and armasm64. |
| ARM Architecture Reference Manual on the ARM Developer website. | Choose the relevant manual for your ARM architecture. Each contains reference sections about ARM, Thumb, NEON, and VFP, and additional information about the ARM assembly language. |
| ARM Compiler armasm User Guide on the ARM Developer website. | Choose a recent version to find up-to-date information about the ARM assembly language. |

> **Important**
>
> The armasm assembler that the ARM Developer website describes isn't the same as the Microsoft armasm assembler that's included in Visual Studio and is documented in this section.

## See also

ARM intrinsics
ARM64 intrinsics
Compiler intrinsics

# ARM Assembler command-line reference

The Microsoft ARM assemblers, **armasm** and **armasm64**, support several command-line options. By default, **armasm** assembles ARMv7 Thumb assembly language into the Microsoft implementation of the Common Object File Format (COFF). The **armasm64** assembler creates COFF object code for ARM64 and ARM64EC targets. The linker can link COFF code objects produced by both the ARM assembler and the MSVC compiler. It can link either together with object libraries created by the librarian.

## Syntax

> `armasm` [*options*] *source_file object_file*
> `armasm` [*options*] *source_file*

> `armasm64` [*options*] *source_file object_file*
> `armasm64` [*options*] *source_file*

### Parameters

*options*
A combination of zero or more of the following options:

- `-16`
  Available only in **armasm**. Assemble source as 16-bit Thumb instructions. This option is the default.

- `-32`
  Available only in **armasm**. Assemble source as 32-bit ARM instructions.

- `-coff_thumb2_only`
  Available only in **armasm**. Allow only Thumb-2 code.

- `-errorReport:` *option*
  This option is deprecated. In Windows Vista and later, error reporting is controlled by Windows Error Reporting (WER) settings.

- `-errors` *filename*
  `-e` *filename*
  Redirect error and warning messages to *filename*. By default, these messages are sent to `stdout`.

- `-funcOverride:` *function_name*
  Available only in **armasm64**. Emit function overriding support for *function_name*.

- `-g`
  Generate debugging information.

- `-gh:SHA1`
  Use the SHA-1 cryptographic hash algorithm to generate a checksum of each source file in debug info. Overrides `-gh:SHA256`.

- `-gh:SHA256`
  Use the SHA-256 cryptographic hash algorithm to generate a checksum of each source file in debug info. This option is on by default in Visual Studio 2022 version 17.0 and later.

- `-guard:ehcont`[`-`]
  Generate a sorted list of the relative virtual addresses (RVA) of all the valid exception handling continuation targets for a binary. It's used at runtime for `NtContinue` and `SetThreadContext` instruction pointer validation. By default, `-guard:ehcont` is off and must be explicitly enabled. To explicitly disable this option, use `-guard:ehcont-`. This option is available in Visual Studio 2019 version 16.7 and later. The feature is supported for 64-bit processes on a 64-bit OS.

- `-help`
  `-h`
  Print the command-line help message.

- `-i` *dir*[`;` *dir*]
  Add one or more specified directories to the include search path. Separate directories by using a semi-colon (`;`).

- `-ignore` *warning_number*
  Disable the specified warning number. For possible values, see ARM Assembler diagnostic messages.

- `-list` *list_file*
  Create a detailed listing of the generated assembly language to *list_file*. The *list_file* parameter is optional. If it's omitted, the assembler appends `.lst` to the base name of *source_file* to create the listing file.

- `-machine` *machine*
  Specify the machine type to set in the PE header. In **armasm**, possible values for *machine* are:

- **ARM**—Sets the machine type to `IMAGE_FILE_MACHINE_ARMNT`. This option is the default.
- **THUMB**—Sets the machine type to `IMAGE_FILE_MACHINE_THUMB`.

In **armasm64**, possible values are:

- **ARM64**—Sets the machine type to `IMAGE_FILE_MACHINE_ARM64`. This option is the default.
- **ARM64EC**—Sets the machine type to `IMAGE_FILE_MACHINE_ARM64EC`.

- **`-noesc`**
  **`-noe`**
  Ignore C-style escaped special characters, such as `\n` or `\t`.

- **`-nologo`**
  Suppress the copyright banner.

- **`-nowarn`**
  **`-now`**
  Disable all warning messages.

- **`-o`** *object_file*
  Specify the name of the object (output) file. The **`-o`** option is optional; you can instead specify an object file name as the last element of the command line.

- **`-oldit`**
  Available only in **armasm**. Generate ARMv7-style IT blocks. By default, ARMv8-compatible IT blocks are generated.

- **`-predefine`** *directive*
  **`-pd`** *directive*
  Specify a SETA, SETL, or SETS directive to predefine a symbol.
  Example: `armasm.exe -predefine "COUNT SETA 150" source.asm`
  For more information, see the ARM Compiler armasm Reference Guide.

- **`-sourcelink:`** *sourcelink_filename*
  *sourcelink_filename* specifies a JSON-formatted configuration file that contains a simple mapping of local file paths to URLs for source files to display in the debugger. For more information on the format of this file, see Source Link JSON Schema. Source Link is a language- and source-control agnostic system for providing source debugging for binaries. Source Link is supported for native binaries starting in Visual Studio 2017 version 15.8. For an overview of Source Link, see Source Link. For information on how to use Source Link in your projects, and how to generate the SourceLink file as part of your project, see Using Source Link.

- `-via` *filename*

  Read extra command-line arguments from *filename*.

*source_file*
The name of the source file.

*object_file*
The last element of the command line can specify the name of the object (output) file. If it's omitted, and no `-o` option is specified, the assembler appends `.obj` to the base name of *source_file* to create the object file.

# Remarks

The following example demonstrates how to use armasm in a typical scenario. First, use armasm to build an assembly language source (.asm) file to an object (.obj) file. Then, use the CL command-line C compiler to compile a source (.c) file, and also specify the linker option to link the ARM object file.

Windows Command Prompt

```
armasm -o myasmcode.obj myasmcode.asm
cl myccode.c /link myasmcode.obj
```

# See also

ARM Assembler diagnostic messages
ARM Assembler directives

---

Last updated on 11/05/2025

# ARM Assembler diagnostic messages

Article • 05/10/2022

The Microsoft ARM assemblers, **armasm** and **armasm64**, emit diagnostic warnings and errors when they encounter them. This article describes the most commonly encountered messages.

## Syntax

> *filename* **(** *line-number* **) : [error|warning] A** *number* **:** *message*

## Diagnostic messages - Errors

> A2193: this instruction generates unpredictable behavior

The ARM architecture can't guarantee what happens when this instruction is executed. For details about the well-defined forms of this instruction, consult the ARM Architecture Reference Manual.

```asm
    ADD r0, r8, pc          ; A2193: this instruction generates unpredictable
behavior
```

> A2196: instruction cannot be encoded in 16 bits

The specified instruction can't be encoded as a 16-bit Thumb instruction. Specify a 32-bit instruction, or rearrange code to bring the target label into the range of a 16-bit instruction.

The assembler may attempt to encode a branch in 16 bits and fail with this error, even though a 32-bit branch is encodable. You can solve this problem by using the `.W` specifier to explicitly mark the branch as 32-bit.

```asm
    ADD.N r0, r1, r2      ; A2196: instruction can't be encoded in 16 bits

    B.W label             ; OK
    B.N label             ; A2196: instruction can't be encoded in 16 bits
```

```
    SPACE 10000
label
```

> A2202: Pre-UAL instruction syntax not allowed in THUMB region

Thumb code must use the Unified Assembler Language (UAL) syntax. The old syntax is no longer accepted

asm

```
    ADDEQS r0, r1         ; A2202: Pre-UAL instruction syntax not allowed in
THUMB region
    ADDSEQ r0, r1         ; OK
```

> A2513: Rotation must be even

In ARM mode, there's an alternate syntax for specifying constants. Instead of writing `#<const>`, you can write `#<byte>,#<rot>`, which represents the constant value that's obtained by rotating the value `<byte>` right by `<rot>`. When you use this syntax, you must make the value of `<rot>` even.

asm

```
    MOV r0, #4, #2       ; OK
    MOV r0, #4, #1       ; A2513: Rotation must be even
```

> A2557: Incorrect number of bytes to write back

On the NEON structure load and store instructions (`VLDn`, `VSTn`), there's an alternate syntax for specifying writeback to the base register. Instead of putting an exclamation point (!) after the address, you can specify an immediate value that indicates the offset to be added to the base register. If you use this syntax, you must specify the exact number of bytes that were loaded or stored by the instruction.

asm

```
    VLD1.8 {d0-d3}, [r0]!         ; OK
    VLD1.8 {d0-d3}, [r0], #32     ; OK
    VLD1.8 {d0-d3}, [r0], #100    ; A2557: Incorrect number of bytes to
write back
```

# Diagnostic messages - Warnings

> A4228: Alignment value exceeds AREA alignment; alignment not guaranteed

The alignment that's specified in an `ALIGN` directive is greater than the alignment of the enclosing `AREA`. As a result, the assembler can't guarantee that the `ALIGN` directive will be honored.

To fix this warning, you can specify on the `AREA` directive an `ALIGN` attribute that's equal to or greater than the desired alignment.

asm

```
AREA |.myarea1|
ALIGN 8           ; A4228: Alignment value exceeds AREA alignment; alignment
not guaranteed

AREA |.myarea2|,ALIGN=3
ALIGN 8           ; OK
```

> A4508: Use of this rotated constant is deprecated

In ARM mode, there's an alternate syntax for specifying constants. Instead of writing `#<const>`, you can write `#<byte>,#<rot>`, which represents the constant value that's obtained by rotating the value `<byte>` right by `<rot>`. In some contexts, ARM has deprecated the use of these rotated constants. In these cases, use the basic `#<const>` syntax instead.

asm

```
    ANDS r0, r0, #1               ; OK
    ANDS r0, r0, #4, #2           ; A4508: Use of this rotated constant is
deprecated
```

> A4509: This form of conditional instruction is deprecated

This form of conditional instruction has been deprecated by ARM in the ARMv8 architecture. We recommend that you change the code to use conditional branches. To see which conditional instructions are still supported, consult the ARM Architecture Reference Manual.

This warning isn't emitted when the `-oldit` command-line switch is used.

```asm
    ADDEQ r0, r1, r8              ; A4509: This form of conditional
instruction is deprecated
```

## See also

ARM Assembler Command-Line Reference

ARM Assembler Directives

# ARM Assembler Directives

Article • 08/25/2021

Generally, the Microsoft ARM assembler uses the ARM assembly language, which is documented in the ARM Compiler armasm Reference Guide. However, the Microsoft implementations of some assembly directives differ from the ARM assembly directives. This article explains the differences.

## Microsoft Implementations of ARM Assembly Directives

- `AREA`

  The Microsoft ARM assembler supports these `AREA` attributes: `ALIGN`, `CODE`, `CODEALIGN`, `DATA`, `NOINIT`, `READONLY`, `READWRITE`, `THUMB`, `ARM`.

  All except `THUMB` and `ARM` work as documented in the ARM Compiler armasm Reference Guide.

  In the Microsoft ARM assembler, `THUMB` indicates that a `CODE` section contains Thumb code, and is the default for `CODE` sections. `ARM` indicates that the section contains ARM code.

- `ATTR`

  Not supported.

- `CODE16`

  Not supported because it implies pre-UAL Thumb syntax, which the Microsoft ARM assembler doesn't allow. Use the `THUMB` directive instead, along with UAL syntax.

- `COMMON`

  Specification of an alignment for the common region isn't supported.

- `DCDO`

  Not supported.

- `DN`, `QN`, `SN`

  Specification of a type or a lane on the register alias isn't supported.

- `ENTRY`

  Not supported.

- `EQU`

  Specification of a type for the defined symbol isn't supported.

- `EXPORT` and `GLOBAL`

  Specifies exports using this syntax:

  > **`EXPORT`**|**`GLOBAL`** *`symbol`*{*`[type]`*}

  *`symbol`* is the symbol to be exported. *`[type]`*, if specified, can be either `[DATA]` to indicate that the symbol points to data or `[FUNC]` to indicate that the symbol points to code. `GLOBAL` is a synonym for `EXPORT`.

- `EXPORTAS`

  Not supported.

- `FRAME`

  Not supported.

- `FUNCTION` and `PROC`

  The assembly syntax lets you specify a custom calling convention on procedures: you list the registers that are caller-save, and the ones that are callee-save. However, while the Microsoft ARM assembler accepts the syntax, it ignores the register lists. The debug information that's produced by the assembler supports only the default calling convention.

- `IMPORT` and `EXTERN`

  Specifies imports using this syntax:

  > **`IMPORT`**|**`EXTERN`** *`symbol`*{**`, WEAK`** *`alias`*{**`, TYPE`** *`t`*}}

  *`symbol`* is the name of the symbol to be imported.

  If `WEAK` *`alias`* is specified, it indicates that *`symbol`* is a weak external. If a definition for it isn't found at link time, then all references to it bind instead to *`alias`*.

If `TYPE` *`t`* is specified, then *`t`* indicates how the linker should attempt to resolve *`symbol`*. These values for *`t`* are possible:

| Value | Description |
|---|---|
| 1 | Don't search libraries for *`symbol`*. |
| 2 | Search libraries for *`symbol`*. |
| 3 | *`symbol`* is an alias for *`alias`* (default). |

`EXTERN` is a synonym for `IMPORT`, except that *`symbol`* is imported only if there are references to it in the current assembly.

- `MACRO`

  The use of a variable to hold the condition code of a macro isn't supported. Default values for macro parameters aren't supported.

- `NOFP`

  Not supported.

- `OPT`, `TTL`, `SUBT`

  Not supported, because the Microsoft ARM assembler doesn't produce listings.

- `PRESERVE8`

  Not supported.

- `RELOC`

  `RELOC n` can only follow an instruction or a data definition directive. There's no "anonymous symbol" that can be relocated.

- `REQUIRE`

  Not supported.

- `REQUIRE8`

  Not supported.

- `THUMBX`

Not supported, because the Microsoft ARM assembler doesn't support the Thumb-2EE instruction set.

## See also

ARM Assembler Command-Line Reference
ARM Assembler Diagnostic Messages

# Microsoft Macro Assembler reference

Article • 09/22/2021

The Microsoft Macro Assembler (MASM) provides several advantages over inline assembly. MASM contains a macro language that has features such as looping, arithmetic, and string processing. MASM gives you greater control over the hardware. By using MASM, you also can reduce time and memory overhead in your code.

## In This Section

ML and ML64 command-line options
Describes the ML and ML64 command-line options.

MASM for x64 (ml64.exe)
Information about how to create output files for x64.

Instruction Format
Describes basic instruction format and instruction prefixes for MASM.

Directives reference
Provides links to articles that discuss the use of directives in MASM.

Symbols Reference
Provides links to articles that discuss the use of symbols in MASM.

Operators Reference
Provides links to articles that discuss the use of operators in MASM.

ML error messages
Describes fatal and nonfatal error messages and warnings.

Processor Manufacturer Programming Manuals
Provides links to programming information about processors not manufactured, sold, or supported by Microsoft.

MASM BNF Grammar
Formal BNF description of MASM for x64.

## Related Sections

C++ in Visual Studio
Provides links to different areas of the Visual Studio and Visual C++ documentation.

## See also

Compiler Intrinsics
x86 Intrinsics
x64 (amd64) Intrinsics

## Feedback

Was this page helpful? Yes No

Provide product feedback | Get help at Microsoft Q&A

# MASM for x64 (ml64.exe)

Visual Studio includes both 32-bit and 64-bit hosted versions of MASM (the Microsoft Macro Assembler) to target x64 code. Named ml64.exe, it's the assembler that accepts x64 assembly language. The MASM command-line tools are installed when you choose a C++ workload during Visual Studio installation. The MASM tools aren't available as a separate download. For instructions on how to download and install a copy of Visual Studio, see Install Visual Studio. If you only want the command-line tools, not the full IDE, download the Build Tools for Visual Studio.

To use ml64.exe on the command line, start a developer command prompt for x64 targets. A developer command prompt sets the required path and other environment variables. For information on how to start a developer command prompt, see Build C/C++ code on the command line.

For information on ml64.exe command-line options, see ML and ML64 Command-Line Reference.

Inline assembler or use of the `ASM` keyword isn't supported for x64 or ARM64 targets. To port your x86 code that uses inline assembler to x64 or ARM64, you can convert your code to C++, use compiler intrinsics, or create assembler-language source files. The Microsoft C++ compiler supports intrinsics to allow you to use special-function instructions, for example, privileged, bit scan or test, interlocked, and so on, in as close to a cross-platform manner as possible. For information on available intrinsics, see Compiler Intrinsics.

## Add an assembler-language file to a Visual Studio C++ project

The Visual Studio project system supports assembler-language files built by using MASM in your C++ projects. MASM fully supports x64 assembler-language source files, and builds them into object files. You can then link these object files to your C++ code built for x64 targets. It's one way to overcome the lack of an x64 inline assembler.

### To add an assembler-language file to an existing Visual Studio C++ project

1. Select the project in **Solution Explorer**. On the menu bar, choose **Project**, **Build Customizations**.

2. In the **Visual C++ Build Customization Files** dialog box, check the checkbox next to **masm(.targets,.props)**. Choose **OK** to save your selection and close the dialog box.
3. On the menu bar, choose **Project**, **Add New Item**.
4. In the **Add New Item** dialog box, select **C++ file (.cpp)** in the center pane. In the **Name** edit control, enter a new file name that has a `.asm` extension instead of `.cpp`. Choose **Add** to add the file to your project and close the dialog box.

Create your assembler-language code in the `.asm` file you added. When you build your solution, the MASM assembler is invoked to assemble the `.asm` file into an object file that is then linked into your project. To make symbol access easier, declare your assembler functions as `extern "C"` in your C++ source code, rather than using the C++ name decoration conventions in your assembler-language source files.

# ml64-specific directives

You can use the following ml64-specific directives in your assembler-language source code that targets x64:

- .ALLOCSTACK
- .ENDPROLOG
- .PUSHFRAME
- .PUSHREG
- .SAVEREG
- .SAVEXMM128
- .SETFRAME

The PROC directive is also updated for use with ml64.exe.

# Unwind Version 3 directives (experimental)

> **Important**
>
> Unwind Version 3 support is experimental and is subject to change.

Unwind Version 3 is an updated unwind code scheme for x64 that generates richer stack unwind metadata. It introduces new epilogue directives, extends register support, and changes the directive ordering model.

## Enable Unwind Version 3

Assemble with the `/unwindv3` option: `ml64 /unwindv3 filename.asm /link /entry:FunctionName`

## Key differences from Version 1

Expand table

| Feature | Version 1 | Version 3 |
|---|---|---|
| Directive ordering | Directive follows instruction | Directive precedes instruction |
| Unwind codes | `UWOP_*` | `WOD_*` |
| `.ALLOCSTACK` size variants | 2 (`UWOP_ALLOC_SMALL`, `UWOP_ALLOC_LARGE`) | 3 (adds `WOD_ALLOC_HUGE` for > 32 KB) |
| `.PUSHREG` register range | R0–R15 | R0–R31 |
| Epilogue recording | Optional | Mandatory (`.BEGINEPILOG`/`.ENDEPILOG` required) |
| Register-pair push/pop | Not supported | `.PUSH2REG`/`.POP2REG` directives |

Epilogue recording is mandatory in Version 3: `.BEGINEPILOG` and `.ENDEPILOG` must surround the epilogue instructions. Without them, no epilogue unwind codes are emitted.

> **Note**
>
> In Unwind Version 1 the directives come after the instruction. In Unwind Version 3, the directives come before the instruction.

For a complete list of Version 3 directives, see x64 Unwind Version 3 directives and @UnwindVersion.

# 32-Bit Address Mode (Address Size Override)

MASM emits the 0x67 address size override if a memory operand includes 32-bit registers. For example, the following examples cause the address size override to be emitted:

**asm**

```
mov rax, QWORD PTR [ecx]
mov eax, DWORD PTR [ecx*2+r10d]
mov eax, DWORD PTR [ecx*2+r10d+0100h]
```

```
prefetch [eax]
movnti rax, QWORD PTR [r8d]
```

MASM assumes 64-bit addressing if a 32-bit displacement appears alone as a memory operand. There's currently no support for 32-bit addressing with such operands.

Finally, mixing register sizes within a memory operand, as demonstrated in the following code, generates an error.

**asm**

```
mov eax, DWORD PTR [rcx*2+r10d]
mov eax, DWORD PTR [ecx*2+r10+0100h]
```

## See also

Microsoft Macro Assembler Reference

ⓘ **Note:** The author created this article with assistance from AI. Learn more

---

Last updated on 05/12/2026

# MASM instruction format

Article • 01/13/2022

## Syntax

Instructions are written in source code according to this syntax:

> *prefix mnemonic operand-list*

For information on instruction definitions, options, and encoding, see the Processor Manufacturer Programming Manuals. Some instructions and instruction options may not be supported by the Microsoft Macro Assembler.

## Prefix

You can prefix some instructions with keywords that set options for how the instruction is encoded. The **`REP`**, **`REPE`**, **`REPZ`**, **`REPNE`**, and **`REPNZ`** keywords are used with string instructions to do `memcpy` or `strlen` kinds of operations in a single instruction. The **`LOCK`** keyword makes certain operations on memory operands atomic. You can combine it with the **`XACQUIRE`** and **`XRELEASE`** keywords to do Hardware Lock Elision (HLE) on supported processors, which allows greater transactional parallelism in certain cases.

The remaining prefixes control how AVX instructions are encoded. AVX instructions are encoded using a **`VEX`** prefix, which appears before the opcode. It takes the place of certain byte instruction prefixes and opcode lead-in bytes. Many AVX instructions are also AVX-512 instructions, which are encoded using an **`EVEX`** prefix that supports more options. MASM tries to encode instructions as compactly as possible, but these keywords allow more control over which encoding to use with a particular instruction. They're also used to force generation of AVX instruction forms that were introduced after the corresponding AVX-512 instruction. For example, `vex vpdpbusd` specifies the AVX-VNNI form of the `VPDPBUSD` instructions rather than the AVX512-VNNI form. When an AVX instruction appears without an explicit prefix keyword, the encoding chosen depends on the current AVX encoding setting. The OPTION AVXENCODING directive lets you change this setting.

The **`VEX2`**, **`VEX3`**, **`VEX`**, and **`EVEX`** options are available in Visual Studio 2019 version 16.7 and later.

| Keyword | Usage |
| --- | --- |
| `REP` | Repeat the string operation by the count in (E)CX. |
| `REPE`<br>`REPZ` | Repeat the string operation while the comparison is equal, limited by the count in (E)CX. |
| `REPNE`<br>`REPNZ` | Repeat the string operation while the comparison is not-equal, limited by the count in (E)CX. |
| `LOCK` | Perform the operation atomically on a memory operand. |
| `XACQUIRE` | Begin an HLE transaction, most often used with `LOCK` prefix. |
| `XRELEASE` | Complete an HLE transaction, most often used with `LOCK` prefix. |
| `VEX` | Encode an AVX instruction using a `VEX` prefix. |
| `VEX2` | Encode an AVX instruction using a 2-byte `VEX` prefix. |
| `VEX3` | Encode an AVX instruction using a 3-byte `VEX` prefix. |
| `EVEX` | Encode an AVX instruction using an `EVEX` prefix. |

# Mnemonic

The mnemonic identifies a particular instruction, which determines the prefixes and operand patterns that are allowed.

# Operand list

Most instructions use an operand list to specify the explicit source and destination operands to the instruction. The operand list may contain memory references, registers, and constant values. Each instruction allows only certain types of operands to appear at each position in the operand list. Except for the `MOVS` and `CMPS` instructions, only one of the operands may be a memory reference; all other operands must be register references or constants.

# AVX-512 options

Some AVX-512 instructions allow more options to be specified. These options are: masking, zero-masking, embedded broadcast, embedded rounding, and exception suppression.

*Masking* allows an operation to be applied only to selected elements of a vector. This option is controlled by placing a mask register from `{k1}` to `{k7}` after the destination operand. If the mask register is followed by `{z}`, all non-selected elements of the destination are set to zero. This alternative is known as *zero-masking*.

*Embedded Broadcast* allows a scalar value in memory to be applied to all elements of a vector. This option is enabled by adding the element size and the keyword `BCST` to the memory operand, which is similar to the use of `PTR` for normal memory references.

*Embedded Rounding* controls the rounding mode for an individual floating-point instruction, without having to set and reset the global rounding mode. It's enabled by following the instruction with the rounding mode enclosed in braces. When enabled, it also suppresses all exceptions for only that instruction. Floating-point instructions that don't round can also suppress all exceptions using a similar option.

asm

```
; Examples of AVX-512 options
vaddps xmm1 {k1}, xmm2, xmm3            ; merge-masking
vsubps ymm0 {k4}{z}, ymm1, ymm2         ; zero-masking
vmulps zmm0, zmm1, dword bcst scalar    ; embedded broadcast
vdivps zmm0, zmm1, zmm2 {rz-sae}        ; embedded rounding
vmaxss xmm1, xmm2, xmm3 {sae}           ; suppress all exceptions
```

## Rounding modes

| Mode | Effect |
|---|---|
| `rn-sae` | Round to nearest, ties to even, suppress all exceptions. |
| `rz-sae` | Round toward zero, suppress all exceptions. |
| `rd-sae` | Round down (toward negative infinity), suppress all exceptions. |
| `ru-sae` | Round up (toward positive infinity), suppress all exceptions. |
| `sae` | Suppress all exceptions (no rounding needed). |

# See also

Microsoft Macro Assembler Reference
Processor Manufacturer Programming Manuals

# ML and ML64 command-line reference

Assembles and links one or more assembly-language source files. The command-line options are case-sensitive.

For more information on ml64.exe, see MASM for x64 (ml64.exe).

## Syntax

> **ML** [*options*] *filename* [ [*options*] *filename*]
>
> **ML64** [*options*] *filename* [ [*options*] *filename*] ... [**/link** *link_options*]

## Parameters

*options*
The options listed in the following table:

Expand table

| Option | Action |
|---|---|
| **/AT** | Enables tiny-memory-model support. Enables error messages for code constructs that violate the requirements for *.com* format files. This option isn't equivalent to the .MODEL **TINY** directive.<br>Not available in ml64.exe. |
| **/Bl** *filename* | Selects an alternate linker in *filename*. |
| **/c** | Assembles only. Does no linking. |
| **/coff** | Generates common object file format (COFF) type of object module. Required for Win32 assembly language development.<br>Not available in ml64.exe. |
| **/Cp** | Preserves case of all user identifiers. |
| **/Cu** | Maps all identifiers to upper case (default).<br>Not available in ml64.exe. |
| **/Cx** | Preserves case in public and extern symbols. |
| **/D** *symbol*[=*value*] | Defines a text macro with the given name *symbol*. If *value* is missing, it's blank. Multiple tokens separated by spaces must be enclosed in quotation marks. |

| Option | Action |
| --- | --- |
| `/EP` | Generates a preprocessed source listing (sent to `STDOUT`). See `/Sf`. |
| `/ERRORREPORT` [ `NONE` \| `PROMPT` \| `QUEUE` \| `SEND` ] | Deprecated. Error reporting is controlled by Windows Error Reporting (WER) settings. |
| `/F` *hexnum* | Sets stack size to *hexnum* bytes (the same as `/link /STACK:<number>`). The value must be expressed in hexadecimal notation. There must be a space between `/F` and *hexnum*. |
| `/Fe` *filename* | Names the executable file. |
| `/Fl`[*filename*] | Generates an assembled code listing. See **/Sf**. |
| `/Fm`[*filename*] | Creates a linker map file. |
| `/Fo` *filename* | Names an object file. For more information, see Remarks. |
| `/FPi` | Generates emulator fix-ups for floating-point arithmetic (mixed language only).<br>Not available in ml64.exe. |
| `/Fr`[*filename*] | Generates a source browser *.sbr* file. |
| `/FR`[*filename*] | Generates an extended form of a source browser *.sbr* file. |
| `/Gc` | Specifies use of FORTRAN- or Pascal-style conventions for function calls and names. Same as `OPTION LANGUAGE:PASCAL`.<br>Not available in ml64.exe. |
| `/Gd` | Specifies use of C-style conventions for function calls and names. Same as `OPTION LANGUAGE:C`.<br>Not available in ml64.exe. |
| `/Gz` | Specifies use of `__stdcall` conventions for function calls and names. Same as `OPTION LANGUAGE:STDCALL`.<br>Not available in ml64.exe. |
| `/H` *number* | Restricts external names to *number* significant characters. The default is 31 characters.<br>Not available in ml64.exe. |
| `/help` | Displays a summary of ML command-line syntax and options. |
| `/I` *pathname* | Sets path for include file. A maximum of 10 `/I` options is allowed. |
| `/nologo` | Suppresses messages for successful assembly. |
| `/omf` | Generates object module file format (OMF) type of object module. `/omf` implies `/c`.<br>ML.exe doesn't support linking OMF objects. |

| Option | Action |
| --- | --- |
| | Not available in ml64.exe. |
| `/quiet` | Suppresses 'Assembling' message. Available in Visual Studio 17.6 and later. |
| `/Sa` | Turns on listing of all available information. |
| `/safeseh` | Marks the object file: either it contains no exception handlers, or it contains exception handlers that are all declared with .SAFESEH.<br>Not available in ml64.exe. |
| `/Sf` | Adds the first-pass listing to the listing file. |
| `/Sl` *`width`* | Sets the line width of source listing in characters per line to *`width`*. Range is 60-255 or 0. Default is 0. Same as PAGE *`width`*. |
| `/Sn` | Turns off symbol table when a listing is produced. |
| `/Sp` *`length`* | Sets the page length of source listing in lines per page to *`length`*. Range is 10-255 or 0. Default is 0. Same as PAGE *`length`*. |
| `/Ss` *`text`* | Specifies text for source listing. Same as SUBTITLE text. |
| `/St` *`text`* | Specifies title for source listing. Same as TITLE text. |
| `/Sx` | Turns on false conditionals in listing. |
| `/Ta` *`filename`* | Assembles source file whose name doesn't end with the *`.asm`* extension. |
| `/unwindv3` | Enables experimental Unwind Version 3 unwind directive support. When specified, V3 unwind directives must appear *before* their associated instruction (the reverse of V1 behavior), epilogue recording via .BEGINEPILOG/.ENDEPILOG is required, and the predefined macro @UnwindVersion returns `3`. For more information, see MASM for x64 (ml64.exe).<br>Only available in ml64.exe. |
| `/w` | Same as `/W0 /WX`. |
| `/W` *`level`* | Sets the warning level, where *`level`* = 0, 1, 2, or 3. |
| `/WX` | If warnings are generated, returns an error code. |
| `/X` | Ignore `INCLUDE` environment path. |
| `/Zd` | Generates line-number information in object file. |
| `/Zf` | Makes all symbols public. |
| `/ZH:MD5` | Use MD5 for checksum in debug info. |
| `/ZH:SHA_256` | Use SHA256 for checksum in debug info (default in Visual Studio 2022 version 17.0 |

| Option | Action |
| --- | --- |
| | and later). |
| `/ZH:SHA384` | Use SHA384 for checksum in debug info.[18.6.0] |
| `/ZH:SHA512` | Use SHA512 for checksum in debug info.[18.6.0] |
| `/Zi` | Generates CodeView information in object file. |
| `/Zm` | Enables `M510` option for maximum compatibility with MASM 5.1.<br>Not available in ml64.exe. |
| `/Zp`[[ *`alignment`* ]] | Packs structures on the specified byte boundary. The *`alignment`* can be 1, 2, 4, 8, or 16. |
| `/Zs` | Performs a syntax check only. |
| `/?` | Displays a summary of ML command-line syntax and options. |

[18.6.0] This option is available starting in Visual Studio 2026 version 18.6.0 and MSVC version 14.51.

*`filename`*

The name of the file.

*`link_options`*

The link options. For more information, see Linker options.

# Remarks

Some command-line options to ML and ML64 are placement-sensitive. For example, because ML and ML64 can accept several `/c` options, any corresponding `/Fo` options must be specified before `/c`. The following command-line example illustrates an object file specification for each assembly file specification:

**Windows Command Prompt**

```
ml.exe /Fo a1.obj /c a.asm /Fo b1.obj /c b.asm
```

# Environment Variables

Expand table

| Variable | Description |
| --- | --- |
| `INCLUDE` | Specifies search path for include files. |
| `ML` | Specifies default command-line options. |
| `TMP` | Specifies path for temporary files. |

## See also

ML error messages

Microsoft Macro Assembler reference

ⓘ **Note:** The author created this article with assistance from AI. Learn more

Last updated on 05/25/2026

# Directives Reference

## x64

.ALLOCSTACK
.ENDPROLOG
PROC

.PUSHFRAME
.PUSHREG
.SAVEREG

.SAVEXMM128
.SETFRAME

## x64 Unwind Version 3 (experimental)

The following directives are used with `ml64.exe` when the `/unwindv3` option is enabled. Unwind Version 3 is experimental and is subject to change.

**Prologue directives**

.ALLOCSTACK
.ENDPROLOG
.PUSH2REG

.PUSHFRAME
.PUSHREG
.SAVEREG

.SAVEXMM128
.SETFRAME

**Epilogue directives**

.BEGINEPILOG
.ENDEPILOG
.FREESTACK

.POP2REG

.POPFRAME

.POPREG

.RESTOREREG

.RESTOREXMM128

.UNSETFRAME

## Code Labels

ALIGN

EVEN

LABEL

ORG

## Conditional Assembly

ELSE

ELSEIF

ELSEIF2

IF

IF2

IFB

IFNB

IFDEF

IFNDEF

IFDIF

IFDIFI

IFE

IFIDN

IFIDNI

## Conditional Control Flow

.BREAK

.CONTINUE

.ELSE

.ELSEIF

.ENDIF

.ENDW

.IF

.REPEAT

.UNTIL

.UNTILCXZ

.WHILE

## Conditional Error

.ERR

.ERR2

.ERRB

.ERRDEF

.ERRDIF

.ERRDIFI

.ERRE

.ERRIDN

.ERRIDNI

.ERRNB

.ERRNDEF

.ERRNZ

## Data Allocation

ALIGN

BYTE

SBYTE

DWORD

SDWORD
EVEN

FWORD
LABEL
ORG
QWORD
REAL4

REAL8
REAL10
TBYTE
WORD
SWORD

## Equates

=

EQU

TEXTEQU

## Listing Control

.CREF
.LIST
.LISTALL
.LISTIF
.LISTMACRO

.LISTMACROALL
.NOCREF
.NOLIST
.NOLISTIF
.NOLISTMACRO

PAGE
SUBTITLE

.TFCOND
TITLE

## Macros

ENDM
EXITM

GOTO
LOCAL

MACRO
PURGE

## Miscellaneous

ALIAS
ASSUME
COMMENT
ECHO
END
.FPO

INCLUDE
INCLUDELIB
MMWORD
OPTION
POPCONTEXT

PUSHCONTEXT
.RADIX
.SAFESEH
XMMWORD
YMMWORD

## Procedures

ENDP
INVOKE

PROC

PROTO

## Processor

.386
.386P
.387
.486

.486P
.586
.586P
.686

.686P
.K3D
.MMX
.XMM

## Repeat Blocks

ENDM
FOR

FORC
GOTO

REPEAT
WHILE

## Scope

COMM
EXTERN

EXTERNDEF
INCLUDELIB

PUBLIC

## Segment

.ALPHA
ASSUME
.DOSSEG

END
ENDS
GROUP

SEGMENT
.SEQ

## Simplified Segment

.CODE
.CONST
.DATA
.DATA?

.DOSSEG
.EXIT
.FARDATA
.FARDATA?

.MODEL
.STACK
.STARTUP

## String

CATSTR

INSTR

SIZESTR

SUBSTR

## Structure and Record

ENDS

RECORD

STRUCT

TYPEDEF

UNION

## See also

Microsoft Macro Assembler reference

MASM BNF Grammar

ⓘ **Note:** The author created this article with assistance from AI. Learn more

---

Last updated on 05/12/2026

# =

Article • 08/03/2021

Assigns the numeric value of *expression* to *name*.

## Syntax

> *name* = *expression*

## Remarks

The symbol can be redefined later.

## See also

Directives reference
MASM BNF Grammar

# .386 (32-bit MASM)

Article • 08/03/2021

Enables assembly of nonprivileged instructions for the 80386 processor; disables assembly of instructions introduced with later processors. (32-bit MASM only.)

## Syntax

```
.386
```

## Remarks

Also enables 80387 instructions.

## See also

Directives Reference
MASM BNF Grammar

# .386P (32-bit MASM)

Article • 08/03/2021

Enables assembly of all instructions (including privileged) for the 80386 processor; disables assembly of instructions introduced with later processors. (32-bit MASM only.)

## Syntax

```
.386P
```

## Remarks

Also enables 80387 instructions.

## See also

Directives Reference
MASM BNF Grammar

# .387 (32-bit MASM)

Article • 08/03/2021

Enables assembly of instructions for the 80387 coprocessor. (32-bit MASM only.)

## Syntax

```
.387
```

## See also

Directives Reference
MASM BNF Grammar

# .486 (32-bit MASM)

Article • 08/03/2021

Enables assembly of nonprivileged instructions for the 80486 processor. (32-bit MASM only.)

## Syntax

```
.486
```

## See also

Directives Reference
MASM BNF Grammar

# .486P (32-bit MASM)

Article • 08/03/2021

Enables assembly of all instructions (including privileged) for the 80486 processor. (32-bit MASM only.)

## Syntax

> **.486P**

## See also

Directives Reference
MASM BNF Grammar

# .586 (32-bit MASM)

Article • 08/03/2021

Enables assembly of nonprivileged instructions for the Pentium processor. (32-bit MASM only.)

## Syntax

```
.586
```

## See also

Directives Reference
MASM BNF Grammar

# .586P (32-bit MASM)

Article • 08/03/2021

Enables assembly of all instructions (including privileged) for the Pentium processor. (32-bit MASM only.)

## Syntax

> **.586P**

## See also

Directives Reference
MASM BNF Grammar

# .686 (32-bit MASM)

Article • 08/03/2021

Enables assembly of nonprivileged instructions for the Pentium Pro processor. (32-bit MASM only.)

## Syntax

```
.686
```

## See also

Directives Reference
MASM BNF Grammar

# .686P (32-bit MASM)

Article • 08/03/2021

Enables assembly of all instructions (including privileged) for the Pentium Pro processor. (32-bit MASM only.)

## Syntax

> **.686P**

## See also

Directives Reference
MASM BNF Grammar

# ALIAS

Article • 08/03/2021

The **ALIAS** directive creates an alternate name for a function. This lets you create multiple names for a function, or create libraries that allow the linker (LINK.exe) to map an old function to a new function.

## Syntax

> **ALIAS** <*alias*> = <*actual-name*>

### Parameters

*actual-name*
The actual name of the function or procedure. The angle brackets are required.

*alias*
The alternate or alias name. The angle brackets are required.

## See also

Directives Reference
MASM BNF Grammar

# ALIGN

Article • 08/03/2021

The **ALIGN** directive aligns the next data element or instruction on an address that is a multiple of its parameter. The parameter must be a power of 2 (for example, 1, 2, 4, and so on) that is less than or equal to the segment alignment.

## Syntax

> **ALIGN** [[*constantExpression*]]

## Remarks

The **ALIGN** directive allows you to specify the beginning offset of a data element or an instruction. Aligned data can improve performance, at the expense of wasted space between data elements. Large performance improvements can be seen when data accesses are on boundaries that fit within cache lines. Accesses on natural boundaries for native types means less time spent in internal hardware realignment microcode.

The need for aligned instructions is rare on modern processors that use a flat addressing model, but may be required for jump targets in older code for other addressing models.

When data is aligned, the skipped space is padded with zeroes. When instructions are aligned, the skipped space is filled with appropriately-sized NOP instructions.

## See also

EVEN
Directives reference
MASM BNF Grammar

# .ALLOCSTACK

`.ALLOCSTACK` generates a **UWOP_ALLOC_SMALL** or a **UWOP_ALLOC_LARGE** with the specified size for the current offset in the prologue.

## Syntax

> `.ALLOCSTACK` *size*

## Remarks

Microsoft Assembler (MASM) chooses the most efficient encoding for a given size.

`.ALLOCSTACK` allows `ml64.exe` users to specify how a frame function unwinds and is only allowed within the prologue, which extends from the PROC `FRAME` declaration to the .ENDPROLOG directive.

- These directives generate unwind metadata (`.xdata` and `.pdata` sections) but don't produce executable code.
- Precede `.ALLOCSTACK` with the instructions that actually implement the actions to be unwound. Wrap both the unwind directives and the code they're meant to unwind in a macro to ensure agreement.

The *size* operand must be a multiple of 8.

The epilogue counterpart is .FREESTACK.

## Example: specify an unwind version 3 unwind/exception handler

The following sample shows how to specify an unwind/exception handler:

asm

```
; ml64 ex3.asm /link /entry:Example3 /SUBSYSTEM:Console
text SEGMENT
PUBLIC Example3
PUBLIC Example3_UW
Example3_UW PROC
   ; exception/unwind handler body
```

```
   ret 0

Example3_UW ENDP

Example3 PROC FRAME : Example3_UW

   sub rsp, 16
.allocstack 16

.endprolog

   ; function body
    add rsp, 16
   ret 0

Example3 ENDP
text ENDS
END
```

# Unwind Version 3 behavior

> **Important**
>
> Unwind Version 3 support is experimental and is subject to change. Enable it by using `ml64.exe /unwindv3`.

`.ALLOCSTACK` generates a `WOD_ALLOC_SMALL`, `WOD_ALLOC_LARGE`, or `WOD_ALLOC_HUGE` Unwind Version 3 unwind code entry with the specified size for the current offset in the prolog.

`.ALLOCSTACK` must appear **before** the `sub rsp, N` instruction it describes. This requirement is the opposite of Version 1, where the directive follows the instruction.

MASM emits one of three unwind codes depending on the allocation size:

Expand table

| Unwind code | Condition |
|---|---|
| `WOD_ALLOC_SMALL` | *size* ≤ 128 bytes |
| `WOD_ALLOC_LARGE` | *size* ≤ 32 KB |
| `WOD_ALLOC_HUGE` | *size* > 32 KB |

Version 1 generates only `UWOP_ALLOC_SMALL` or `UWOP_ALLOC_LARGE`. Version 3 adds a third variant, `WOD_ALLOC_HUGE`, for allocations larger than 32 KB.

> **Note**
>
> In Unwind Version 1, the directive appears after the corresponding instruction. In Unwind Version 3, the directive appears **before** the instruction.

## Example for unwindv3

asm

```
; ml64 ex3.asm /unwindv3 /link /entry:Example3 /SUBSYSTEM:Console
text SEGMENT
PUBLIC Example3
PUBLIC Example3_UW
Example3_UW PROC
   ; exception/unwind handler body

   ret 0

Example3_UW ENDP

Example3 PROC FRAME : Example3_UW

.allocstack 16
   sub rsp, 16

.endprolog

   ; function body
    add rsp, 16
   ret 0

Example3 ENDP
text ENDS
END
```

## See also

x64 Unwind Version 3 (experimental)

Directives Reference

MASM BNF Grammar

MASM for x64 (ml64.exe)

**Note:** The author created this article with assistance from AI. Learn more

Last updated on 05/12/2026

# .ALPHA (32-bit MASM)

Article • 08/03/2021

Orders segments alphabetically. (32-bit MASM only.)

## Syntax

> **.ALPHA**

## See also

Directives Reference
MASM BNF Grammar

# ASSUME

Article • 08/03/2021

Enables error checking for register values. (32-bit MASM only.)

## Syntax

> **ASSUME** *segregister*:*name* [[, *segregister*:*name*...]]
> **ASSUME** *dataregister*:*type* [[, *dataregister*:*type*...]]
> **ASSUME** *register*:**ERROR** [[, *register*:**ERROR**...]]
> **ASSUME** [[*register*:]]**NOTHING** [[, *register*:**NOTHING**...]]

## Remarks

After an **ASSUME** is put into effect, the assembler watches for changes to the values of the given registers. **ERROR** generates an error if the register is used. **NOTHING** removes register error checking. You can combine different kinds of assumptions in one statement.

## See also

Directives Reference
MASM BNF Grammar

# .BEGINEPILOG

Marks the start of an epilogue unwind region. This directive doesn't emit an Unwind Operation Descriptors (WOD) code.

## Syntax

```
.BEGINEPILOG
```

## Remarks

> **Important**
>
> This directive is experimental and is subject to change. Enable Unwind Version 3 by using `ml64.exe /unwindv3`.

`.BEGINEPILOG` is only valid in Unwind Version 3.

- It marks the start of an epilogue region where epilogue unwind operations are recorded. Pair `.BEGINEPILOG` with a closing .ENDEPILOG or no epilog unwind codes are emitted.
- In Unwind Version 3, epilog directives are mandatory for epilog unwind code generation.
- Epilogues must contain at least one directive.
- These directives generate unwind metadata (`.xdata` and `.pdata` sections) but don't produce executable code.

> **Note**
>
> In Unwind Version 1, the directive appears after the corresponding instruction. In Unwind Version 3, the directive appears **before** the instruction.

## Example for unwindv3

asm

```
; ml64 /unwindv3 ex1.asm /link /entry:Example1 /SUBSYSTEM:CONSOLE
_text SEGMENT
Example1 PROC FRAME
```

```
    .allocstack 16
    sub rsp, 16
.endprolog
    ; function body
    .beginepilog
    .freestack 16
    add rsp, 16
    .endepilog
    ret
Example1 ENDP
_text ENDS
END
```

## See also

x64 Unwind Version 3 (experimental)

Directives Reference

.ENDEPILOG

.ENDPROLOG

MASM BNF Grammar

ⓘ **Note:** The author created this article with assistance from AI. Learn more

---

Last updated on 05/12/2026

# .BREAK (32-bit MASM)

Article • 08/03/2021

Generates code to terminate a .WHILE or .REPEAT block if *condition* is true. (32-bit MASM only.)

## Syntax

> **.BREAK** [[**.IF** *condition*]]

## See also

Directives Reference
MASM BNF Grammar

# BYTE

Article • 08/03/2021

Allocates and optionally initializes a byte of storage for each *initializer*.

## Syntax

> [[*name*]] **BYTE** *initializer* [[**,** *initializer* ...]]

## Remarks

Can also be used as a type specifier anywhere a type is legal.

## See also

Directives Reference
DB
SBYTE
MASM BNF Grammar

# CATSTR

Article • 08/03/2021

Concatenates text items.

## Syntax

> *name* **CATSTR** [[*textitem1* [[**,** *textitem2* ...]]]]

## Remarks

Each text item can be a literal string, a constant preceded by a %, or the string returned by a macro function. **CATSTR** is a synonym for TEXTEQU.

## See also

Directives Reference
MASM BNF Grammar

# .CODE

Indicates the start of a code segment.

When using 32-bit MASM, this should be used along with .MODEL.

## Syntax

> **.CODE** ⟦*name*⟧
> ⟦ *segmentItem* ⟧...
> ⟦ *codesegmentnameId* **ENDS**;;⟧

### Parameters

*name*
Optional parameter that specifies the name of the code segment. The default name is **_TEXT** for tiny, small, compact, and flat models. The default name is *modulename*_TEXT for other models.

## See also

Directives Reference
.DATA
MASM BNF Grammar

---

Last updated on 03/18/2025

# COMM

Article • 08/03/2021

Creates a communal variable with the attributes specified in *definition*.

## Syntax

> **COMM** *definition* [[, *definition* ...]]

## Remarks

Communal variables are allocated by the linker, and can't be initialized. This means that you can't depend on the location or sequence of such variables.

Each *definition* has the following form:

[[*language-type*]] [[**NEAR** | **FAR**]] *label*:*type*[[:*count*]]

The *language-type*, **NEAR**, and **FAR** arguments are valid only in 32-bit MASM.

The optional *language-type* sets the naming conventions for the name that follows. It overrides any language specified by the **.MODEL** directive. The optional **NEAR** or **FAR** override the current memory model. The *label* is the name of the variable. The *type* can be any type specifier (BYTE, WORD, and so on) or an integer specifying the number of bytes. The optional *count* specifies the number of elements in the declared data object. The default *count* is one.

## Example

This example creates an array of 512 BYTE elements:

```asm
COMM FAR ByteArray:BYTE:512
```

## See also

Directives reference
MASM BNF Grammar

# COMMENT

Article • 08/03/2021

Treats all *text* between or on the same line as the delimiters as a comment.

## Syntax

> **COMMENT** *delimiter* [[*text*]]
> [[*text*]]
> [[*text*]] *delimiter* [[*text*]]

## See also

Directives Reference
MASM BNF Grammar

# .CONST (32-bit MASM)

Article • 08/03/2021

When used with .MODEL, starts a constant data segment (with segment name **CONST**).

## Syntax

> **.CONST**
> [[ *segmentItem* ]]...

## Remarks

This segment has the read-only attribute.

## See also

Directives Reference
MASM BNF Grammar

# .CONTINUE (32-bit MASM)

Article • 08/03/2021

Generates code to jump to the top of a .WHILE or .REPEAT block if *condition* is true. (32-bit MASM only.)

## Syntax

> **.CONTINUE** ⟦**.IF** *condition*⟧

## See also

Directives Reference
MASM BNF Grammar

# .CREF

Article • 08/03/2021

Enables listing of symbols in the symbol portion of the symbol table and browser file.

## Syntax

> **.CREF**

## See also

Directives Reference
MASM BNF Grammar

# .DATA

Article • 12/06/2024

Indicates the start of a data segment.

When using 32-bit MASM, this starts a near data segment for initialized data (segment name _DATA) and should be used along with .MODEL.

## Syntax

> **.DATA**
> [[ *segmentItem* ]]...

## See also

Directives Reference
.DATA?
.CONST
.FARDATA
.FARDATA?
MASM BNF Grammar

---

## Feedback

**Was this page helpful?** Yes No

Provide product feedback | Get help at Microsoft Q&A

# .DATA?

Article • 08/03/2021

(32-bit MASM only.) When used with .MODEL, starts a near data segment for uninitialized data (segment name _BSS).

## Syntax

> **.DATA?**
> [[ *segmentItem* ]]...

## See also

Directives Reference
MASM BNF Grammar

# DB

Article • 08/03/2021

Allocates and optionally initializes a byte of storage for each *initializer*. **DB** is a synonym of BYTE.

## Syntax

> [[*name*]] **DB** *initializer* [[**,** *initializer* ...]]

## Remarks

Can also be used as a type specifier anywhere a type is legal.

## See also

Directives Reference
DB
SBYTE
MASM BNF Grammar

# DD

Article • 08/03/2021

Allocates and optionally initializes a double word (4 bytes) of storage for each *initializer*. **DD** is a synonym of DWORD.

## Syntax

> [[*name*]] **DD** *initializer* [[, *initializer* ...]]

## Remarks

Can also be used as a type specifier anywhere a type is legal.

## See also

Directives Reference
MASM BNF Grammar

# DF

Article • 08/03/2021

Allocates and optionally initializes 6 bytes of storage for each *initializer*. **DF** is a synonym of FWORD.

## Syntax

> [[*name*]] **DF** *initializer* [[**,** *initializer* ...]]

## Remarks

Also can be used as a type specifier anywhere a type is legal.

## See also

Directives Reference
MASM BNF Grammar

# .DOSSEG (32-bit MASM)

Article • 08/03/2021

Orders the segments according to the MS-DOS segment convention: CODE first, then segments not in DGROUP, and then segments in DGROUP. (32-bit MASM only.)

## Syntax

> **.DOSSEG**

## Remarks

The segments in DGROUP follow this order: segments not in BSS or STACK, then BSS segments, and finally STACK segments. Primarily used for ensuring CodeView support in MASM stand-alone programs. Same as DOSSEG.

## See also

Directives Reference
MASM BNF Grammar

# DOSSEG

Article • 08/03/2021

(32-bit MASM only.) Identical to .DOSSEG, which is the preferred form.

## Syntax

> **DOSSEG**

## See also

Directives Reference
MASM BNF Grammar

# DQ

Article • 08/03/2021

Allocates and optionally initializes 8 bytes of storage for each *initializer*. Also can be used as a type specifier anywhere a type is legal. **DQ** is a synonym of QWORD.

## Syntax

> [[*name*]] **DQ** *initializer* [[, *initializer* ...]]

## See also

Directives Reference
QWORD
MASM BNF Grammar

# DT

Article • 08/03/2021

Allocates and optionally initializes 10 bytes of storage for each *initializer*. Can also be used as a type specifier anywhere a type is legal. **DT** is a synonym of TBYTE.

## Syntax

> [[*name*]] **DT** *initializer* [[**,** *initializer* ...]]

## See also

Directives reference
MASM BNF Grammar

# DW

Article • 08/03/2021

Allocates and optionally initializes a word (2 bytes) of storage for each *initializer*. Can also be used as a type specifier anywhere a type is legal. **DW** is a synonym of WORD.

## Syntax

> [[*name*]] **DW** *initializer* [[**,** *initializer* ...]]

## See also

Directives Reference
SWORD
MASM BNF Grammar

# DWORD

Article • 08/03/2021

Allocates and optionally initializes a double word (4 bytes) of storage for each *initializer*. **DWORD** is a synonym of DD.

## Syntax

> [[*name*]] **DWORD** *initializer* [[, *initializer* ...]]

## Remarks

Can also be used as a type specifier anywhere a type is legal.

## See also

Directives Reference
SDWORD
DD
MASM BNF Grammar

# ECHO

Article • 08/03/2021

Displays *message* to the standard output device (by default, the screen).

## Syntax

> **ECHO** *message*

## Remarks

Same as %OUT.

## See also

Directives reference
MASM BNF Grammar

# .ELSE (32-bit MASM)

Article • 08/03/2021

## Syntax

> **.ELSE**

## Remarks

(32-bit MASM only.) See .IF.

## See also

Directives Reference
MASM BNF Grammar

# ELSE (MASM)

Article • 08/03/2021

Marks the beginning of an alternate block within a conditional block.

## Syntax

```
ELSE
```

## Remarks

See IF.

## See also

Directives reference
MASM BNF Grammar

# ELSEIF

Article • 08/03/2021

Combines ELSE and IF into one statement.

## Syntax

> **ELSEIF** *constantExpression*
> *statements*
> [[**ELSE**
> *else-statements*]]
> **ENDIF**

## Remarks

See IF for more information.

## See also

Directives reference
MASM BNF Grammar

# ELSEIF2

Article • 08/03/2021

ELSEIF block evaluated on every assembly pass if **OPTION:SETIF2** is **TRUE**.

## Syntax

> **ELSEIF2**

## Remarks

For more information about 2-pass behavior in MASM 5.1 vs MASM 6.1, see IF1 and IF2.

## See also

Directives reference
MASM BNF Grammar

# END

Article • 08/03/2021

Marks the end of a module and, optionally, sets the program entry point to *procId*.

## Syntax

> **END** ⟦*procId*⟧

## Remarks

The *procId* argument is valid in 32-bit MASM only.

## See also

Directives reference
MASM BNF Grammar

# .ENDIF (32-bit MASM)

Article • 08/03/2021

## Syntax

> **.ENDIF**

## Remarks

(32-bit MASM only.) See .IF.

## See also

Directives Reference
MASM BNF Grammar

# .ENDEPILOG

Marks the end of an epilogue unwind region. This directive doesn't emit a Windows unwind data (WOD) unwind code.

## Syntax

> **.ENDEPILOG**

## Remarks

> **ⓘ Important**
>
> This directive is experimental and is subject to change. Enable Unwind Version 3 by using `ml64.exe /unwindv3`.

**.ENDEPILOG** is only valid in Unwind Version 3.

- It marks the end of an epilogue region started by .BEGINEPILOG.
- These directives generate unwind metadata (`.xdata` and `.pdata` sections) but don't produce executable code.
- Epilogues must contain at least one directive.
- Epilogue directives are mandatory for epilogue unwind code generation. No epilogue unwind codes are emitted without a `.BEGINEPILOG`/`.ENDEPILOG` pair.
- It's an error to use epilogue directives outside of the region between .BEGINEPILOG and **.ENDEPILOG**.

> **ⓘ Note**
>
> In Unwind Version 1, the directive appears after the corresponding instruction. In Unwind Version 3, the directive appears **before** the instruction.

## Example

```asm
```

```
; ml64 /unwindv3 ex1.asm /link /entry:Example1 /SUBSYSTEM:CONSOLE
_text SEGMENT
Example1 PROC FRAME
   .allocstack 16
   sub rsp, 16
.endprolog
   ; function body
   .beginepilog
   .freestack 16
   add rsp, 16
   .endepilog
   ret
Example1 ENDP
_text ENDS
END
```

## See also

x64 Unwind Version 3 (experimental)

Directives Reference

.BEGINEPILOG

.ENDPROLOG

MASM BNF Grammar

ⓘ **Note:** The author created this article with assistance from AI. Learn more

---

Last updated on 05/12/2026

# ENDM

Article • 08/03/2021

Terminates a macro or repeat block.

## Syntax

```
ENDM
```

## Remarks

See MACRO, FOR, FORC, REPEAT, or WHILE.

## See also

Directives reference
MASM BNF Grammar

# ENDP

Article • 08/03/2021

Marks the end of procedure *name* previously begun with **PROC**.

## Syntax

> *name* **ENDP**

## Remarks

See PROC.

## See also

Directives reference
MASM BNF Grammar

# .ENDPROLOG

Marks the end of the prologue declarations.

## Syntax

```
.ENDPROLOG
```

## Remarks

It's an error to use any of the prologue declarations outside of the region between PROC `FRAME` and **.ENDPROLOG**.

For more information, see MASM for x64 (ml64.exe).

## Unwind Version 3 behavior

> **Important**
>
> Unwind Version 3 support is experimental and is subject to change. Enable it with `ml64.exe /unwindv3`.

`.ENDPROLOG` is also used in Unwind Version 3 and continues to mark the end of the function prologue. It doesn't generate a Windows unwind data (WOD) unwind code entry.

It's an error to use any prologue directive outside the region between `PROC FRAME` and `.ENDPROLOG`.

## See also

x64 Unwind Version 3 (experimental)
Directives Reference
MASM BNF Grammar

**Note:** The author created this article with assistance from AI. Learn more

Last updated on 05/12/2026

# ENDS

Article • 08/03/2021

Marks the end of segment, structure, or union *name* previously begun with SEGMENT, STRUCT, UNION, or a simplified segment directive.

## Syntax

> *name* **ENDS**

## See also

Directives reference
MASM BNF Grammar

# .ENDW (32-bit MASM)

Article • 08/03/2021

## Syntax

> **.ENDW**

## Remarks

(32-bit MASM only.) See .WHILE.

## See also

Directives Reference
MASM BNF Grammar

# EQU

Article • 08/03/2021

The first directive assigns numeric value of *expression* to *name*.

## Syntax

> *name* **EQU** *expression*
>
> *name* **EQU** <*text*>

## Remarks

The *name* cannot be redefined later.

The second directive assigns specified *text* to *name*. The *name* can be assigned a different *text* later. See TEXTEQU.

## See also

Directives reference
MASM BNF Grammar

# .ERR

Article • 08/03/2021

Generates an error.

## Syntax

> **.ERR** [[*message*]]

## See also

Directives Reference
.ERRNB
.ERR2
.ERRDEF
.ERRNZ
MASM BNF Grammar

# .ERR2

Article • 08/03/2021

.ERR block evaluated on every assembly pass if **OPTION:SETIF2** is **TRUE.**

## Syntax

> **.ERR2** ⟦*message*⟧

## Remarks

For more information about 2-pass behavior in MASM 5.1 vs MASM 6.1, see IF1 and IF2.

## See also

Directives Reference
MASM BNF Grammar

# .ERRB

Article • 08/03/2021

Generates an error if *textitem* is blank.

## Syntax

> **.ERRB** *textitem* [[, *message*]]

## See also

Directives Reference
MASM BNF Grammar

# .ERRDEF

Article • 08/03/2021

Generates an error if *name* is a previously defined label, variable, or symbol.

## Syntax

> **.ERRDEF** *name* [[**,** *message*]]

## See also

Directives Reference
MASM BNF Grammar

# .ERRDIF, .ERRDIFI

Article • 08/03/2021

Generates an error if the text items are different.

## Syntax

> **.ERRDIF** *textitem1*, *textitem2* [[, *message*]]
> **.ERRDIFI** *textitem1*, *textitem2* [[, *message*]]

## Remarks

If **.ERRDIFI** is given, the comparison is case insensitive.

## See also

Directives reference
MASM BNF Grammar

# .ERRE

Article • 08/03/2021

Generates an error if *expression* is false (0).

## Syntax

> **.ERRE** *expression* [[, *message*]]

## See also

Directives Reference
MASM BNF Grammar

# `.ERRIDN`, `.ERRIDNI`

Article • 12/07/2021

Generates an error if the text items are identical.

## Syntax

> **`.ERRIDN`** *textitem1*, *textitem2* [[, *message*]]
> **`.ERRIDNI`** *textitem1*, *textitem2* [[, *message*]]

## Remarks

If **`.ERRIDNI`** is given, the comparison is case insensitive.

## See also

Directives reference
MASM BNF Grammar

# .ERRNB

Article • 08/03/2021

Generates an error if *textitem* is not blank.

## Syntax

> **.ERRNB** *textitem* [[, *message*]]

## See also

Directives Reference
MASM BNF Grammar

# .ERRNDEF

Article • 08/03/2021

Generates an error if *name* has not been defined.

## Syntax

> **.ERRNDEF** *name* [[, *message*]]

## See also

Directives Reference
MASM BNF Grammar

# .ERRNZ

Article • 08/03/2021

Generates an error if *expression* is true (nonzero).

## Syntax

> `.ERRNZ` *expression* [[`,` *message*]]

## See also

Directives Reference
MASM BNF Grammar

# EVEN

08/22/2025

Aligns the next variable or instruction on an even byte.

## Syntax

```
EVEN
```

## See also

ALIGN
Directives reference
MASM BNF Grammar

# .EXIT (32-bit MASM)

Article • 08/03/2021

Generates termination code. (32-bit MASM only.)

## Syntax

> **.EXIT** [[*expression*]]

## Remarks

Returns optional *expression* to shell.

## See also

Directives reference
MASM BNF Grammar

# EXITM

Article • 08/03/2021

Terminates expansion of the current repeat or macro block and begins assembly of the next statement outside the block.

## Syntax

> **EXITM** ⟦*textitem*⟧

## Remarks

In a macro function, *textitem* is the value returned.

## See also

Directives reference
MASM BNF Grammar

# EXTERN

Article • 01/10/2025

Defines one or more external variables, labels, or symbols called *name* whose type is *type*.

## Syntax

> **EXTERN** ⟦*language-type*⟧ *name* ⟦ (*altid*) ⟧ : *type* ⟦, ⟦*language-type*⟧ *name* ⟦ (*altid*) ⟧ : *type* ...⟧

## Remarks

The *language-type* argument is valid in 32-bit MASM only.

The *type* can be ABS, which imports *name* as a constant. Same as EXTRN.

The *type* can also be PROC, in which case *name* is treated as an external procedure.

## See also

Directives Reference
MASM BNF Grammar

---

## Feedback

**Was this page helpful?** Yes No

Provide product feedback | Get help at Microsoft Q&A

# EXTERNDEF

Article • 08/03/2021

Defines one or more external variables, labels, or symbols called *name* whose type is *type*.

## Syntax

> **EXTERNDEF** ⟦*language-type*⟧ *name*:*type* ⟦**,** ⟦*language-type*⟧ *name*:*type* ...⟧

## Remarks

The *language-type* argument is valid in 32-bit MASM only.

If *name* is defined in the module, it is treated as PUBLIC. If *name* is referenced in the module, it is treated as EXTERN. If *name* is not referenced, it is ignored. The *type* can be ABS, which imports *name* as a constant. Normally used in include files.

## See also

Directives Reference
MASM BNF Grammar

# EXTRN

Article • 08/03/2021

## Syntax

> **EXTRN**

## Remarks

See EXTERN.

## See also

Directives reference
MASM BNF Grammar

# .FARDATA (32-bit MASM)

Article • 08/03/2021

When used with .MODEL, starts a far data segment for initialized data (segment name FAR_DATA or *name*). (32-bit MASM only.)

## Syntax

> **.FARDATA** [[*name*]]

## See also

Directives reference
MASM BNF Grammar

# .FARDATA? (32-bit MASM)

Article • 08/03/2021

When used with .MODEL, starts a far data segment for uninitialized data (segment name FAR_BSS or *name*). (32-bit MASM only.)

## Syntax

> **.FARDATA?** ⟦*name*⟧

## See also

Directives reference
MASM BNF Grammar

# FOR

Article • 08/03/2021

Marks a block that will be repeated once for each *argument*, with the current *argument* replacing *parameter* on each repetition.

## Syntax

> **FOR** *parameter* [[**:REQ** | **:=***default*]] , < *argument* [[**,** *argument* ...]]>
> *statements*
> **ENDM**

## Remarks

Same as IRP.

## See also

Directives reference
MASM BNF Grammar

# FORC

Article • 08/03/2021

Marks a block that will be repeated once for each character in *string*, with the current character replacing *parameter* on each repetition.

## Syntax

> **FORC** *parameter*, *<string>*
> *statements*
> **ENDM**

## Remarks

Same as IRPC.

## See also

Directives reference
MASM BNF Grammar

# .FPO (32-bit MASM)

07/23/2025

The `.FPO` directive controls the emission of debug records to the `.debug$F` segment or section. Use this directive with 32-bit MASM only.

## Syntax

> `.FPO` (*`cdwLocals`*, *`cdwParams`*, *`cbProlog`*, *`cbRegs`*, *`fUseBP`*, *`cbFrame`*)

### Parameters

*`cdwLocals`*

Number of local variables, an unsigned 32-bit value.

*`cdwParams`*

Size of the parameters in DWORDS, an unsigned 16-bit value.

*`cbProlog`*

Number of bytes in the function prolog code, an unsigned 8-bit value.

*`cbRegs`*

Number registers saved.

*`fUseBP`*

Indicates whether the EBP register is allocated. Use either 0 or 1.

*`cbFrame`*

Indicates the frame type. For more information, see FPO_DATA.

## See also

Directives reference
MASM BNF Grammar

# .FREESTACK

Generates a **WOD_ALLOC_SMALL**, **WOD_ALLOC_LARGE**, or **WOD_ALLOC_HUGE** unwind code entry with the specified size for the current offset in the epilogue.

## Syntax

> **.FREESTACK** *size*

## Parameters

*size*
The number of bytes to deallocate from the stack. Must be a multiple of 8.

## Remarks

> **Important**
>
> This directive is experimental and is subject to change. Enable Unwind Version 3 by using `ml64.exe /unwindv3`.

- You can use **.FREESTACK** only in Unwind Version 3. It's the epilogue counterpart to .ALLOCSTACK.
- You can use **.FREESTACK** only within an epilogue region, between .BEGINEPILOG and .ENDEPILOG.
- In Unwind Version 3, **.FREESTACK** must appear **before** the `add rsp, N` instruction that implements the stack deallocation.
- To ensure agreement, wrap both the unwind directives and the code they're meant to unwind in a macro.
- These directives generate unwind metadata (`.xdata` and `.pdata` sections) but don't produce executable code.
- Microsoft Assembler (MASM) chooses the most efficient encoding for a given size.

MASM emits one of three unwind codes depending on the size:

Expand table

| Unwind code | Condition |
| --- | --- |
| `WOD_ALLOC_SMALL` | *size* ≤ 128 bytes |
| `WOD_ALLOC_LARGE` | *size* ≤ 32 KB |
| `WOD_ALLOC_HUGE` | *size* > 32 KB |

In Unwind Version 3, epilogue directives are mandatory for epilogue unwind code generation. The assembler doesn't emit epilogue unwind codes without a `.BEGINEPILOG`/`.ENDEPILOG` pair.

> **Note**
>
> In Unwind Version 1, the directive appears after the corresponding instruction. In Unwind Version 3, the directive appears **before** the instruction.

## Example for unwindv3

**asm**

```
; ml64 /unwindv3 ex1.asm /link /entry:Example1 /SUBSYSTEM:CONSOLE
_text SEGMENT
Example1 PROC FRAME
   .allocstack 16
   sub rsp, 16
.endprolog
   ; function body
   .beginepilog
   .freestack 16
   add rsp, 16
   .endepilog
   ret
Example1 ENDP
_text ENDS
END
```

## See also

x64 Unwind Version 3 (experimental)

Directives Reference

.ALLOCSTACK

.BEGINEPILOG

MASM BNF Grammar

ⓘ **Note:** The author created this article with assistance from AI. Learn more

---

Last updated on 05/12/2026

# FWORD

Article • 08/03/2021

Allocates and optionally initializes 6 bytes of storage for each *initializer*.

## Syntax

> [[*name*]] **FWORD** *initializer* [[**,** *initializer* ...]]

## Remarks

Also can be used as a type specifier anywhere a type is legal.

## See also

Directives Reference
DF
MASM BNF Grammar

# GOTO

Article • 08/03/2021

Transfers assembly to the line marked *:macrolabel*.

## Syntax

> **GOTO** *macrolabel*

## Remarks

**GOTO** is permitted only inside MACRO, FOR, FORC, REPEAT, and WHILE blocks. The *macrolabel* target must be the only directive on the line and must be preceded by a leading colon.

## See also

Directives reference
MASM BNF Grammar

# GROUP

Article • 08/03/2021

(32-bit MASM only.) Add the specified *segments* to the group called *name*.

## Syntax

> *name* **GROUP** *segment* [[, *segment* ...]]

## See also

Directives reference
MASM BNF Grammar

# .IF (32-bit MASM)

Article • 08/03/2021

Generates code that tests *condition1* (for example, AX > 7) and executes the *statements* if that condition is true. (32-bit MASM only.)

## Syntax

> **.IF** *condition1*
> *statements*
> [[**.ELSEIF** *condition2*
> *statements*]]
> [[**.ELSE**
> *statements*]]
> **.ENDIF**

## Remarks

If a .ELSE follows, its statements are executed if the original condition was false. Note that the conditions are evaluated at run time.

## See also

Directives reference
MASM BNF Grammar

# IF

Article • 08/03/2021

Grants assembly of *ifstatements* if *expression1* is true (nonzero) or *elseifstatements* if *expression1* is false (0) and *expression2* is true.

## Syntax

> **IF** *expression1*
> *if-statements*
> [[**ELSEIF** *expression2*
> *elseif-statements*]]
> [[**ELSE**
> *else-statements*]]
> **ENDIF**

## Remarks

The following directives may be substituted for ELSEIF: **ELSEIFB**, **ELSEIFDEF**, **ELSEIFDIF**, **ELSEIFDIFI**, **ELSEIFE**, **ELSEIFIDN**, **ELSEIFIDNI**, **ELSEIFNB**, and **ELSEIFNDEF**. Optionally, assembles *else-statements* if the previous expression is false. Note that the expressions are evaluated at assembly time.

## See also

Directives reference
MASM BNF Grammar

# IF1 and IF2

Article • 08/03/2021

**IF1** block is evaluated on first assembly pass.

**IF2** block is evaluated on every assembly pass if **OPTION:SETIF2** is **TRUE**.

## Syntax

> **IF1;;**

> **IF2;;**

## Remarks

See IF for complete syntax.

Unlike version 5.1, MASM 6.1 and above do most of its work on its first pass, then performs as many subsequent passes as necessary. In contrast, MASM 5.1 always assembles in two source passes. As a result, you may need to revise or delete some pass-dependent constructs under MASM 6.1 and above.

### Two-Pass Directives

To assure compatibility, MASM 6.1 and above support 5.1 directives referring to two passes. These include **.ERR1**, **.ERR2**, **IF1**, **IF2**, **ELSEIF1**, and **ELSEIF2**. For second-pass constructs, you must specify OPTION SETIF2. Without **OPTION SETIF2**, the **IF2** and **.ERR2** directives cause an error:

Output

```
.ERR2 not allowed : single-pass assembler
```

MASM 6.1 and above handle first-pass constructs differently. It treats the **.ERR1** directive as **.ERR**, and the **IF1** directive as **IF**.

## See also

# MASM BNF Grammar

# IFB

Article • 08/03/2021

Grants assembly if *textitem* is blank.

## Syntax

> **IFB** *textitem*

## Remarks

See IF for complete syntax.

## See also

Directives reference
MASM BNF Grammar

# IFDEF

Article • 08/03/2021

Grants assembly if *name* is a previously defined label, variable, or symbol.

## Syntax

> **IFDEF** *name*

## Remarks

See IF for complete syntax.

## See also

Directives reference
MASM BNF Grammar

# IFDIF, IFDIFI

Article • 08/03/2021

Grants assembly if the text items are different.

## Syntax

> **IFDIF** *textitem1*, *textitem2*
> **IFDIFI** *textitem1*, *textitem2*

## Remarks

If **IFDIFI** is given, the comparison is case insensitive. See IF for complete syntax.

## See also

Directives reference
MASM BNF Grammar

# IFE

Article • 08/03/2021

Grants assembly if *expression* is false (0).

## Syntax

> **IFE** *expression*

## Remarks

See IF for complete syntax.

## See also

Directives reference
MASM BNF Grammar

# IFIDN, IFIDNI

Article • 08/03/2021

Grants assembly if the text items are identical.

## Syntax

> **IFIDN** *textitem1*, *textitem2*
> **IFIDNI** *textitem1*, *textitem2*

## Remarks

If **IFIDNI** is given, the comparison is case insensitive. See IF for complete syntax.

## See also

Directives reference
MASM BNF Grammar

# IFNB

Article • 08/03/2021

Grants assembly if *textitem* is not blank.

## Syntax

> **IFNB** *textitem*

## Remarks

See IF for complete syntax.

## See also

Directives reference
MASM BNF Grammar

# IFNDEF

Article • 08/03/2021

Grants assembly if *name* has not been defined.

## Syntax

> **IFNDEF** *name*

## Remarks

See IF for complete syntax.

## See also

Directives reference
MASM BNF Grammar

# INCLUDE

Article • 08/03/2021

Inserts source code from the source file given by *filename* into the current source file during assembly.

## Syntax

> **INCLUDE** *filename*

## Remarks

The *filename* must be enclosed in angle brackets if it includes a backslash, semicolon, greater-than symbol, less-than symbol, single quotation mark, or double quotation mark.

## See also

Directives reference
MASM BNF Grammar

# INCLUDELIB

Article • 08/03/2021

Informs the linker that the current module should be linked with *libraryname*.

## Syntax

> **INCLUDELIB** *libraryname*

## Remarks

The *libraryname* must be enclosed in angle brackets if it includes a backslash, semicolon, greater-than symbol, less-than symbol, single quotation mark, or double quotation mark.

## See also

Directives reference
MASM BNF Grammar

# INSTR

Article • 08/03/2021

Finds the first occurrence of *textitem2* in *textitem1*.

## Syntax

> *name* **INSTR** [[*position*,]] *textitem1*, *textitem2*

## Remarks

The starting *position* is optional. Each text item can be a literal string, a constant preceded by a %, or the string returned by a macro function.

## See also

Directives reference
MASM BNF Grammar

# INVOKE

Article • 08/03/2021

(32-bit MASM only.) Calls the procedure at the address given by *expression*, passing the arguments on the stack or in registers according to the standard calling conventions of the language type.

## Syntax

> **INVOKE** *expression* [[**,** *argument* ...]]

## Remarks

Each argument passed to the procedure may be an expression, a register pair, or an address expression (an expression preceded by **ADDR**).

## See also

Directives reference
MASM BNF Grammar

# IRP

Article • 08/03/2021

## Syntax

> **IRP**

## Remarks

See FOR.

## See also

Directives reference
MASM BNF Grammar

# IRPC

Article • 08/03/2021

## Syntax

> **IRPC**

## Remarks

See FORC.

## See also

Directives reference
MASM BNF Grammar

# .K3D (32-bit MASM)

Article • 08/03/2021

Enables assembly of **K3D** instructions. (32-bit MASM only.)

## Syntax

> **.K3D**

## See also

Directives reference
MASM BNF Grammar

# LABEL

Article • 08/03/2021

Creates a new label by assigning the current location-counter value and the given *qualifiedType* to *name*.

## Syntax

> *name* **LABEL** *qualifiedType*
>
> *name* **LABEL** ⟦**NEAR** | **FAR** | **PROC**⟧ **PTR** ⟦*qualifiedType*⟧

## See also

Directives reference
MASM BNF Grammar

# .LALL

Article • 08/03/2021

## Syntax

> **.LALL**

## Remarks

See .LISTMACROALL.

## See also

Directives reference
MASM BNF Grammar

# .LFCOND

Article • 08/03/2021

## Syntax

> **.LFCOND**

## Remarks

See .LISTIF.

## See also

Directives reference
MASM BNF Grammar

# .LIST

Article • 08/03/2021

Starts listing of statements.

## Syntax

```
.LIST
```

## Remarks

This is the default.

## See also

Directives reference
MASM BNF Grammar

# .LISTALL

Article • 08/03/2021

Starts listing of all statements.

## Syntax

```
.LISTALL
```

## Remarks

Equivalent to the combination of .LIST, .LISTIF, and .LISTMACROALL.

## See also

Directives reference
MASM BNF Grammar

# .LISTIF

Article • 08/03/2021

Starts listing of statements in false conditional blocks.

## Syntax

```
.LISTIF
```

## Remarks

Same as .LFCOND.

## See also

Directives reference
MASM BNF Grammar

# .LISTMACRO

Article • 08/03/2021

Starts listing of macro expansion statements that generate code or data.

## Syntax

```
.LISTMACRO
```

## Remarks

This is the default. Same as .XALL.

## See also

Directives reference
MASM BNF Grammar

# .LISTMACROALL

Article • 08/03/2021

Starts listing of all statements in macros.

## Syntax

```
.LISTMACROALL
```

## Remarks

Same as .LALL.

## See also

Directives reference
MASM BNF Grammar

# LOCAL

Article • 08/03/2021

In the first directive, within a macro, **LOCAL** defines labels that are unique to each instance of the macro.

## Syntax

> **LOCAL** *localId* [[, *localId* ...]]
>
> **LOCAL** *labelId* [[ **[***count***]** ]] [[**:***qualifiedType*]] [[**,** *labelId* [[ **[***count***]** ]] [[*qualifiedType*]] ...]]

## Remarks

In the second directive, within a procedure definition (**PROC**), **LOCAL** creates stack-based variables that exist for the duration of the procedure. The *labelId* may be a simple variable or an array containing *count* elements, where *count* is a constant expression.

## See also

Directives reference
MASM BNF Grammar

# MACRO

Article • 08/03/2021

Marks a macro block called *name* and establishes *parameter* placeholders for arguments passed when the macro is called.

## Syntax

> *name* **MACRO** ⟦*parameter* ⟦**:REQ** | :=*default* | *args* **:VARARG**⟧ ...⟧
> *statements*
> ⟦**GOTO** :*macrolabelId*⟧
> ⟦**EXITM**⟧
> **ENDM** ⟦*value*⟧

## Remarks

A macro function returns *value* to the calling statement.

## See also

Directives reference
GOTO (MASM)
ENDM
MASM BNF Grammar

# MMWORD

Used for 64-bit multimedia operands with MMX and SSE (XMM) instructions.

## Syntax

> **MMWORD**

## Remarks

**MMWORD** is a type. Prior to **MMWORD** being added to MASM, equivalent functionality could have been achieved with:

```asm
    mov mm0, qword ptr [ebx]
```

While both instructions work on 64-bit operands, **QWORD** is the type for 64-bit unsigned integers and **MMWORD** is the type for a 64-bit multimedia value.

**MMWORD** is intended to represent the same type as __m64.

## Example

```asm
    movq     mm0, mmword ptr [ebx]
```

## See also

MASM BNF Grammar

---

Last updated on 08/04/2025

# .MMX (32-bit MASM)

Article • 08/03/2021

Enables assembly of MMX or single-instruction, multiple data (SIMD) instructions. (32-bit MASM only.)

## Syntax

```
.MMX
```

## See also

Directives reference
MASM BNF Grammar

# .MODEL (32-bit MASM)

Article • 08/03/2021

Initializes the program memory model. (32-bit MASM only.)

## Syntax

> **.MODEL** *memory-model* [[, *language-type*]] [[, *stack-option*]]

### Parameters

*memory-model*
Required parameter that determines the size of code and data pointers.

*language-type*
Optional parameter that sets the calling and naming conventions for procedures and public symbols.

*stack-option*
Optional parameter.

*stack-option* is not used if *memory-model* is **FLAT**.

Specifying **NEARSTACK** groups the stack segment into a single physical segment (**DGROUP**) along with data. The stack segment register (**SS**) is assumed to hold the same address as the data segment register (**DS**). **FARSTACK** does not group the stack with **DGROUP**; thus **SS** does not equal **DS**.

## Remarks

**.MODEL** is not used in MASM for x64 (ml64.exe).

The following table lists the possible values for each parameter when targeting 16-bit and 32-bit platforms:

| Parameter | 32-bit values | 16-bit values (support for earlier 16-bit development) |
|---|---|---|
| *memory-model* | **FLAT** | **TINY**, **SMALL**, **COMPACT**, **MEDIUM**, **LARGE**, **HUGE**, **FLAT** |
| *language-type* | **C**, **STDCALL** | **C**, **BASIC**, **FORTRAN**, **PASCAL**, **SYSCALL**, **STDCALL** |
| *stack-option* | Not used | **NEARSTACK**, **FARSTACK** |

# Code

For MASM-related samples, download the Compiler samples from Visual C++ Samples and Related Documentation for Visual Studio 2010.

The following example demonstrates the use of the `.MODEL` directive.

# Example

asm

```
; file simple.asm
; For x86 (32-bit), assemble with debug information:
;   ml -c -Zi simple.asm
; For x64 (64-bit), assemble with debug information:
;   ml64 -c -DX64 -Zi simple.asm
;
; In this sample, the 'X64' define excludes source not used
;  when targeting the x64 architecture

ifndef X64
.686p
.XMM
.model flat, C
endif

.data
; user data

.code
; user code

fxn PROC public
  xor eax, eax ; zero function return value
  ret
fxn ENDP

end
```

# See also

Directives Reference
MASM BNF Grammar

# NAME

Article • 08/03/2021

Ignored.

## Syntax

```
NAME
```

## See also

Directives reference
MASM BNF Grammar

# .NOCREF

Article • 08/03/2021

Suppresses listing of symbols in the symbol table and browser file.

## Syntax

> **.NOCREF** [[*name*[[**,** *name* ...]]]]

## Remarks

If names are specified, then only the given names are suppressed. Same as .XCREF.

## See also

Directives Reference
MASM BNF Grammar

# .NOLIST

Article • 08/03/2021

Suppresses program listing.

## Syntax

```
.NOLIST
```

## Remarks

Same as .XLIST.

## See also

Directives reference
MASM BNF Grammar

# .NOLISTIF

Suppresses listing of conditional blocks whose condition evaluates to false (0).

## Syntax

> **.NOLISTIF**

## Remarks

This is the default. Same as .SFCOND.

## See also

Directives reference
MASM BNF Grammar

---

Last updated on 02/21/2025

# .NOLISTMACRO

Article • 08/03/2021

Suppresses listing of macro expansions.

## Syntax

```
.NOLISTMACRO
```

## Remarks

Same as .SALL.

## See also

Directives reference
MASM BNF Grammar

# `OPTION`

Article • 09/22/2021

Enables and disables features of the assembler.

## Syntax

> `OPTION` *option-list*

## Remarks

Available options include:

AVXENCODING

`CASEMAP`

`DOTNAME`

`NODOTNAME`

`EMULATOR`

`NOEMULATOR`

`EPILOGUE`

`EXPR16`

`EXPR32`

LANGUAGE

`LJMP`

`NOLJMP`

`M510`

`NOM510`

`NOKEYWORD`

`NOSIGNEXTEND`

`OFFSET`

`OLDMACROS`

`NOOLDMACROS`

`OLDSTRUCTS`

`NOOLDSTRUCTS`

`PROC`

`PROLOGUE`

`READONLY`

`NOREADONLY`

`SCOPED`

`NOSCOPED`

`SEGMENT`

`SETIF2`

The `OPTION AVXENCODING` directive is available in Visual Studio 2019 version 16.7 and later.

## See also

Directives reference
MASM BNF Grammar

# OPTION AVXENCODING

Selects the preferred encoding of AVX instructions.

## Syntax

> `OPTION AVXENCODING:` *preference*

## Background

Intel AVX and AVX-512 instructions are encoded differently from other Intel Architecture instructions. Rather than relying on instruction prefix bytes and opcode lead-in bytes to select an instruction, they use a single multi-byte prefix. This prefix can encode an additional register operand. In 64-bit mode, it also has bits to select registers 8 through 15 for the other operands, and a REX prefix isn't needed.

Three different multi-byte prefixes are used. AVX instructions can be encoded with a 3-byte VEX prefix, where the second and third bytes help to select the specific operation executed. If some of the fields in the 3-byte VEX prefix have specific values, a 2-byte form of the VEX prefix can be used instead. Finally, some AVX instructions also have an AVX-512 form, which is encoded with a 4-byte EVEX prefix. The EVEX prefix has an additional fields to select 512-bit vector length and other options specific to AVX-512 instructions.

Because some AVX instructions might be encoded in up to three different ways, there are rules to select which form to use. The `OPTION AVXENCODING` directive is used to select which set of rules to use.

## Preferences

Expand table

| Preference | Encoding preference priority |
|---|---|
| `PREFER_FIRST` | Use first defined form if possible. |
| `PREFER_VEX` | Use `VEX` encoding in preference to `EVEX` encoding. |
| `PREFER_VEX3` | Use 3-byte `VEX` encoding in preference to `EVEX` encoding. |
| `PREFER_EVEX` | Use `EVEX` encoding in preference to `VEX` encoding. |
| `NO_EVEX` | Don't encode using `EVEX`. |

The AVXENCODING order applies only if the instruction prefix form isn't specified for the instruction. If the instruction prefix is explicitly coded, it takes precedence over the **`AVXENCODING`** option. For example, `evex vpmaddwd` will use **`EVEX`**-prefixed `VPMADDWD` even if `OPTION AVXENCODING:no_EVEX` is specified.

If an assembly source doesn't use `OPTION AVXENCODING` to select encoding preferences, the **`PREFER_FIRST`** rules are used by default. By these rules, the earliest instruction encodings added to Microsoft Macro Assembler (MASM) have precedence, and among those possibilities the encoding with the shortest prefix form is preferred. These rules ensure that code assembled by MASM still generates the same object code if new instruction forms are added later.

The **`PREFER_VEX`** rules always tries to encode with a **`VEX`** prefix before trying to encode with **`EVEX`**. If possible, the 2-byte form of **`VEX`** is used instead of the 3-byte form.

The **`PREFER_VEX3`** preference always tries to encode with a 3-byte **`VEX`** prefix before trying to encode with **`EVEX`**. The 2-byte form of **`VEX`** won't be used.

The **`PREFER_EVEX`** option always tries to encode with an **`EVEX`** prefix, and only tries to use **`VEX`** if the instruction has no AVX-512 form.

The **`NO_EVEX`** preference only tries to encode by using a **`VEX`** prefix. No AVX-512 instruction forms are allowed. This option lets you assemble code for use on platforms that have no AVX-512 support at all.

The **`OPTION AVXENCODING`** directive is available in Visual Studio 2019 version 16.7 and later.

## Example

This example uses `VPDPBUSD` and `VPMADDWD` instructions to illustrate how the **`AVXENCODING`** option works. `VPDPBUSD` was first defined to be encoded only with `EVEX`, and was later extended with a VEX-encoded form for platforms without AVX-512 support, while `VPMADDWD` was AVX and extended to AVX-512. The listing output from assembling the example shows how changing the **`AVXENCODING`** mode affects the object code generated for each instruction. The prefix for each instruction ends at the "/".

asm

```
00000000  62 F2 6D 08/ 50        vpdpbusd xmm1, xmm2, xmm3
          CB
00000006  C5 E9/ F5 CB           vpmaddwd xmm1, xmm2, xmm3

                                 option avxencoding:no_EVEX
0000000A  C4 E2 69/ 50 CB        vpdpbusd xmm1, xmm2, xmm3
0000000F  C5 E9/ F5 CB           vpmaddwd xmm1, xmm2, xmm3
```

```
                                option avxencoding:prefer_VEX
00000013  C4 E2 69/ 50 CB        vpdpbusd xmm1, xmm2, xmm3
00000018  C5 E9/ F5 CB           vpmaddwd xmm1, xmm2, xmm3

                                option avxencoding:prefer_VEX3
0000001C  C4 E2 69/ 50 CB        vpdpbusd xmm1, xmm2, xmm3
00000021  C4 E1 69/ F5 CB        vpmaddwd xmm1, xmm2, xmm3

                                option avxencoding:prefer_EVEX
00000026  62 F2 6D 08/ 50        vpdpbusd xmm1, xmm2, xmm3
          CB
0000002C  62 F1 6D 08/ F5        vpmaddwd xmm1, xmm2, xmm3
          CB

                                option avxencoding:prefer_first
00000032  62 F2 6D 08/ 50        vpdpbusd xmm1, xmm2, xmm3
          CB
00000038  C5 E9/ F5 CB           vpmaddwd xmm1, xmm2, xmm3
```

# See also

Option (MASM)

Directives Reference

---

Last updated on 02/21/2025

# `OPTION LANGUAGE`

10/03/2025

Sets default language features such as calling convention.

## Syntax

> `OPTION LANGUAGE:` *language*

## Remarks

Available languages include:

`C`
`SYSCALL`

`STDCALL`
`PASCAL`

`FORTRAN`
`BASIC`

`SYSCALL`, `PASCAL`, `FORTRAN`, and `BASIC` aren't supported when used with .MODEL FLAT.

## See also

OPTION directive
Directives reference

# ORG

Article • 08/03/2021

Sets the location counter to *expression*.

## Syntax

> **ORG** *expression*

## See also

Directives reference
MASM BNF Grammar

# %OUT

Article • 08/03/2021

See ECHO.

## Syntax

> **%OUT**

## See also

Directives reference
MASM BNF Grammar

# OWORD

Article • 08/03/2021

Used as a type specifier when an 16-byte data type is required.

## Syntax

> [[*name*]] **OWORD** *initializer* [[, *initializer* ...]]

## See also

Directives reference
MASM BNF Grammar

# PAGE

Article • 08/03/2021

The first directive sets line *length* and character *width* of the program listing. If no arguments are given, generates a page break. The second directive increments the section number and resets the page number to 1.

## Syntax

> **PAGE** [[*length*]][[, *width*]]

> **PAGE +**

## See also

Directives reference
MASM BNF Grammar

# .POP2REG

Generates a two-register pop unwind code entry for the specified register pair using the current offset in the epilogue.

## Syntax

> **.POP2REG** *register1*, *register2*

## Parameters

*register1*
The first register to pop. Must be a general-purpose 64-bit register.

*register2*
The second register to pop. Must be a general-purpose 64-bit register.

*register1* and *register2* may each be one of:
`RAX, RCX, RDX, RBX, RDI, RSI, RBP, R8, R9, R10, R11, R12, R13, R14, R15, R16, R17, R18, R19, R20, R21, R22, R23, R24, R25, R26, R27, R28, R29, R30, R31`.

## Remarks

> **ⓘ Important**
>
> This directive is experimental and is subject to change. Enable Unwind Version 3 with `ml64.exe /unwindv3`.

- **.POP2REG** is only valid in Unwind Version 3. It's the epilogue counterpart to .PUSH2REG.
- **.POP2REG** is only allowed within an epilogue region, between .BEGINEPILOG and .ENDEPILOG.
- These directives generate unwind metadata (`.xdata` and `.pdata` sections) but don't produce executable code.
- In Unwind Version 3, **.POP2REG** must appear **before** the instruction that actually implements the action to be unwound.

- In Unwind Version 3, epilogue directives are mandatory for epilogue unwind code generation. No epilogue unwind codes are emitted without a `.BEGINEPILOG`/`.ENDEPILOG` pair.
- To ensure agreement, wrap both the unwind directives and the code they're meant to unwind in a macro.

If *register1* and *register2* are consecutive and in increasing order, MASM emits a `WOD_PUSH_CONSECUTIVE_2` unwind code. Otherwise, MASM emits a `WOD_PUSH2` unwind code.

> ⓘ **Note**
>
> In Unwind Version 1, the directive appears after the corresponding instruction. In Unwind Version 3, the directive appears **before** the instruction.

## Example

**asm**

```
; ml64 /unwindv3 ex1.asm /link /entry:Example1 /SUBSYSTEM:CONSOLE
_text SEGMENT
Example1 PROC FRAME
   .push2reg rbx, rsi
   push2 rbx, rsi
   .push2reg r10, r11
   push2 r10, r11
.endprolog
   ; rest of function ...
   .beginepilog
   .pop2reg r10, r11
   pop2 r10, r11
   .pop2reg rbx, rsi
   pop2 rbx, rsi
   .endepilog
   ret
Example1 ENDP
_text ENDS
END
```

## See also

x64 Unwind Version 3 (experimental)

Directives Reference

.PUSH2REG

.POPREG

.BEGINEPILOG

MASM BNF Grammar

ⓘ **Note:** The author created this article with assistance from AI. Learn more

---

Last updated on 05/12/2026

# POPCONTEXT

Article • 08/03/2021

Restores part or all of the current *context* (saved by the PUSHCONTEXT directive). The *context* can be **ASSUMES** (32-bit MASSM only), **RADIX**, **LISTING**, **CPU** (32-bit MASSM only), or **ALL**.

## Syntax

> **POPCONTEXT** *context*

## See also

Directives reference
MASM BNF Grammar

# .POPREG

Generates a `WOD_PUSH` unwind code entry for the specified register using the current offset in the epilogue.

## Syntax

> .POPREG *register*

## Parameters

*register*
The register to pop. Must be a general-purpose 64-bit register.

## Remarks

> **ⓘ Important**
>
> This directive is experimental and is subject to change. Enable Unwind Version 3 by using the `/unwindv3` option in ml64.exe.

`.POPREG` is only valid in Unwind Version 3. It's the epilogue counterpart to .PUSHREG.

`.POPREG` is only allowed within an epilogue region, between .BEGINEPILOG and .ENDEPILOG. These directives generate unwind metadata (`.xdata` and `.pdata` sections) but don't produce executable code. In Unwind Version 3, `.POPREG` must appear **before** the `pop` instruction that implements the register restore. To ensure agreement, wrap both the unwind directives and the code they're meant to unwind in a macro.

*register* may be one of:
`RAX, RCX, RDX, RBX, RDI, RSI, RBP, R8, R9, R10, R11, R12, R13, R14, R15, R16, R17, R18, R19, R20, R21, R22, R23, R24, R25, R26, R27, R28, R29, R30, R31`

In Unwind Version 3, epilogue directives are mandatory for epilogue unwind code generation. The assembler doesn't emit epilogue unwind codes without a `.BEGINEPILOG`/`.ENDEPILOG` pair.

> **ⓘ Note**

In Unwind Version 1, the directive appears after the corresponding instruction. In Unwind Version 3, the directive appears **before** the instruction.

## Example for unwindv3

asm

```
; ml64 /unwindv3 ex1.asm /link /entry:Example1 /SUBSYSTEM:CONSOLE
_text SEGMENT
Example1 PROC FRAME
   .pushreg rbx
   push rbx
   .pushreg rsi
   push rsi
.endprolog
   ; rest of function ...
   .beginepilog
   .popreg rsi
   pop rsi
   .popreg rbx
   pop rbx
   .endepilog
   ret
Example1 ENDP
_text ENDS
END
```

## See also

x64 Unwind Version 3 (experimental)

Directives Reference

.PUSHREG

.POP2REG

.BEGINEPILOG

MASM BNF Grammar

ⓘ **Note:** The author created this article with assistance from AI. Learn more

Last updated on 05/12/2026

# .POPFRAME

Generates a `WOD_PUSH_CANONICAL_FRAME` Windows unwind data (WOD) unwind code entry in the epilogue.

## Syntax

> **.POPFRAME** [[**CODE** | *value*]]

## Parameters

*value*
An optional numeric modifier in the range 0–255.
Mutually exclusive with **CODE**. If you specify the optional **CODE** keyword, the unwind code entry receives a modifier of 1.
If you specify an optional numeric *value* instead, it receives the specified value.
If you don't pass an argument, the value is 0.

## Remarks

> **Important**
>
> This directive is experimental and is subject to change. Enable Unwind Version 3 with `ml64.exe /unwindv3`.

The **.POPFRAME** directive lets `ml64.exe` users specify how a frame function unwinds in an epilog. You can only use it within an epilog region, between .BEGINEPILOG and .ENDEPILOG.

- **.POPFRAME** is only valid in Unwind Version 3. It's the epilog counterpart of .PUSHFRAME. The operand value should match the corresponding **.PUSHFRAME** in the prologue.
- In Unwind Version 3, **.POPFRAME** must appear **before** the instruction that implements the action to be unwound. To ensure agreement, wrap both the unwind directives and the code they're meant to unwind in a macro.
- In Unwind Version 3, epilogue directives are mandatory for epilogue unwind code generation. No epilogue unwind codes are emitted without a `.BEGINEPILOG`/`.ENDEPILOG` pair.

- These directives generate unwind metadata (`.xdata` and `.pdata` sections) but don't produce executable code.

> ⓘ **Note**
>
> In Unwind Version 1, directives appear after the corresponding instruction. In Unwind Version 3, directives appear **before** the instruction.

## Example for unwindv3

The following example shows how to use **.POPFRAME** in an epilogue to indicate an interrupt handler that pushes and pops a canonical machine frame.

**asm**

```
; ml64 /unwindv3 ex1.asm /link /entry:Example1 /SUBSYSTEM:CONSOLE
_text SEGMENT
Example1 PROC FRAME
   .pushframe
   .pushreg rbx
   push rbx
   .allocstack 32
   sub rsp, 32
.endprolog
   ; interrupt handler body ...
   .beginepilog
   .freestack 32
   add rsp, 32
   .popreg rbx
   pop rbx
   .popframe
   .endepilog
   iretq
Example1 ENDP
_text ENDS
END
```

## See also

x64 Unwind Version 3 (experimental)
.PUSHFRAME
.BEGINEPILOG
.ENDEPILOG
MASM BNF Grammar

ⓘ **Note:** The author created this article with assistance from AI. Learn more

---

Last updated on 05/12/2026

# PROC

Article • 08/03/2021

Marks start and end of a procedure block called *label*. The statements in the block can be called with the **CALL** instruction or INVOKE directive.

## Syntax

> *label* **PROC** ⟦*distance*⟧ ⟦*language-type*⟧ ⟦ **PUBLIC** | **PRIVATE** | **EXPORT** ⟧
> ⟦<*prologuearg*>⟧ ⟦**USES** *reglist*⟧ ⟦, *parameter* ⟦:*tag*⟧ ...⟧
> ⟦**FRAME** ⟦:*ehandler-address*⟧ ⟧
> *statements*
> *label* **ENDP**

## Remarks

The ⟦*distance*⟧ and ⟦*language-type*⟧ arguments are valid only in 32-bit MASM.

⟦**FRAME** ⟦:*ehandler-address*⟧ ⟧ is only valid with ml64.exe, and causes MASM to generate a function table entry in .pdata and unwind information in .xdata for a function's structured exception handling unwind behavior.

When the **FRAME** attribute is used, it must be followed by an .ENDPROLOG directive.

See MASM for x64 (ml64.exe) for more information on using ml64.exe.

## Example

asm

```
; ml64 ex1.asm /link /entry:Example1 /SUBSYSTEM:CONSOLE
_text SEGMENT
Example1 PROC FRAME
   push r10
.pushreg r10
   push r15
.pushreg r15
   push rbx
.pushreg rbx
   push rsi
.pushreg rsi
.endprolog
   ; rest of function ...
```

```
   ret
Example1 ENDP
_text ENDS
END
```

The above code will emit the following function table and unwind information:

Output

```
FileHeader->Machine 34404
Dumping Unwind Information for file ex2.exe

.pdata entry 1 0x00001000 0x00001023

  Unwind data: 0x00002000

    Unwind version: 1
    Unwind Flags: None
    Size of prologue: 0x08
    Count of codes: 3
    Frame register: rbp
    Frame offset: 0x0
    Unwind codes:

      Code offset: 0x08, SET_FPREG, register=rbp, offset=0x00
      Code offset: 0x05, ALLOC_SMALL, size=0x10
      Code offset: 0x01, PUSH_NONVOL, register=rbp
```

# See also

Directives reference
MASM BNF Grammar

# PROTO

Article • 08/03/2021

Prototypes a function or procedure. You can call the function prototyped by the PROTO directive by using the INVOKE directive.

## Syntax

> *label* **PROTO** [[*distance*]] [[*language-type*]] [[, [[*parameter*]]:*tag* ...]]

### Parameters

*label*
The name of the prototyped function.

*distance* (32-bit MASM only.)
(Optional) Used in 16-bit memory models to override the default and indicate **NEAR** or **FAR** calls.

*language-type* (32-bit MASM only.)
(Optional) Sets the calling and naming convention for procedures and public symbols. Supported conventions are:

- 32-bit **FLAT** model: **C**, **STDCALL**
- 16-bit models: **C**, **BASIC**, **FORTRAN**, **PASCAL**, **SYSCALL**, **STDCALL**

*parameter*
The optional name for a function parameter.

*tag*
The type of a function parameter.

The *parameter* and *tag* parameters may appear multiple times, once for each passed argument.

## Example

This sample shows a **PROTO** declaration for a function named `addup3` that uses a **NEAR** call to override the 16-bit model default for procedure calls, and uses the **C** calling

convention for stack parameters and return values. It takes two arguments, a **WORD** and a **VARARG**.

```
MASM
```

```
addup3 PROTO NEAR C, argcount:WORD, arg1:VARARG
```

## See also

Directives Reference
.MODEL Reference
MASM BNF Grammar

# PUBLIC

Article • 08/03/2021

Makes each variable, label, or absolute symbol specified as *name* available to all other modules in the program.

## Syntax

> **PUBLIC** ⟦*language-type*⟧ *name* ⟦**,** ⟦*language-type*⟧ *name* ...⟧

## Remarks

The *language-type* argument is valid in 32-bit MASM only.

## See also

Directives reference
MASM BNF Grammar

# PURGE

Deletes the specified macros from memory.

## Syntax

> **PURGE** *macronameId* [[, *macronameId* ...]]

## See also

Directives reference
MASM BNF Grammar

---

Last updated on 02/21/2025

# .PUSH2REG

Generates a two-register push unwind code entry for the specified register pair using the current offset in the prologue.

## Syntax

> `.PUSH2REG` *register1*, *register2*

## Parameters

*register1*
The first register to push. Must be a general-purpose 64-bit register.

*register2*
The second register to push. Must be a general-purpose 64-bit register.

*register1* and *register2* may each be one of:
`RAX, RCX, RDX, RBX, RDI, RSI, RBP, R8, R9, R10, R11, R12, R13, R14, R15, R16, R17, R18, R19, R20, R21, R22, R23, R24, R25, R26, R27, R28, R29, R30, R31.`

## Remarks

> **ⓘ Important**
>
> This directive is experimental and is subject to change. Enable Unwind Version 3 by using `ml64.exe /unwindv3`.

- Use `.PUSH2REG` to specify how a frame function unwinds. You can only use this directive within the prologue, which extends from the PROC `FRAME` declaration to the .ENDPROLOG directive.
- These directives generate unwind metadata (`.xdata` and `.pdata` sections) but don't produce executable code. In Unwind Version 3, `.PUSH2REG` must appear **before** the instruction that actually implements the action to be unwound. To ensure agreement, it's a good practice to wrap both the unwind directives and the code they're meant to unwind in a macro.

- If *register1* and *register2* are consecutive and in increasing order, MASM emits a `WOD_PUSH_CONSECUTIVE_2` unwind code. Otherwise, MASM emits a `WOD_PUSH2` unwind code.

> **Note**
>
> In Unwind Version 1, the directive appears after the corresponding instruction. In Unwind Version 3, the directive appears **before** the instruction.

## Example for unwindv3

**asm**

```
; ml64 /unwindv3 ex1.asm /link /entry:Example1 /SUBSYSTEM:CONSOLE
_text SEGMENT
Example1 PROC FRAME
   .push2reg r10, r11
   push2 r10, r11
   .push2reg rbx, rsi
   push2 rbx, rsi
.endprolog
   ; rest of function ...
   ret
Example1 ENDP
_text ENDS
END
```

## See also

x64 Unwind Version 3 (experimental)

Directives Reference

.POP2REG

.PUSHREG

.ENDPROLOG

MASM BNF Grammar

**Note:** The author created this article with assistance from AI. Learn more

Last updated on 05/12/2026

# PUSHCONTEXT

Article • 08/03/2021

Saves part or all of the current *context*: segment register assumes, radix value, listing and cref flags, or processor/coprocessor values. The *context* can be **ASSUMES** (32-bit MASM only), **RADIX**, **LISTING**, **CPU** (32-bit MASM only), or **ALL**.

## Syntax

> **PUSHCONTEXT** *context*

## See also

Directives reference
MASM BNF Grammar

# .PUSHFRAME

Generates a `UWOP_PUSH_MACHFRAME` unwind code entry. If you specify the optional **CODE** keyword, the unwind code entry gets a modifier of 1. Otherwise, the modifier is 0.

## Syntax

> `.PUSHFRAME` [[**CODE**]] ;Prior to Unwind Version 3 `.PUSHFRAME` [[**CODE**]] | [0-255] ;Unwind Version 3 specific

## Remarks

Use `.PUSHFRAME` with `ml64.exe` to specify how a frame function unwinds. You can only use it within the prologue, which extends from the PROC `FRAME` declaration to the .ENDPROLOG directive.

- These directives generate unwind metadata (`.xdata` and `.pdata` sections) but don't produce executable code.
- Precede `.PUSHFRAME` with the instructions that implement the actions to be unwound.
- To ensure agreement, wrap both the unwind directives and the code they're meant to unwind in a macro.

## Unwind Version 3 behavior

> **ⓘ Important**
>
> Unwind Version 3 support is experimental and is subject to change. Enable it by using `ml64.exe /unwindv3`.

In Unwind Version 3, `.PUSHFRAME` emits a `WOD_PUSH_CANONICAL_FRAME` unwind code entry.
When you specify the optional **CODE** keyword, the unwind code entry modifier value is 1.
If you specify an optional *value*, the directive uses it directly.
If you don't pass an argument, the value is 0. The *value* must be in the range 0–255.

> **Note:** In Unwind Version 1 the directives come after the instruction. In Unwind Version 3, the directives come before the instruction. `.PUSHFRAME` must appear **before** the instruction it

> describes. This behavior is the opposite of Version 1, where the directive follows the instruction.

## Example for unwindv3

```asm
; ml64 /unwindv3 ex1.asm /link /entry:Example1 /SUBSYSTEM:CONSOLE
_text SEGMENT
Example1 PROC FRAME
   .pushframe CODE
.endprolog
   ; interrupt handler body ...
   iretq
Example1 ENDP
_text ENDS
END
```

The epilogue counterpart is .POPFRAME.

# See also

x64 Unwind Version 3 (experimental)

Directives reference

.POPFRAME

MASM BNF Grammar

MASM for x64 (ml64.exe)

ⓘ **Note:** The author created this article with assistance from AI. Learn more

Last updated on 05/12/2026

# .PUSHREG

Generates a `UWOP_PUSH_NONVOL` unwind code entry for the specified register number using the current offset in the prologue.

## Syntax

> .PUSHREG *register*

*register* may be one of:
`RAX, RCX, RDX, RBX, RDI, RSI, RBP, R8, R9, R10, R11, R12, R13, R14, R15`

Unwind Version 3 extends register support to include R16–R31. *register* may be one of:
`RAX, RCX, RDX, RBX, RDI, RSI, RBP, R8, R9, R10, R11, R12, R13, R14, R15, R16, R17, R18, R19, R20, R21, R22, R23, R24, R25, R26, R27, R28, R29, R30, R31`

## Remarks

Use `.PUSHREG` with `ml64.exe` to specify how a frame function unwinds. You can only use `.PUSHREG` within the prologue, which extends from the PROC `FRAME` declaration to the .ENDPROLOG directive. These directives generate unwind metadata (`.xdata` and `.pdata` sections) but don't produce executable code. Precede `.PUSHREG` with the instructions that actually implement the actions to be unwound. To ensure agreement, wrap both the unwind directives and the code they're meant to unwind in a macro.

## Code

The following sample shows how to push nonvolatile registers.

**asm**

```
; ml64 ex1.asm /link /entry:Example1 /SUBSYSTEM:CONSOLE
_text SEGMENT
Example1 PROC FRAME
   push r10
.pushreg r10
   push r15
.pushreg r15
   push rbx
.pushreg rbx
   push rsi
```

```
.pushreg rsi
.endprolog
   ; rest of function ...
   ret
Example1 ENDP
_text ENDS
END
```

# Unwind Version 3 behavior

> **ⓘ Important**
>
> Unwind Version 3 support is experimental and is subject to change. Enable it by using `ml64.exe /unwindv3`.

> **ⓘ Note**
>
> In Unwind Version 1, the directive appears after the corresponding instruction. In Unwind Version 3, the directive appears **before** the instruction. In Unwind Version 3, `.PUSHREG` generates a `WOD_PUSH` unwind code entry and must appear **before** the `push reg` instruction it describes. This requirement is the opposite of Version 1, where the directive follows the instruction.

# Example for unwindv3

**asm**

```
; ml64 /unwindv3 ex1.asm /link /entry:Example1 /SUBSYSTEM:CONSOLE
_text SEGMENT
Example1 PROC FRAME
   .pushreg r10
   push r10
   .pushreg r15
   push r15
   .pushreg rbx
   push rbx
   .pushreg rsi
   push rsi
.endprolog
   ; rest of function ...
   ret
Example1 ENDP
_text ENDS
END
```

The epilogue counterpart is .POPREG.

## See also

x64 Unwind Version 3 (experimental)

Directives reference

MASM BNF Grammar

MASM for x64 (ml64.exe)

ⓘ **Note:** The author created this article with assistance from AI. Learn more

---

Last updated on 05/12/2026

# QWORD

Article • 08/03/2021

Allocates and optionally initializes 8 bytes of storage for each *initializer*. Also can be used as a type specifier anywhere a type is legal. **QWORD** is a synonym of DQ.

## Syntax

> [[*name*]] **QWORD** *initializer* [[, *initializer* ...]]

## See also

Directives Reference
SQWORD
DQ
MASM BNF Grammar

# .RADIX

Article • 08/03/2021

Sets the default radix, in the range 2 to 16, to the value of *expression*.

## Syntax

> **.RADIX** *expression*

## See also

Directives reference
MASM BNF Grammar

# REAL10

Article • 08/03/2021

Allocates and optionally initializes a 10-byte floating-point number for each *initializer*.

## Syntax

> *name* **REAL10** *initializer* [[, *initializer* ...]]

## See also

Directives reference
MASM BNF Grammar

# REAL4

Article • 08/03/2021

Allocates and optionally initializes a single-precision (4-byte) floating-point number for each *initializer*.

## Syntax

> *name* **REAL4** *initializer* [[, *initializer* ...]]

## See also

Directives reference
MASM BNF Grammar

# REAL8

Article • 08/03/2021

Allocates and optionally initializes a double-precision (8-byte) floating-point number for each *initializer*.

## Syntax

> *name* **REAL8** *initializer* [[, *initializer* ...]]

## See also

Directives reference
MASM BNF Grammar

# RECORD

Article • 08/03/2021

Declares a record type consisting of the specified fields. *fieldname* names the field, *width* specifies the number of bits, and *expression* gives its initial value.

## Syntax

> *recordname* **RECORD** *fieldname*:*width* [[= *expression*]] [[, *fieldname*:*width* [[= *expression*]] ...]]

## See also

Directives reference
MASM BNF Grammar

# .REPEAT (32-bit MASM)

Article • 08/03/2021

Generates code that repeats execution of the block of *statements* until *condition* becomes true. .UNTILCXZ, which becomes true when CX is zero, may be substituted for .UNTIL. The *condition* is optional with **.UNTILCXZ**. (32-bit MASM only.)

## Syntax

> **.REPEAT**
> *statements*
> **.UNTIL** *condition*

## See also

Directives reference
MASM BNF Grammar

# REPEAT

Article • 08/03/2021

Marks a block that is to be repeated *expression* times. Same as REPT.

## Syntax

> **REPEAT** *expression*
> *statements*
> **ENDM**

## See also

Directives reference
MASM BNF Grammar

# REPT

Article • 08/03/2021

See REPEAT.

## Syntax

> **REPT**

## See also

Directives reference
MASM BNF Grammar

# .RESTOREREG

Generates either a `WOD_SAVE_NONVOL` or a `WOD_SAVE_NONVOL_FAR` unwind code entry for the specified register and offset, using the current epilogue offset.

## Syntax

> `.RESTOREREG` *reg*, *offset*

## Parameters

*reg*
The nonvolatile register to restore.

*offset*
The stack offset from which the register is restored.

## Remarks

> **ⓘ Important**
>
> This directive is experimental and is subject to change. Enable Unwind Version 3 by using `ml64.exe /unwindv3`.

`.RESTOREREG` is only valid in Unwind Version 3. It's the epilogue counterpart to .SAVEREG.

Microsoft Assembler (MASM) chooses the most efficient encoding for a given offset.

- `.RESTOREREG` is only allowed within an epilogue region, between .BEGINEPILOG and .ENDEPILOG.
- These directives generate unwind metadata (`.xdata` and `.pdata` sections) but don't produce executable code.
- In Unwind Version 3, `.RESTOREREG` must appear **before** the instruction that loads the register from the stack.
- To ensure agreement, wrap both the unwind directives and the code they're meant to unwind in a macro.

- In Unwind Version 3, epilogue directives are mandatory for epilogue unwind code generation. No epilogue unwind codes are emitted without a `.BEGINEPILOG`/`.ENDEPILOG` pair.

> ⓘ Note
>
> In Unwind Version 1, the directive appears after the corresponding instruction. In Unwind Version 3, the directive appears **before** the instruction.

## Example for unwindv3

asm

```
; ml64 /unwindv3 ex1.asm /link /entry:Example1 /SUBSYSTEM:CONSOLE
_text SEGMENT
Example1 PROC FRAME
   .allocstack 020h
   sub rsp, 020h
   .savereg rbx, 0
   mov [rsp], rbx
   .savereg rsi, 8
   mov [rsp+8], rsi
.endprolog
   ; rest of function ...
   .beginepilog
   .restorereg rsi, 8
   mov rsi, [rsp+8]
   .restorereg rbx, 0
   mov rbx, [rsp]
   .freestack 020h
   add rsp, 020h
   .endepilog
   ret
Example1 ENDP
_text ENDS
END
```

## See also

x64 Unwind Version 3 (experimental)

Directives Reference

.SAVEREG

.BEGINEPILOG

MASM BNF Grammar

ⓘ **Note:** The author created this article with assistance from AI. Learn more

---

Last updated on 05/12/2026

# .RESTOREXMM128

Generates either a `WOD_SAVE_XMM128` or a `WOD_SAVE_XMM128_FAR` Unwind Version 3 unwind code entry for the specified XMM register and offset using the current epilogue offset.

## Syntax

> `.RESTOREXMM128` *xmmreg*, *offset*

## Parameters

*xmmreg*
The XMM register to restore.

*offset*
The stack offset from which the register is restored. Must be a multiple of 16.

## Remarks

> **ⓘ Important**
>
> This directive is experimental and is subject to change. Enable Unwind Version 3 with `ml64.exe /unwindv3`.

`.RESTOREXMM128` is only valid in Unwind Version 3. It's the epilogue counterpart to .SAVEXMM128.

Microsoft Assembler (MASM) chooses the most efficient encoding for a given offset.

- `.RESTOREXMM128` is only allowed within an epilogue region, between .BEGINEPILOG and .ENDEPILOG.
- These directives generate unwind metadata (`.xdata` and `.pdata` sections) but don't produce executable code.
- In Unwind Version 3, `.RESTOREXMM128` must appear **before** the instruction that loads the XMM register from the stack.
- To ensure agreement, wrap both the unwind directives and the code they're meant to unwind in a macro.

- In Unwind Version 3, epilogue directives are mandatory for epilogue unwind code generation. No epilogue unwind codes are emitted without a `.BEGINEPILOG`/`.ENDEPILOG` pair.

> ⓘ Note
>
> In Unwind Version 1, the directive appears after the corresponding instruction. In Unwind Version 3, the directive appears **before** the instruction.

## Example for unwindv3

asm

```
; ml64 /unwindv3 ex1.asm /link /entry:Example1 /SUBSYSTEM:CONSOLE
_text SEGMENT
Example1 PROC FRAME
   .allocstack 030h
   sub rsp, 030h
   .savexmm128 xmm6, 0
   movdqa [rsp], xmm6
   .savexmm128 xmm7, 010h
   movdqa [rsp+010h], xmm7
.endprolog
   ; rest of function ...
   .beginepilog
   .restorexmm128 xmm7, 010h
   movdqa xmm7, [rsp+010h]
   .restorexmm128 xmm6, 0
   movdqa xmm6, [rsp]
   .freestack 030h
   add rsp, 030h
   .endepilog
   ret
Example1 ENDP
_text ENDS
END
```

## See also

x64 Unwind Version 3 (experimental)

Directives Reference

.SAVEXMM128

.BEGINEPILOG

MASM BNF Grammar

ⓘ **Note:** The author created this article with assistance from AI. Learn more

---

Last updated on 05/12/2026

# .SAFESEH (32-bit MASM)

Article • 08/03/2021

Registers a function as a structured exception handler. (32-bit MASM only.)

## Syntax

> **.SAFESEH** *identifier*

## Remarks

*identifier* must be the ID for a locally defined PROC or EXTRN PROC. A LABEL is not allowed. The .SAFESEH directive requires the /safeseh ml.exe command-line option.

For more information about structured exception handlers, see /SAFESEH.

For example, to register a safe exception handler, create a new MASM file (as follows), assemble with /safeseh, and add it to the linked objects.

```asm
.386
.model  flat
MyHandler   proto
.safeseh    MyHandler
end
```

## See also

Directives reference
MASM BNF Grammar

# .SALL

Article • 08/03/2021

See .NOLISTMACRO.

## Syntax

> **.SALL**

## See also

Directives reference
MASM BNF Grammar

# .SAVEREG

Generates either a `UWOP_SAVE_NONVOL` or a `UWOP_SAVE_NONVOL_FAR` unwind code entry for the specified register (*reg*) and offset (*offset*) using the current prologue offset. Microsoft Assembler (MASM) chooses the most efficient encoding.

## Syntax

> `.SAVEREG` *reg*, *offset*

## Remarks

Use `.SAVEREG` with `ml64.exe` to specify how a frame function unwinds. You can only use it within the prologue, which extends from the PROC `FRAME` declaration to the .ENDPROLOG directive.

- These directives generate unwind metadata (`.xdata` and `.pdata` sections) but don't produce executable code.
- Precede `.SAVEREG` with instructions that actually implement the actions to be unwound.
- To ensure agreement, wrap both the unwind directives and the code they're meant to unwind in a macro.

## Unwind Version 3 behavior

> **Important**
>
> Unwind Version 3 support is experimental and is subject to change. Enable it by using `ml64.exe /unwindv3`.

In Unwind Version 3, `.SAVEREG` emits either a `WOD_SAVE_NONVOL` or a `WOD_SAVE_NONVOL_FAR` unwind code entry. MASM selects the most efficient encoding based on the size of *offset*:

Expand table

| Unwind code | Condition |
| --- | --- |
| `WOD_SAVE_NONVOL` | *offset* can be encoded as a scaled 16-bit value |
| `WOD_SAVE_NONVOL_FAR` | *offset* requires a full 32-bit value |

> **Note**
>
> In Unwind Version 1, the directive appears after the corresponding instruction. In Unwind Version 3, the directive appears **before** the instruction. `.SAVEREG` must appear **before** the instruction it describes. This requirement is the opposite of Version 1, where the directive follows the instruction.

## Example for unwindv3

**asm**

```
; ml64 /unwindv3 ex1.asm /link /entry:Example1 /SUBSYSTEM:CONSOLE
_text SEGMENT
Example1 PROC FRAME
   .allocstack 020h
   sub rsp, 020h
   .savereg rbx, 0
   mov [rsp], rbx
   .savereg rsi, 8
   mov [rsp+8], rsi
.endprolog
   ; rest of function ...
   mov rsi, [rsp+8]
   mov rbx, [rsp]
   add rsp, 020h
   ret
Example1 ENDP
_text ENDS
END
```

The epilogue counterpart is .RESTOREREG.

## See also

x64 Unwind Version 3 (experimental)

Directives reference

MASM for x64 (ml64.exe)

MASM BNF Grammar

**Note:** The author created this article with assistance from AI. Learn more

Last updated on 05/12/2026

# .SAVEXMM128

Generates either a `UWOP_SAVE_XMM128` or a `UWOP_SAVE_XMM128_FAR` unwind code entry for the specified XMM register and offset using the current prologue offset. Microsoft Assembler (MASM) chooses the most efficient encoding.

## Syntax

> `.SAVEXMM128` *xmmreg* , *offset*

## Remarks

Use `.SAVEXMM128` with `ml64.exe` to specify how a frame function unwinds. You can only use it within the prologue, which extends from the PROC `FRAME` declaration to the .ENDPROLOG directive. These directives generate unwind metadata (`.xdata` and `.pdata` sections) but don't produce executable code. Precede `.SAVEXMM128` by the instructions that actually implement the actions to be unwound. To ensure agreement, wrap both the unwind directives and the code they're meant to unwind in a macro.

*offset* must be a multiple of 16.

## Unwind Version 3 behavior

> **Important**
>
> Unwind Version 3 support is experimental and is subject to change. Enable it by using `ml64.exe /unwindv3`.

In Unwind Version 3, `.SAVEXMM128` emits either a `WOD_SAVE_XMM128` or a `WOD_SAVE_XMM128_FAR` unwind code entry. Microsoft Assembler (MASM) selects the most efficient encoding based on the size of *offset*:

Expand table

| Unwind code | Condition |
| --- | --- |
| `WOD_SAVE_XMM128` | *offset* can be encoded as a scaled 16-bit value |

| Unwind code | Condition |
| --- | --- |
| `WOD_SAVE_XMM128_FAR` | *offset* requires a full 32-bit value |

> **Note**
>
> In Unwind Version 1 the directives come after the instruction. In Unwind Version 3, the directives come before the instruction. `.SAVEXMM128` must appear **before** the instruction it describes. This requirement is the opposite of Version 1, where the directive follows the instruction.

## Example for unwindv3

**asm**

```
; ml64 /unwindv3 ex1.asm /link /entry:Example1 /SUBSYSTEM:CONSOLE
_text SEGMENT
Example1 PROC FRAME
   .allocstack 030h
   sub rsp, 030h
   .savexmm128 xmm6, 0
   movdqa [rsp], xmm6
   .savexmm128 xmm7, 010h
   movdqa [rsp+010h], xmm7
.endprolog
   ; rest of function ...
   movdqa xmm7, [rsp+010h]
   movdqa xmm6, [rsp]
   add rsp, 030h
   ret
Example1 ENDP
_text ENDS
END
```

The epilogue counterpart is .RESTOREXMM128.

## See also

x64 Unwind Version 3 (experimental)

Directives reference

MASM BNF Grammar

MASM for x64 (ml64.exe)

**Note:** The author created this article with assistance from AI. Learn more

Last updated on 05/12/2026

# SBYTE

Article • 08/03/2021

Allocates and optionally initializes a signed byte of storage for each *initializer*. Can also be used as a type specifier anywhere a type is legal.

## Syntax

> *name* **SBYTE** *initializer* [[, *initializer* ...]]

## See also

Directives Reference
BYTE
DB
MASM BNF Grammar

# SDWORD

Article • 08/03/2021

Allocates and optionally initializes a signed double word (4 bytes) of storage for each *initializer*. Also can be used as a type specifier anywhere a type is legal.

## Syntax

> *name* **SDWORD** *initializer* [[**,** *initializer* ...]]

## See also

Directives Reference
DWORD
DD
MASM BNF Grammar

# SEGMENT

Article • 08/03/2021

Defines a program segment called *name* having segment attributes

# Syntax

> *name* **SEGMENT** ⟦**READONLY**⟧ ⟦*align*⟧ ⟦*combine*⟧ ⟦*use*⟧ ⟦*characteristics*⟧
> **ALIAS(***string***)** ⟦'*class*'⟧
> *statements*
> *name* **ENDS**

## Parameters

*align*
The range of memory addresses from which a starting address for the segment can be selected. The alignment type can be any one of the following:

| Align Type | Starting Address |
|---|---|
| **BYTE** | Next available byte address. |
| **WORD** | Next available word address (2 bytes per word). |
| **DWORD** | Next available double word address (4 bytes per double word). |
| **PARA** | Next available paragraph address (16 bytes per paragraph). |
| **PAGE** | Next available page address (256 bytes per page). |
| **ALIGN**(*n*) | Next available *n*th byte address. See Remarks section for more information. |

If this parameter is not specified, **PARA** is used by default.

*combine* (32-bit MASM only)
**PUBLIC**, **STACK**, **COMMON**, **MEMORY**, **AT***address*, **PRIVATE**

*use* (32-bit MASM only)
**USE16**, **USE32**, **FLAT**

*characteristics*
**INFO**, **READ**, **WRITE**, **EXECUTE**, **SHARED**, **NOPAGE**, **NOCACHE**, and **DISCARD**

These are supported for COFF only and correspond to the COFF section characteristics of similar name (for example, **SHARED** corresponds to IMAGE_SCN_MEM_SHARED). READ sets the IMAGE_SCN_MEM_READ flag. The obsolete READONLY flag caused the section to clear the IMG_SCN_MEM_WRITE flag. If any *characteristics* are set, the default characteristics are not used and only the programmer-specified flags are in effect.

*string*
This string is used as the section name in the emitted COFF object. Creates multiple sections with the same external name, with distinct MASM segment names.

Not supported with **/omf**.

*class*
Designates how segments should be combined and ordered in the assembled file. Typical values are, `'DATA'`, `'CODE'`, `'CONST'` and `'STACK'`

# Remarks

For `ALIGN(n)`, *n* may be any power of 2 from 1 to 8192; not supported with **/omf**.

# See also

Directives reference
MASM BNF Grammar

# .SEQ (32-bit MASM)

Article • 08/03/2021

Orders segments sequentially (the default order). (32-bit MASM only.)

## Syntax

```
.SEQ
```

## See also

Directives reference
MASM BNF Grammar

# .SETFRAME

Fills in the frame register field and offset in the unwind information using the specified register (*reg*) and offset (*offset*). The offset must be a multiple of 16 and less than or equal to 240. This directive also generates a `UWOP_SET_FPREG` unwind code entry for the specified register using the current prologue offset.

## Syntax

> `.SETFRAME` *reg*, *offset*

## Remarks

Use `.SETFRAME` to specify how a frame function unwinds. You can only use it within the prologue, which extends from the PROC `FRAME` declaration to the .ENDPROLOG directive. These directives generate unwind metadata (`.xdata` and `.pdata` sections) but don't produce executable code. Precede `.SETFRAME` with instructions that implement the actions to be unwound. To ensure agreement, wrap both the unwind directives and the code they're meant to unwind in a macro.

## Sample

### Description

The following sample shows how to use a frame pointer:

### Code

asm

```
; ml64 frmex2.asm /link /entry:frmex2 /SUBSYSTEM:CONSOLE
_text SEGMENT
frmex2 PROC FRAME
   push rbp
.pushreg rbp
   sub rsp, 010h
.allocstack 010h
   mov rbp, rsp
.setframe rbp, 0
.endprolog
   ; modify the stack pointer outside of the prologue (similar to alloca)
   sub rsp, 060h
```

```
   ; we can unwind from the following AV because of the frame pointer
   mov rax, 0
   mov rax, [rax] ; AV!

   add rsp, 060h
   add rsp, 010h
   pop rbp
   ret
frmex2 ENDP
_text ENDS
END
```

## Unwind Version 3 behavior

> **ⓘ Important**
>
> Unwind Version 3 support is experimental and is subject to change. Enable it by using `ml64.exe /unwindv3`.

In Unwind Version 3, `.SETFRAME` emits a `WOD_SET_FPREG` unwind code entry for the specified register using the current prologue offset. It also fills in the frame register field and offset in the unwind information. The *offset* must be a multiple of 16 and less than or equal to 240.

> **ⓘ Note**
>
> In Unwind Version 1 the directives come after the instruction. In Unwind Version 3, the directives come before the instruction. `.SETFRAME` must appear **before** the instruction it describes. This requirement is the opposite of Version 1, where the directive follows the instruction.

## Example for unwindv3

**asm**

```
; ml64 /unwindv3 frmex2.asm /link /entry:frmex2 /SUBSYSTEM:CONSOLE
_text SEGMENT
frmex2 PROC FRAME
   .pushreg rbp
   push rbp
   .allocstack 010h
   sub rsp, 010h
   .setframe rbp, 0
   mov rbp, rsp
```

```
.endprolog
   ; modify the stack pointer outside of the prologue (similar to alloca)
   sub rsp, 060h

   ; we can unwind from the following AV because of the frame pointer
   mov rax, 0
   mov rax, [rax] ; AV!

   add rsp, 060h
   add rsp, 010h
   pop rbp
   ret
frmex2 ENDP
_text ENDS
END
```

The epilogue counterpart is .UNSETFRAME.

## See also

x64 Unwind Version 3 (experimental)

MASM for x64 (ml64.exe)

Directives reference

MASM BNF Grammar

ⓘ **Note:** The author created this article with assistance from AI. Learn more

---

Last updated on 05/12/2026

# .SFCOND

Article • 08/03/2021

See .NOLISTIF.

## Syntax

```
.SFCOND
```

## See also

Directives reference
MASM BNF Grammar

# SIZESTR

Article • 08/03/2021

Finds the size of a text item.

## Syntax

> *name* **SIZESTR** *textitem*

## See also

Directives reference
MASM BNF Grammar

# SQWORD

Article • 08/03/2021

Allocates and optionally initializes 8 signed bytes of storage for each *initializer*. Also can be used as a type specifier anywhere a type is legal.

## Syntax

> *name* **SQWORD** *initializer* [[**,** *initializer* ...]]

## See also

Directives Reference
QWORD
DQ
MASM BNF Grammar

# .STACK (32-bit MASM)

Article • 08/03/2021

When used with .MODEL, defines a stack segment (with segment name **STACK**). The optional *size* specifies the number of bytes for the stack (default 1,024). The **.STACK** directive automatically closes the stack statement. (32-bit MASM only.)

## Syntax

> **.STACK** [[*size*]]

## See also

Directives reference
MASM BNF Grammar

# .STARTUP (32-bit MASM)

Article • 08/03/2021

Generates program start-up code. (32-bit MASM only.)

## Syntax

```
.STARTUP
```

## See also

Directives reference
MASM BNF Grammar

# STRUC

Article • 08/03/2021

See STRUCT.

## Syntax

> **STRUC**

## See also

Directives reference
MASM BNF Grammar

# STRUCT

Article • 08/03/2021

Declares a structure type having the specified *field-declarations*. Each field must be a valid data definition. Same as STRUC.

## Syntax

> *name* **STRUCT** ⟦*alignment*⟧ ⟦**, NONUNIQUE**⟧
> *field-declarations*
> *name* **ENDS**

## Remarks

The *name* argument must be the same in the opening and closing statement.

## See also

Directives reference
MASM BNF Grammar

# SUBSTR

Article • 08/03/2021

Returns a substring of *textitem*, starting at *position*. The *textitem* can be a literal string, a constant preceded by a `%`, or the string returned by a macro function.

## Syntax

> *name* **SUBSTR** *textitem*, *position* [[, *length*]]

## See also

Directives reference
MASM BNF Grammar

# SUBTITLE

Article • 08/03/2021

Defines the listing subtitle. Same as SUBTTL.

## Syntax

> **SUBTITLE** *text*

## See also

Directives reference
MASM BNF Grammar

# SUBTTL

See SUBTITLE.

## Syntax

> **SUBTTL**

## See also

Directives reference
MASM BNF Grammar

---

Last updated on 02/21/2025

# SWORD

Article • 08/03/2021

Allocates and optionally initializes a signed word (2 bytes) of storage for each *initializer*. Can also be used as a type specifier anywhere a type is legal.

## Syntax

> *name* **SWORD** *initializer* [[, *initializer* ...]]

## See also

Directives reference
WORD
MASM BNF Grammar

# TBYTE

Article • 08/03/2021

Allocates and optionally initializes 10 bytes of storage for each *initializer*. Can also be used as a type specifier anywhere a type is legal. **DT** is a synonym of **TBYTE**.

## Syntax

> [[*name*]] **TBYTE** *initializer* [[, *initializer* ...]]

## See also

Directives Reference
DT
MASM BNF Grammar

# TEXTEQU

Article • 08/03/2021

Assigns *textitem* to *name*. The *textitem* can be a literal string, a constant preceded by a `%`, or the string returned by a macro function.

## Syntax

> *name* **TEXTEQU** [[*textitem*]]

## See also

Directives reference
MASM BNF Grammar

# .TFCOND

Article • 08/03/2021

Toggles listing of false conditional blocks.

## Syntax

```
.TFCOND
```

## See also

Directives reference
MASM BNF Grammar

# TITLE

Article • 08/03/2021

Defines the program listing title.

## Syntax

> **TITLE** *text*

## See also

Directives reference
MASM BNF Grammar

# TYPEDEF

Article • 08/03/2021

Defines a new type or **PROTO** called *name*, which is equivalent to *type* or *protoDefinition*.

## Syntax

> *name* **TYPEDEF** *type* | **PROTO** *protoDefinition*

## See also

Directives reference
MASM BNF Grammar

# UNION

Article • 08/03/2021

Declares a union of one or more data types. The *field-declarations* must be valid data definitions. Omit the ENDS *name* label on nested **UNION** definitions.

## Syntax

> *name* **UNION** ⟦*alignment*⟧ ⟦, **NONUNIQUE**⟧
> *field-declarations*
> ⟦*name*⟧ **ENDS**

## See also

Directives reference
MASM BNF Grammar

# .UNSETFRAME

Generates a `WOD_SET_FPREG` unwind code entry for the specified register and offset using the current offset in the epilogue.

## Syntax

> `.UNSETFRAME` *reg*, *offset*

## Parameters

*reg*
The frame pointer register to unset.

*offset*
The offset into the stack frame where the frame pointer was established.

## Remarks

> **ⓘ Important**
>
> This directive is experimental and is subject to change. Enable Unwind Version 3 with `ml64.exe /unwindv3`.

`.UNSETFRAME` is only valid in Unwind Version 3. It's the epilogue counterpart to .SETFRAME.

`.UNSETFRAME` is only allowed within an epilogue region, between .BEGINEPILOG and .ENDEPILOG. These directives generate unwind metadata (`.xdata` and `.pdata` sections) but don't produce executable code.
In Unwind Version 3, `.UNSETFRAME` must appear **before** the instruction that restores the stack pointer from the frame pointer register.
It's a good practice to wrap both the unwind directives and the code they're meant to unwind in a macro to ensure agreement.

In Unwind Version 3, epilogue directives are mandatory for epilogue unwind code generation. No epilogue unwind codes are emitted without a `.BEGINEPILOG`/`.ENDEPILOG` pair.

> **Note**
>
> In Unwind Version 1, the directive appears after the corresponding instruction. In Unwind Version 3, the directive appears **before** the instruction.

## Example for unwindv3

asm

```
; ml64 /unwindv3 frmex2.asm /link /entry:frmex2 /SUBSYSTEM:CONSOLE
_text SEGMENT
frmex2 PROC FRAME
   .pushreg rbp
   push rbp
   .allocstack 010h
   sub rsp, 010h
   .setframe rbp, 0
   mov rbp, rsp
.endprolog
   ; modify the stack pointer outside of the prologue (similar to alloca)
   sub rsp, 060h

   ; we can unwind from the following AV because of the frame pointer
   mov rax, 0
   mov rax, [rax] ; AV!

   add rsp, 060h
   .beginepilog
   .unsetframe rbp, 0
   mov rsp, rbp
   .freestack 010h
   add rsp, 010h
   .popreg rbp
   pop rbp
   .endepilog
   ret
frmex2 ENDP
_text ENDS
END
```

## See also

x64 Unwind Version 3 (experimental)

Directives Reference

.SETFRAME

.BEGINEPILOG

MASM BNF Grammar

ⓘ **Note:** The author created this article with assistance from AI. Learn more

---

Last updated on 05/12/2026

# .UNTIL (32-bit MASM)

Article • 08/03/2021

(32-bit MASM only.) See .REPEAT.

## Syntax

```
.UNTIL
```

## See also

Directives reference
MASM BNF Grammar

# .UNTILCXZ (32-bit MASM)

Article • 08/03/2021

(32-bit MASM only.) See .REPEAT.

## See also

Directives reference
MASM BNF Grammar

# .WHILE (32-bit MASM)

Article • 08/03/2021

Generates code that executes the block of *statements* while *condition* remains true. (32-bit MASM only.)

## Syntax

> **.WHILE** *condition*
> *statements*
> **.ENDW**

## See also

Directives reference
MASM BNF Grammar

# WHILE

Article • 08/03/2021

Repeats assembly of block *statements* as long as *constantExpression* remains true.

## Syntax

> **WHILE** *constantExpression*
> *statements*
> **ENDM**

## See also

Directives reference
MASM BNF Grammar

# WORD

Article • 08/03/2021

Allocates and optionally initializes a word (2 bytes) of storage for each *initializer*. Can also be used as a type specifier anywhere a type is legal. DW is a synonym of **WORD**.

## Syntax

> [[*name*]] **WORD** *initializer* [[**,** *initializer* ...]]

## See also

Directives reference
SWORD
DW
MASM BNF Grammar

# .XALL

Article • 08/03/2021

See .LISTMACRO.

## Syntax

> **.XALL**

## See also

Directives reference
MASM BNF Grammar

# .XCREF

Article • 08/03/2021

See .NOCREF.

## Syntax

> **.XCREF**

## See also

Directives reference
MASM BNF Grammar

# .XLIST

Article • 08/03/2021

See .NOLIST.

## Syntax

> **.XLIST**

## See also

Directives reference
MASM BNF Grammar

# .XMM (32-bit MASM)

Article • 09/28/2022

Enables assembly of Streaming SIMD Extension instructions. (32-bit MASM only.)

## Syntax

```
.XMM
```

## See also

Directives reference
MASM BNF Grammar

# XMMWORD

Used for 128-bit multimedia operands with MMX and SSE (XMM) instructions.

## Syntax

> **XMMWORD**

## Remarks

**XMMWORD** is intended to represent the same type as __m128.

## Example

asm

```
    movdqa   xmm0, xmmword ptr [ebx]
```

## See also

MASM BNF Grammar

---

Last updated on 08/04/2025

# YMMWORD

Article • 08/03/2021

Used for 256-bit multimedia operands with Intel Advanced Vector Extensions (AVX) instructions.

## Syntax

> **YMMWORD**

## Remarks

**YMMWORD** is intended to represent the same type as `__m256` for the AVX intrinsics.

## See also

Directives reference
MASM BNF Grammar

# Symbols reference

## Date and time information

@Date

@Time

## Environment information

@Cpu

@Environ

@Interface

@UnwindVersion

@Version

## File information

@FileCur

@FileName

@Line

## Macro functions

@CatStr

@InStr

@SizeStr

@SubStr

## Miscellaneous

$

?

@@:

@B

@F

## Segment information

@code

@CodeSize

@CurSeg

@data

@DataSize

@fardata

@fardata?

@Model

@stack

@WordSize

## See also

Microsoft Macro Assembler reference

MASM BNF Grammar

ⓘ **Note:** The author created this article with assistance from AI. Learn more

Last updated on 05/12/2026

# $

Article • 08/03/2021

The current value of the location counter.

## Syntax

```
$
```

## See also

Symbols Reference
MASM BNF Grammar

# ?

Article • 08/03/2021

In data declarations, a value that the assembler allocates but does not initialize.

## Syntax

```
?
```

## See also

Symbols reference
MASM BNF Grammar

# @@:

Article • 08/03/2021

Defines a code label recognizable only between *label1* and *label2*, where *label1* is either start of code or the previous `@@:` label, and *label2* is either end of code or the next `@@:` label. See @B and @F.

## Syntax

```
@@:
```

## See also

Symbols reference
MASM BNF Grammar

# @B

Article • 08/03/2021

The location of the previous @@: label.

## Syntax

```
@B
```

## See also

Symbols reference
MASM BNF Grammar

# @CatStr

Article • 08/03/2021

Macro function that concatenates one or more strings. Returns a string.

## Syntax

> **@CatStr(** *string1* [[, *string2*...]] **)**

## See also

Symbols reference
MASM BNF Grammar

# @code

Article • 08/03/2021

The name of the code segment (text macro).

## Syntax

> **@code**

## See also

Symbols reference
MASM BNF Grammar

# @CodeSize

Article • 08/03/2021

0 for **TINY**, **SMALL**, **COMPACT**, and **FLAT** models, and 1 for **MEDIUM**, **LARGE**, and **HUGE** models (numeric equate).

## Syntax

> **@CodeSize**

## See also

Symbols reference
MASM BNF Grammar

# @Cpu

Article • 08/03/2021

A bit mask specifying the processor mode (numeric equate).

## Syntax

```
@Cpu
```

## See also

Symbols reference
MASM BNF Grammar

# @CurSeg

Article • 08/03/2021

The name of the current segment (text macro).

## Syntax

> **@CurSeg**

## See also

Symbols reference
MASM BNF Grammar

# @data

Article • 08/03/2021

The name of the default data group. Evaluates to DGROUP for all models except **FLAT**. Evaluates to **FLAT** under the **FLAT** memory model (text macro).

## Syntax

> **@data**

## See also

Symbols reference
MASM BNF Grammar

# @DataSize

Article • 08/03/2021

0 for **TINY**, **SMALL**, **MEDIUM**, and **FLAT** models, 1 for **COMPACT** and **LARGE** models, and 2 for **HUGE** model (numeric equate).

## Syntax

> **@DataSize**

## See also

Symbols reference
MASM BNF Grammar

# @Date

Article • 08/03/2021

The system date in the format mm/dd/yy (text macro).

## Syntax

> **@Date**

## See also

Symbols reference
MASM BNF Grammar

# @Environ

Article • 08/03/2021

Value of environment variable *envvar* (macro function).

## Syntax

> **@Environ(** *envvar* **)**

## See also

Symbols reference
MASM BNF Grammar

# @F

Article • 08/03/2021

The location of the next @@: label.

## Syntax

```
@F
```

## See also

Symbols reference
MASM BNF Grammar

# @fardata

Article • 08/03/2021

The name of the segment defined by the .FARDATA directive (text macro).

## Syntax

> **@fardata**

## See also

Symbols reference
MASM BNF Grammar

# @fardata?

Article • 08/03/2021

The name of the segment defined by the .FARDATA? directive (text macro).

## Syntax

```
@fardata?
```

## See also

Symbols reference
MASM BNF Grammar

# @FileCur

Article • 08/03/2021

The name of the current file (text macro).

## Syntax

> **@FileCur**

## See also

Symbols reference
MASM BNF Grammar

# @FileName

Article • 08/03/2021

The base name of the main file being assembled (text macro).

## Syntax

> **@FileName**

## See also

Symbols reference
MASM BNF Grammar

# @InStr

Article • 08/03/2021

Macro function that finds the first occurrence of *string2* in *string1*, beginning at *position* within *string1*. If *position* does not appear, search begins at start of *string1*. Returns a position integer or 0 if *string2* is not found.

## Syntax

> **@InStr(** [[*position*]], *string1*, *string2* **)**

## See also

Symbols reference
MASM BNF Grammar

# @Interface

Article • 08/03/2021

Information about the language parameters (numeric equate).

## Syntax

> **@Interface**

## See also

Symbols reference
MASM BNF Grammar

# @Line

Article • 08/03/2021

The source line number in the current file (numeric equate).

## Syntax

> **@Line**

## See also

Symbols reference
MASM BNF Grammar

# @Model

Article • 08/03/2021

1 for **TINY** model, 2 for **SMALL** model, 3 for **COMPACT** model, 4 for **MEDIUM** model, 5 for **LARGE** model, 6 for **HUGE** model, and 7 for **FLAT** model (numeric equate).

## Syntax

> **@Model**

## See also

Symbols reference
MASM BNF Grammar

# @SizeStr

Article • 08/03/2021

A macro function that returns the length of the given string. Returns an integer.

## Syntax

> **@SizeStr(** *string* **)**

## See also

Symbols reference
MASM BNF Grammar

# @stack

Article • 08/03/2021

DGROUP for near stacks or STACK for far stacks (text macro).

## Syntax

> **@stack**

## See also

Symbols reference
MASM BNF Grammar

# @SubStr

Article • 08/03/2021

A macro function that returns a substring starting at *position*.

## Syntax

> **@SubStr(** *string*, *position* ⟦, *length*⟧ **)**

## See also

Symbols reference
MASM BNF Grammar

# @Time

Article • 08/03/2021

The system time in 24-hour hh:mm:ss format (text macro).

## Syntax

> **@Time**

## See also

Symbols reference
MASM BNF Grammar

# @UnwindVersion

A predefined macro that returns the unwind version the `.asm` file is being assembled with.

## Syntax

> **@UnwindVersion**

## Remarks

> **ⓘ Important**
>
> This symbol is experimental and is subject to change. Enable Unwind Version 3 with `ml64.exe /unwindv3`.

**@UnwindVersion** is set to the unwind version used by the assembler. When `/unwindv3` is passed to `ml64.exe`, **@UnwindVersion** is set to `3`; otherwise it's set to `1`.

Use this macro with conditional assembly directives such as `IF` to assemble different code paths depending on the unwind version.

This example is intended to be assembled only as it doesn't define an application entry point:

## Example for unwindv3

asm

```
; ml64 /c /unwindv3 ex1.asm
.code

IF @UnwindVersion EQ 3

foo PROC FRAME
    .pushreg r12
    push r12

    .endprolog

    mov rax, 0

    .beginepilog
    .popreg r12
```

```
    pop r12
    .endepilog
    ret
foo ENDP
ELSE
    .ERR <@UnwindVersion should be 3>
ENDIF

END
```

## See also

x64 Unwind Version 3 (experimental)

MASM BNF Grammar

ⓘ **Note:** The author created this article with assistance from AI. Learn more

---

Last updated on 05/12/2026

# @Version

Article • 08/03/2021

The major and minor version of MASM reported at the command line, as a single number (text macro). For example, MASM version 14.23.28107.0 produces "1423".

## Syntax

> **@Version**

## See also

Symbols reference
MASM BNF Grammar

# @WordSize

Article • 08/03/2021

Two for a 16-bit segment or four for a 32-bit segment (numeric equate).

## Syntax

> **@WordSize**

## See also

Symbols reference
MASM BNF Grammar

# MASM Operators reference

Article • 08/03/2021

## Arithmetic

* (multiply)
+ (add)
- (subtract or negate)

. (field)
/ (divide)

[] (index)
MOD (remainder)

## Control Flow

! (runtime logical not)
!= (runtime not equal)
|| (runtime logical or)
&& (runtime logical and)
< (runtime less than)

<= (runtime less or equal)
== (runtime equal)
> (runtime greater than)
>= (runtime greater or equal)
& (runtime bitwise and)

CARRY? (runtime carry test)
OVERFLOW? (runtime overflow test)
PARITY? (runtime parity test)
SIGN? (runtime sign test)
ZERO? (runtime zero test)

## Logical and Shift

AND (bitwise and)

NOT (bitwise not)

OR (bitwise or)
SHL (shift bits left)

SHR (shift bits right)
XOR (bitwise exclusive or)

# Macro

! (character literal)
% (treat as text)

;; (treat as comment)
< > (treat as one literal)

& & (substitute parameter value)

# Miscellaneous

' ' (treat as string)
" " (treat as string)
`:` (local label definition)

`::` (register segment and offset)
`::` (global label definition)

; (treat as comment)
DUP (repeat declaration)

# Record

MASK (get record or field bitmask)

WIDTH (get record or field width)

# Relational

EQ (equal)
GE (greater or equal)

GT (greater than)
LE (less or equal)

LT (less than)
NE (not equal)

## Segment

: (segment override)
`::` (register segment and offset)
IMAGEREL (image relative offset)

LROFFSET (loader resolved offset)
OFFSET (segment relative offset)

SECTIONREL (section relative offset)
SEG (get segment)

## Type

HIGH (high 8 bits of lowest 16 bits)
HIGH32 (high 32 bits of 64 bits)
HIGHWORD (high 16 bits of lowest 32 bits)
LENGTH (number of elements in array)
LENGTHOF (number of elements in array)
LOW (low 8 bits)

LOW32 (low 32 bits)
LOWWORD (low 16 bits)
OPATTR (get argument type info)
PTR (pointer to or as type)
SHORT (mark short label type)

SIZE (size of type or variable)
SIZEOF (size of type or variable)
THIS (current location)
TYPE (get expression type)
.TYPE (get argument type info)

## See also

Microsoft Macro Assembler Reference

MASM BNF Grammar

# operator +

Article • 08/03/2021

The first operator returns *expression1* plus *expression2*.

## Syntax

> *expression1* + *expression2*

## See also

Operators Reference
MASM BNF Grammar

# operator -

Article • 08/03/2021

Returns *expression1* minus *expression2*. The second operator reverses the sign of *expression*.

## Syntax

> *expression1* - *expression2*
>
> -*expression*

## See also

Operators reference
MASM BNF Grammar

# operator *

Article • 08/03/2021

Returns *expression1* times *expression2*.

## Syntax

> *expression1* * *expression2*

## See also

Operators reference
MASM BNF Grammar

# operator /

Article • 08/03/2021

Returns *expression1* divided by *expression2*.

## Syntax

> *expression1* **/** *expression2*

## See also

Operators reference
MASM BNF Grammar

# operator []

Article • 08/03/2021

Returns *expression1* plus [*expression2*].

## Syntax

> *expression1* [ expression2 ]

## See also

Operators Reference
MASM BNF Grammar

# operator :

Article • 08/03/2021

Overrides the default segment of *expression* with *segment*. The *segment* can be a segment register, group name, segment name, or segment expression. The *expression* must be a constant.

## Syntax

> *segment* : *expression*

## See also

Operators reference
MASM BNF Grammar

# operator .

Article • 08/03/2021

The first operator returns *expression* plus the offset of *field* within its structure or union. The second operator returns value at the location pointed to by *register* plus the offset of *field* within its structure or union.

## Syntax

> *expression*.*field*[[.*field* ...]]
>
> [*register*].*field*[[.*field* ...]]

## See also

Operators reference
MASM BNF Grammar

# MASM operator <>

Article • 11/23/2021

Treats *text* as a single literal element.

## Syntax

> `<`*text*`>`

## See also

Operators reference
MASM BNF grammar

# operator " "

Article • 11/11/2021

Treats "*text*" as a string.

## Syntax

> "*text*"

## See also

Operators reference
MASM BNF Grammar

# operator ' '

Article • 11/11/2021

Treats '*text*' as a string.

## Syntax

> '*text*'

## See also

Operators reference
MASM BNF Grammar

# operator ! (MASM)

Article • 08/03/2021

Treats *character* as a literal character rather than as an operator or symbol.

## Syntax

> ! *character*

## See also

Operators reference
MASM BNF Grammar

# operator ;

Article • 08/03/2021

Treats *text* as a comment.

## Syntax

> ; *text*

## See also

Operators reference
MASM BNF Grammar

# operator ;;

Article • 08/03/2021

Treats *text* as a comment in a macro that appears only in the macro definition. The listing does not show *text* where the macro is expanded.

## Syntax

> ;; *text*

## See also

Operators reference
MASM BNF Grammar

# operator %

Article • 08/03/2021

Treats the value of *expression* in a macro argument as text.

## Syntax

> % *expression*

## See also

Operators reference
MASM BNF Grammar

# Substitution operator (MASM)

Article • 08/03/2021

Replaces *parameter* with its corresponding argument value.

## Syntax

> &*parameter*&

## See also

Operators reference
MASM BNF Grammar

# operator ABS

Article • 08/03/2021

See the EXTERNDEF directive.

## Syntax

> **ABS**

## See also

Operators reference
MASM BNF Grammar

# operator ADDR

Article • 08/03/2021

See the INVOKE directive.

## Syntax

> **ADDR**

## See also

Operators Reference
MASM BNF Grammar

# operator AND

Article • 08/03/2021

Returns the result of a bitwise AND operation for *expression1* and *expression2*.

## Syntax

> *expression1* **AND** *expression2*

## See also

Operators Reference
MASM BNF Grammar

# operator DUP

Article • 08/03/2021

Specifies *count* number of declarations of *initialvalue*.

## Syntax

> *count* **DUP (***initialvalue* [[**,** *initialvalue* ...]]**)**

## See also

Operators reference
MASM BNF Grammar

# operator EQ

Article • 08/03/2021

Returns true (-1) if *expression1* equals *expression2*, or returns false (0) if it does not.

## Syntax

> *expression1* **EQ** *expression2*

## See also

Operators reference
MASM BNF Grammar

# operator GE

Article • 08/03/2021

Returns true (-1) if *expression1* is greater than or equal to *expression2*, or returns false (0) if it is not.

## Syntax

> *expression1* **GE** *expression2*

## See also

Operators reference
MASM BNF Grammar

# operator GT

Article • 08/03/2021

Returns true (-1) if *expression1* is greater than *expression2*, or returns false (0) if it is not.

## Syntax

> *expression1* **GT** *expression2*

## See also

Operators reference
MASM BNF Grammar

# operator HIGH

Article • 08/03/2021

Returns the high 8 bits of the low 16 bits of *expression*. MASM expressions are 64-bit values.

## Syntax

> **HIGH** *expression*

## See also

Operators reference
MASM BNF Grammar

# operator HIGH32

Article • 08/03/2021

Returns the high 32 bits of *expression*. MASM expressions are 64-bit values.

## Syntax

> **HIGH32** *expression*

## See also

Operators reference
MASM BNF Grammar

# operator HIGHWORD

Article • 08/03/2021

Returns the high 16 bits of the low 32 bits of *expression*. MASM expressions are 64-bit values.

## Syntax

> **HIGHWORD** *expression*

## See also

Operators reference
MASM BNF Grammar

# operator IMAGEREL

Article • 08/03/2021

Returns the image relative offset of *expression*.

## Syntax

> **IMAGEREL** *expression*

## Remarks

The resulting value is often referred to as an RVA or Relative Virtual Address.

IMAGEREL is available only with COFF object emission.

## See also

Operators reference
MASM BNF Grammar

# operator LE

Article • 08/03/2021

Returns true (-1) if *expression1* is less than or equal to *expression2*, or returns false (0) if it is not.

## Syntax

> *expression1* **LE** *expression2*

## See also

Operators reference
MASM BNF Grammar

# operator LENGTH

Article • 08/03/2021

Returns the number of data items in *variable* created by the first initializer.

## Syntax

> **LENGTH** *variable*

## See also

Operators reference
MASM BNF Grammar

# operator LENGTHOF

Article • 08/03/2021

Returns the number of data objects in *variable*.

## Syntax

> **LENGTHOF** *variable*

## See also

Operators reference
MASM BNF Grammar

# operator LOW

Article • 08/03/2021

Returns the low 8 bits of *expression*. MASM expressions are 64-bit values.

## Syntax

> **LOW** *expression*

## See also

Operators reference
MASM BNF Grammar

# operator LOW32

Article • 08/03/2021

Returns the low 32 bits of *expression*. MASM expressions are 64-bit values.

## Syntax

> **LOW32** *expression*

## See also

Operators reference
MASM BNF Grammar

# operator LOWWORD

Article • 08/03/2021

Returns the low 16 bits of *expression*. MASM expressions are 64-bit values.

## Syntax

> **LOWWORD** *expression*

## See also

Operators reference
MASM BNF Grammar

# operator LROFFSET

Article • 08/03/2021

Returns the offset of *expression*. Same as **OFFSET**, but it generates a loader resolved offset, which allows Windows to relocate code segments.

## Syntax

> **LROFFSET** *expression*

## See also

Operators reference
MASM BNF Grammar

# operator LT

Article • 08/03/2021

Returns true (-1) if *expression1* is less than *expression2*, or returns false (0) if it is not.

## Syntax

> *expression1* **LT** *expression2*

## See also

Operators reference
MASM BNF Grammar

# operator MASK

Article • 08/03/2021

Returns a bit mask in which the bits in *recordfieldname* or *record* are set and all other bits are cleared.

## Syntax

> **MASK** {*recordfieldname* | *record*}

## See also

Operators reference
MASM BNF Grammar

# operator MOD

Article • 08/03/2021

Returns the integer value of the remainder (modulo) when dividing *expression1* by *expression2*.

## Syntax

> *expression1* **MOD** *expression2*

## See also

Operators reference
MASM BNF Grammar

# operator NE

Article • 08/03/2021

Returns true (-1) if *expression1* does not equal *expression2*, or returns false (0) if it does.

## Syntax

> *expression1* **NE** *expression2*

## See also

Operators reference
MASM BNF Grammar

# operator NOT

Article • 08/03/2021

Returns *expression* with all bits reversed.

## Syntax

> **NOT** *expression*

## See also

Operators reference
MASM BNF Grammar

# operator OFFSET

Article • 08/03/2021

Returns the offset into the relevant segment of *expression*.

## Syntax

> **OFFSET** *expression*

## See also

Operators reference
MASM BNF Grammar

# operator OPATTR

Article • 08/03/2021

Returns a word defining the mode and scope of *expression*. The low byte is identical to the byte returned by .TYPE. The high byte contains additional information.

## Syntax

> **OPATTR** *expression*

## See also

Operators reference
MASM BNF Grammar

# operator OR

Article • 08/03/2021

Returns the result of a bitwise OR operation for *expression1* and *expression2*.

## Syntax

> *expression1* **OR** *expression2*

## See also

Operators reference
MASM BNF Grammar

# operator PTR

Article • 08/03/2021

The first operator forces the *expression* to be treated as having the specified *type*. The second operator specifies a pointer to *type*.

## Syntax

> *type* **PTR** *expression*
>
> [*distance*] **PTR** *type*

## See also

Operators reference
MASM BNF Grammar

# operator SEG

Article • 08/03/2021

Returns the segment of *expression*.

## Syntax

> **SEG** *expression*

## See also

Operators reference
MASM BNF Grammar

# operator SHL

Article • 08/03/2021

Returns the result of shifting the bits of *expression* left *count* number of bits.

## Syntax

> *expression* **SHL** *count*

## See also

Operators reference
MASM BNF Grammar

# operator .TYPE

Article • 08/03/2021

See OPATTR.

## Syntax

> **.TYPE** *expression*

## See also

Operators reference
MASM BNF Grammar

# operator SECTIONREL

Article • 08/03/2021

Returns the section relative offset of expression relative to the section containing the target in the final executable.

## Syntax

> **SECTIONREL** *expression*

## Remarks

**SECTIONREL** is available only with COFF object emission.

## See also

Operators reference
MASM BNF Grammar

# operator SHORT

Article • 08/03/2021

Sets the type of *label* to short. All jumps to *label* must be short (within the range -128 to +127 bytes from the jump instruction to *label*).

## Syntax

> **SHORT** *label*

## See also

Operators reference
MASM BNF Grammar

# operator SHR

Article • 08/03/2021

Returns the result of shifting the bits of *expression* right *count* number of bits.

## Syntax

> *expression* **SHR** *count*

## See also

Operators reference
MASM BNF Grammar

# operator SIZE

Article • 08/03/2021

Returns the number of bytes in *variable* allocated by the first initializer.

## Syntax

> **SIZE** *variable*

## See also

Operators reference
MASM BNF Grammar

# operator SIZEOF

Article • 08/03/2021

Returns the number of bytes in *variable* or *type*.

## Syntax

> **SIZEOF** {*variable* | *type*}

## See also

Operators reference
MASM BNF Grammar

# operator `THIS`

Article • 06/06/2022

Returns an operand of specified *type* whose offset and segment values are equal to the current location counter value.

## Syntax

> `THIS` *type*

## See also

Operators reference
MASM BNF Grammar

# operator TYPE

Article • 08/03/2021

Returns the type of *expression*.

## Syntax

> **TYPE** *expression*

## See also

Operators reference
MASM BNF Grammar

# operator WIDTH

Article • 08/03/2021

Returns the width in bits of the current *recordfieldname* or *record*.

## Syntax

> **WIDTH** {*recordfieldname* | *record*}

## See also

Operators reference
MASM BNF Grammar

# operator XOR

Article • 08/03/2021

Returns the result of a bitwise **XOR** operation for *expression1* and *expression2*.

## Syntax

> *expression1* **XOR** *expression2*

## See also

Operators reference
MASM BNF Grammar

# operator == (MASM Run Time)

Article • 08/03/2021

Is equal to. Used only within .IF, .WHILE, or .REPEAT blocks and evaluated at run time, not at assembly time.

## Syntax

> *expression1* == *expression2*

## See also

Operators reference
MASM BNF Grammar

# operator != (MASM)

Article • 08/03/2021

Is not equal to. Used only within .IF, .WHILE, or .REPEAT blocks and evaluated at run time, not at assembly time.

## Syntax

> *expression1* != *expression2*

## See also

Operators reference
MASM BNF Grammar

# operator `>` (MASM runtime)

Article • 11/23/2021

Greater than operator. Used only within .IF, .WHILE, or .REPEAT blocks and evaluated at run time, not at assembly time.

## Syntax

> *expression1* `>` *expression2*

## See also

Operators reference
MASM BNF grammar

# operator `>=` (MASM runtime)

Article • 11/23/2021

Greater than or equal to operator. Used only within .IF, .WHILE, or .REPEAT blocks and evaluated at runtime, not at assembly time.

## Syntax

> *expression1* `>=` *expression2*

## See also

Operators reference
MASM BNF grammar

# operator < (MASM Run Time)

Article • 12/06/2021

Is less than. Used only within .IF, .WHILE, or .REPEAT blocks and evaluated at run time, not at assembly time.

## Syntax

> *expression1* < *expression2*

## See also

Operators reference
MASM BNF Grammar

# operator <= (MASM Run Time)

Article • 12/06/2021

Is less than or equal to. Used only within .IF, .WHILE, or .REPEAT blocks and evaluated at run time, not at assembly time.

## Syntax

> *expression1* <= *expression2*

## See also

Operators reference
MASM BNF Grammar

# operator ||

Article • 08/03/2021

Logical OR. Used only within .IF, .WHILE, or .REPEAT blocks and evaluated at run time, not at assembly time.

## Syntax

> *expression1* || *expression2*

## See also

Operators reference
MASM BNF Grammar

# operator `&&` (MASM runtime)

Article • 11/23/2021

Logical **AND**. Used only within .IF, .WHILE, or .REPEAT blocks and evaluated at runtime, not at assembly time.

## Syntax

> *expression1* `&&` *expression2*

## See also

Operators reference
MASM BNF grammar

# operator `&` (MASM)

Article • 11/23/2021

Bitwise **AND**. Used only within .IF, .WHILE, or .REPEAT blocks and evaluated at run time, not at assembly time.

## Syntax

> *expression1* `&` *expression2*

## See also

Operators reference
MASM BNF grammar

# operator ! (MASM Run Time)

Article • 08/03/2021

Logical negation. Used only within .IF, .WHILE, or .REPEAT blocks and evaluated at run time, not at assembly time.

## Syntax

> ! *expression*

## See also

Operators reference
MASM BNF Grammar

# operator CARRY?

Article • 08/03/2021

Status of carry flag. Used only within .IF, .WHILE, or .REPEAT blocks and evaluated at run time, not at assembly time.

## Syntax

> **CARRY?**

## See also

Operators reference
MASM BNF Grammar

# operator OVERFLOW?

Article • 08/03/2021

Status of overflow flag. Used only within .IF, .WHILE, or .REPEAT blocks and evaluated at run time, not at assembly time.

## Syntax

> **OVERFLOW?**

## See also

Operators reference
MASM BNF Grammar

# operator PARITY?

Article • 08/03/2021

Status of parity flag. Used only within .IF, .WHILE, or .REPEAT blocks and evaluated at run time, not at assembly time.

## Syntax

> **PARITY?**

## See also

Operators reference
MASM BNF Grammar

# operator SIGN?

Article • 08/03/2021

Status of sign flag. Used only within .IF, .WHILE, or .REPEAT blocks and evaluated at run time, not at assembly time.

## Syntax

> **SIGN?**

## See also

Operators reference
MASM BNF Grammar

# operator ZERO?

Article • 08/03/2021

Status of zero flag. Used only within .IF, .WHILE, or .REPEAT blocks and evaluated at run time, not at assembly time.

## Syntax

> **ZERO?**

## See also

Operators reference
MASM BNF Grammar

# ML Error Messages

Article • 02/13/2023

The error messages generated by MASM components fall into three categories:

- **Fatal errors.** These indicate a severe problem that prevents the utility from completing its normal process.
- **Nonfatal errors.** The utility may complete its process. If it does, its result is not likely to be the one you want.
- **Warnings.** These messages indicate conditions that may prevent you from getting the results you want.

All error messages take the following form:

> *Utility*: *Filename* (*Line*) : {*Error_type*} (*Code*): *Message_text*

where:

*Utility*
The program that sent the error message.

*Filename*
The file that contains the error-generating condition.

*Line*
The approximate line where the error condition exists.

*Error_type*
Fatal Error, Error, or Warning.

*Code*
The unique 5- or 6-digit error code.

*Message_text*
A short and general description of the error condition.

## See also

Microsoft Macro Assembler reference

# ML Fatal Error A1000

Article • 02/13/2023

**cannot open file: filename**

The assembler was unable to open a source, include, or output file.

One of the following may be a cause:

- The file does not exist.
- The file is in use by another process.
- The filename is not valid.
- A read-only file with the output filename already exists.
- The current drive is full.
- The current directory is the root and is full.
- The device cannot be written to.
- The drive is not ready.

## See also

ML Error Messages

# ML Fatal Error A1005

Article • 02/13/2023

**assembler limit : macro parameter name table full**

Too many parameters, locals, or macro labels were defined for a macro. There was no more room in the macro name table.

Define shorter or fewer names, or remove unnecessary macros.

## See also

ML Error Messages

# ML Fatal Error A1007

Article • 02/13/2023

**nesting level too deep**

The assembler reached its nesting limit. The limit is 20 levels except where noted otherwise.

One of the following was nested too deeply:

- A high-level directive such as .IF, .REPEAT, or .WHILE.
- A structure definition.
- A conditional-assembly directive.
- A procedure definition.
- A PUSHCONTEXT directive (the limit is 10).
- A segment definition.
- An include file.
- A macro.

## See also

ML Error Messages

# ML Fatal Error A1008

Article • 02/13/2023

**unmatched macro nesting**

Either a macro was not terminated before the end of the file or the terminating directive ENDM was found outside of a macro block.

One cause of this error is omission of the dot before .REPEAT or .WHILE.

## See also

ML Error Messages

# ML Fatal Error A1009

Article • 02/13/2023

**line too long**

A line in a source file exceeded the limit of 512 characters.

If multiple physical lines are concatenated with the line-continuation character ( \ ), then the resulting logical line is still limited to 512 characters.

## See also

ML Error Messages

# ML Fatal Error A1010

Article • 02/13/2023

**unmatched block nesting :**

A block beginning did not have a matching end, or a block end did not have a matching beginning. One of the following may be involved:

- A high-level directive such as .IF, .REPEAT, or .WHILE.
- A conditional-assembly directive such as IF, REPEAT, or **WHILE**.
- A structure or union definition.
- A procedure definition.
- A segment definition.
- A POPCONTEXT directive.
- A conditional-assembly directive, such as an ELSE, ELSEIF, or **ENDIF** without a matching IF.

## See also

ML Error Messages

# ML Fatal Error A1011

Article • 02/13/2023

**directive must be in control block**

The assembler found a high-level directive where one was not expected. One of the following directives was found:

- .ELSE without .IF
- .ENDIF without .IF
- .ENDW without .WHILE
- .UNTILCXZ without .REPEAT
- .CONTINUE without .WHILE or .REPEAT
- .BREAK without .WHILE or .REPEAT
- .ELSE following `.ELSE`

## See also

ML Error Messages

# ML Fatal Error A1016

Article • 02/13/2023

**Internal Assembler Error**

The MASM driver, called ML.exe, generated a system error.

Note the circumstances of the error and notify Microsoft Corporation. Product Support Services is available at https://support.microsoft.com/.

## See also

ML Error Messages

# ML Fatal Error A1017

Article • 02/13/2023

**missing source filename**

ML could not find a file to assemble or pass to the linker.

This error is generated when you give ML command-line options without specifying a filename to act upon. To assemble files that do not have an .asm extension, use the **/Ta** command-line option.

This error can also be generated by invoking ML with no parameters if the ML environment variable contains command-line options.

## See also

ML Error Messages

# ML Nonfatal Error A2004

Article • 02/13/2023

**symbol type conflict : identifier**

The EXTERNDEF or LABEL directive was used on a variable, symbol, data structure, or label that was defined in the same module but with a different type.

## See also

ML Error Messages

# ML Nonfatal Error A2006

Article • 02/13/2023

**undefined symbol : identifier**

An attempt was made to use a symbol that was not defined.

One of the following may have occurred:

- A symbol was not defined.
- A field was not a member of the specified structure.
- A symbol was defined in an include file that was not included.
- An external symbol was used without an EXTERN or EXTERNDEF directive.
- A symbol name was misspelled.
- A local code label was referenced outside of its scope.

## See also

ML Error Messages

# ML Nonfatal Error A2008

Article • 02/13/2023

**syntax error :**

A token at the current location caused a syntax error.

One of the following may have occurred:

- A dot prefix was added to or omitted from a directive.
- A reserved word (such as **C** or **SIZE**) was used as an identifier.
- An instruction was used that was not available with the current processor or coprocessor selection.
- A comparison run-time operator (such as `==`) was used in a conditional assembly statement instead of a relational operator (such as EQ).
- An instruction or directive was given too few operands.
- An obsolete directive was used.

## See also

ML Error Messages

# ML Nonfatal Error A2010

Article • 02/13/2023

> **invalid type expression**

## Remarks

The operand to THIS or PTR wasn't a valid type expression.

## See also

ML Error Messages

# ML Nonfatal Error A2019

Article • 02/13/2023

**operand must be RECORD type or field**

The operand following the WIDTH or MASK operator was not valid.

The WIDTH operator takes an operand that is the name of a field or a record. The MASK operator takes an operand that is the name of a field or a record type.

## See also

ML Error Messages

# ML Nonfatal Error A2022

Article • 02/13/2023

**instruction operands must be the same size**

The operands to an instruction did not have the same size.

## See also

ML Error Messages

# ML Nonfatal Error A2031

Article • 02/13/2023

**must be index or base register**

An attempt was made to use a register that was not a base or index register in a memory expression.

For example, the following expressions cause this error:

```asm
[ax]
[bl]
```

## See also

ML Error Messages

# ML Nonfatal Error A2034

Article • 02/13/2023

> **must be in segment block**

## Remarks

The assembler found one of the following items outside of a segment block:

- An instruction
- A label definition
- A THIS operator
- A $ operator
- A procedure definition
- An ALIGN directive
- An ORG directive

## See also

ML Error Messages

# ML Nonfatal Error A2037

Article • 02/13/2023

**statement not allowed inside structure definition**

A structure definition contained an invalid statement.

A structure cannot contain instructions, labels, procedures, control-flow directives, .STARTUP, or .EXIT.

## See also

ML Error Messages

# ML Nonfatal Error A2038

Article • 02/13/2023

**missing operand for macro operator**

The assembler found the end of a macro's parameter list immediately after the ! or % operator.

## See also

ML Error Messages

# ML Nonfatal Error A2039

Article • 02/13/2023

**line too long**

A source-file line exceeded the limit of 512 characters.

If multiple physical lines are concatenated with the line-continuation character ( \ ), then the resulting logical line is still limited to 512 characters.

## See also

ML Error Messages

# ML Nonfatal Error A2044

Article • 02/13/2023

**invalid character in file**

The source file contained a character outside a comment, string, or literal that was not recognized as an operator or other legal character.

## See also

ML Error Messages

# ML Nonfatal Error A2047

Article • 02/13/2023

**empty (null) string**

A string consisted of a delimiting pair of quotation marks and no characters within.

For a string to be valid, it must contain 1-255 characters.

## See also

ML Error Messages

# ML Nonfatal Error A2050

Article • 02/13/2023

**real or BCD number not allowed**

A floating-point (real) number or binary coded decimal (BCD) constant was used other than as a data initializer.

One of the following occurred:

- A real number or a BCD was used in an expression.
- A real number was used to initialize a directive other than DWORD, QWORD, or TBYTE.
- A BCD was used to initialize a directive other than `TBYTE`.

## See also

ML Error Messages

# ML Nonfatal Error A2054

Article • 02/13/2023

**forced error : value not equal to 0**

The conditional-error directive .ERRNZ was used to generate this error.

## See also

ML Error Messages

# ML Nonfatal Error A2055

Article • 02/13/2023

**forced error : symbol not defined**

The conditional-error directive .ERRNDEF was used to generate this error.

## See also

ML Error Messages

# ML Nonfatal Error A2057

Article • 02/13/2023

**forced error : string blank**

The conditional-error directive .ERRB was used to generate this error.

## See also

ML Error Messages

# ML Nonfatal Error A2059

Article • 02/13/2023

**forced error : strings equal**

The conditional-error directive .ERRIDN or **.ERRIDNI** was used to generate this error.

## See also

ML Error Messages

# ML Nonfatal Error A2060

Article • 02/13/2023

**forced error : strings not equal**

The conditional-error directive .ERRDIF or **.ERRDIFI** was used to generate this error.

## See also

ML Error Messages

# ML Nonfatal Error A2063

Article • 02/13/2023

**can ALIGN only to power of 2 : expression**

The expression specified with the ALIGN directive was invalid.

The **ALIGN** expression must be a power of 2 between 2 and 256, and must be less than or equal to the alignment of the current segment, structure, or union.

## See also

ML Error Messages

# ML Nonfatal Error A2064

Article • 02/13/2023

**structure alignment must be 1, 2, 4, 8, or 16**

The alignment specified in a structure definition was invalid.

## See also

ML Error Messages

# ML Nonfatal Error A2065

Article • 02/13/2023

**expected : token**

The assembler expected the given token.

## See also

ML Error Messages

# ML Nonfatal Error A2066

Article • 02/13/2023

**incompatible CPU mode and segment size**

An attempt was made to open a segment with a **USE16**, **USE32**, or **FLAT** attribute that was not compatible with the specified CPU, or to change to a 16-bit CPU while in a 32-bit segment.

The **USE32** and **FLAT** attributes must be preceded by the .386 or greater processor directive.

## See also

ML Error Messages

# ML Nonfatal Error A2069

Article • 02/13/2023

**no operands allowed for this instruction**

One or more operands were specified with an instruction that takes no operands.

## See also

ML Error Messages

# ML Nonfatal Error A2070

Article • 02/13/2023

**invalid instruction operands**

One or more operands were not valid for the instruction with which they were specified.

## See also

ML Error Messages

# ML Nonfatal Error A2074

Article • 02/13/2023

**cannot access label through segment registers**

An attempt was made to access a label through a segment register that was not assumed to its segment or group.

## See also

ML Error Messages

# ML Nonfatal Error A2078

Article • 02/13/2023

**instruction does not allow FAR indirect addressing**

A conditional jump or loop cannot take a memory operand. It must be given a relative address or label.

## See also

ML Error Messages

# ML Nonfatal Error A2079

Article • 02/13/2023

**instruction does not allow FAR direct addressing**

A conditional jump or loop cannot be to a different segment or group.

## See also

ML Error Messages

# ML Nonfatal Error A2083

Article • 02/13/2023

**invalid scale value**

A register scale was specified that was not 1, 2, 4, or 8.

## See also

ML Error Messages

# ML Nonfatal Error A2085

Article • 02/13/2023

**instruction or register not accepted in current CPU mode**

An attempt was made to use an instruction, register, or keyword that was not valid for the current processor mode.

For example, 32-bit registers require .386 or above. Control registers such as CR0 require privileged mode .386P or above. This error will also be generated for the **NEAR32**, **FAR32**, and **FLAT** keywords, which require **.386** or above.

## See also

ML Error Messages

# ML Nonfatal Error A2096

Article • 02/13/2023

**segment, group, or segment register expected**

A segment or group was expected but was not found.

One of the following occurred:

- The left operand specified with the segment override operator (:) was not a segment register (CS, DS, SS, ES, FS, or GS), group name, segment name, or segment expression.
- The ASSUME directive was given a segment register without a valid segment address, segment register, group, or the special **FLAT** group.

## See also

ML Error Messages

# ML Nonfatal Error A2097

Article • 02/13/2023

**segment expected : identifier**

The GROUP directive was given an identifier that was not a defined segment.

## See also

ML Error Messages

# ML Nonfatal Error A2107

Article • 02/13/2023

**cannot have implicit far jump or call to near label**

An attempt was made to make an implicit far jump or call to a near label in another segment.

## See also

ML Error Messages

# ML Nonfatal Error A2119

Article • 02/13/2023

**language type must be specified**

A procedure definition or prototype was not given a language type.

A language type must be declared in each procedure definition or prototype if a default language type is not specified. A default language type is set using either the .MODEL directive, **OPTION LANG**, or the ML command-line options **/Gc** or **/Gd**.

## See also

ML Error Messages

# ML Nonfatal Error A2133

Article • 02/13/2023

**register value overwritten by INVOKE**

A register was passed as an argument to a procedure, but the code generated by INVOKE to pass other arguments destroyed the contents of the register.

The AX, AL, AH, EAX, DX, DL, DH, and EDX registers may be used by the assembler to perform data conversion.

Use a different register.

## See also

ML Error Messages

# ML Nonfatal Error A2137

Article • 02/13/2023

**too few arguments to INVOKE**

The number of arguments passed using the INVOKE directive was fewer than the number of required parameters specified in the prototype for the procedure being invoked.

## See also

ML Error Messages

# ML Nonfatal Error A2189

Article • 02/13/2023

**invalid combination with segment alignment**

The alignment specified by the **ALIGN** or `EVEN` directive was greater than the current segment alignment as specified by the **SEGMENT** directive.

## See also

ML Error Messages

# ML Nonfatal Error A2206

Article • 02/13/2023

**missing operator in expression**

An expression cannot be evaluated because it is missing an operator. This error message may also be a side-effect of a preceding program error.

The following line will generate this error:

```asm
value1 = ( 1 + 2 ) 3
```

## See also

ML Error Messages

# ML Nonfatal Error A2219

Article • 02/13/2023

> Bad alignment for offset in unwind code

## Remarks

The operand for .ALLOCSTACK and .SAVEREG must be a multiple of 8. The operand for .SAVEXMM128 and .SETFRAME must be a multiple of 16.

## See also

ML error messages

# ML Warning A4004

Article • 02/13/2023

**cannot ASSUME CS**

An attempt was made to assume a value for the CS register. CS is always set to the current segment or group.

## See also

ML Error Messages

# ML Warning A4012

Article • 02/13/2023

**line number information for segment without class 'CODE'**

There were instructions in a segment that did not have a class name that ends with "CODE." The assembler did not generate CodeView information for these instructions.

CodeView cannot process modules with code in segments with class names that do not end with "CODE."

## See also

ML Error Messages

# ML Warning A4014

Article • 02/13/2023

instructions and initialized data not supported in BSS segments

An attempt was made to define initialized data within a BSS section. A BSS section is defined as a class whose name is BSS. This includes the simplified segment `.data?`.

## See also

ML Error Messages

# Processor manufacturer programming manuals

Article • 12/09/2024

This article provides links to websites that may contain programming info about processors that Microsoft doesn't make, sell, or support. Microsoft doesn't control the websites or their content.

## Processor manufacturer websites

- AMD Documentation Hub
- ARM Architecture Reference Manual
- Intel 64 and IA-32 Architectures Software Developer Manuals
- Introducing Intel® Advanced Performance Extensions (Intel APX)

## Remarks

Visual Studio and the Microsoft Macro Assembler don't support all processors.

## See also

Microsoft Macro Assembler reference
MASM BNF Grammar

---

## Feedback

Was this page helpful? Yes No

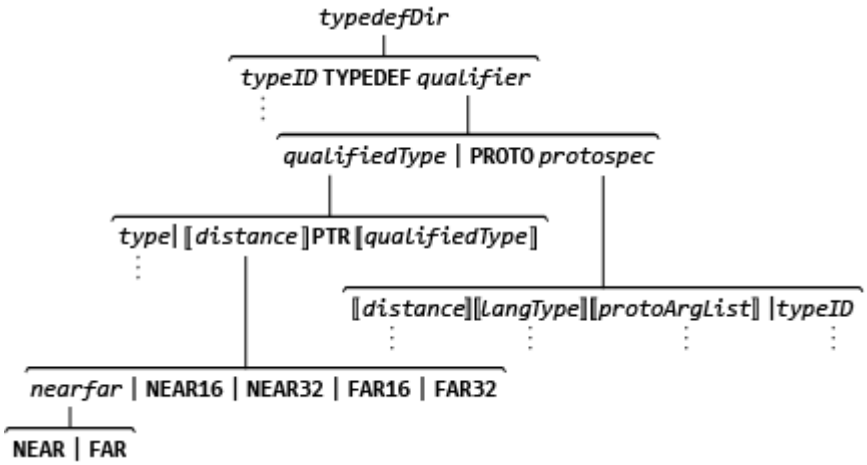
Provide product feedback | Get help at Microsoft Q&A

# Microsoft Macro Assembler BNF Grammar

Article • 03/13/2023

This page contains a BNF description of the MASM grammar. It's provided as a supplement to the reference and isn't guaranteed to be complete. Consult the reference for full information on keywords, parameters, operations, and so on.

To illustrate the use of the BNF, the following diagram shows the definition of the **`TYPEDEF`** directive, starting with the nonterminal *`typedefDir`*.




The entries under each horizontal brace are terminals, such as **`NEAR16`**, **`NEAR32`**, **`FAR16`**, and **`FAR32`**. Or, they're nonterminals such as *`qualifier`*, *`qualifiedType`*, *`distance`*, and *`protoSpec`* that can be further defined. Each italicized nonterminal in the *`typedefDir`* definition is also an entry in the BNF. Three vertical dots indicate a branching definition for a nonterminal that, for the sake of simplicity, this figure doesn't illustrate.

The BNF grammar allows recursive definitions. For example, the grammar uses *`qualifiedType`* as a possible definition for *`qualifiedType`*, which is also a component of the definition for *`qualifier`*. The "|" symbol specifies a choice between alternate expressions, for example *`endOfLine`* | *`comment`*. Double braces specify an optional parameter, for example [[ *`macroParmList`* ]]. The brackets don't actually appear in the source code.

# MASM Nonterminals

*`;;`*

*`endOfLine`* | *`comment`*

*=Dir*

*id* **=** *immExpr* *;;*

*addOp*

**+** | **-**

*aExpr*

*term* | *aExpr* **&&** *term*

*altId*

*id*

*alpha*

Any upper or lowercase letter (A-Z) or one of these four characters: `@ _ $ ?`

*arbitraryText*

*charList*

*asmInstruction*

*mnemonic* ⟦ *exprList* ⟧

*assumeDir*

**ASSUME** *assumeList* *;;*

| **ASSUME NOTHING** *;;*

*assumeList*

*assumeRegister* | *assumeList* **,** *assumeRegister*

*assumeReg*

*register* **:** *assumeVal*

*assumeRegister*

*assumeSegReg* | *assumeReg*

*assumeSegReg*

*segmentRegister* **:** *assumeSegVal*

*assumeSegVal*

*frameExpr* | **NOTHING** | **ERROR**

*assumeVal*

*qualifiedType* | **NOTHING** | **ERROR**

*bcdConst*

⟦ *sign* ⟧ *decNumber*

*binaryOp*

**==** | **!=** | **>=** | **<=** | **>** | **<** | **&**

*bitDef*

*bitFieldId* **:** *bitFieldSize* ⟦ **=** *constExpr* ⟧

*bitDefList*

*bitDef* | *bitDefList* **,** ⟦ *;;* ⟧ *bitDef*

*bitFieldId*

*id*

*bitFieldSize*

*constExpr*

*blockStatements*

*directiveList*

| **.CONTINUE** ⟦ **.IF** *cExpr* ⟧

| **.BREAK** ⟦ **.IF** *cExpr* ⟧

*bool*

**TRUE** | **FALSE**

*byteRegister*

**AL** | **AH** | **CL** | **CH** | **DL** | **DH** | **BL** | **BH** | **R8B** | **R9B** | **R10B** | **R11B** | **R12B** | **R13B** | **R14B** | **R15B**

*cExpr*

*aExpr* | *cExpr* **||** *aExpr*

*character*

Any character with ordinal in the range 0–255 except linefeed (10).

*charList*

*character* | *charList* *character*

*className*

*string*

*commDecl*

⟦ *nearfar* ⟧ ⟦ *langType* ⟧ *id* : *commType*

⟦ : *constExpr* ⟧

*commDir*

**COMM**

*commList* ;;

*comment*

; *text* ;;

*commentDir*

**COMMENT** *delimiter*

*text*

*text* *delimiter* *text* ;;

*commList*

*commDecl* | *commList* , *commDecl*

*commType*

*type* | *constExpr*

*constant*

*digits* ⟦ *radixOverride* ⟧

*constExpr*

*expr*

*contextDir*

**PUSHCONTEXT** *contextItemList* ;;

| **POPCONTEXT** *contextItemList* ;;

*contextItem*

**ASSUMES** | **RADIX** | **LISTING** | **CPU** | **ALL**

*contextItemList*

*contextItem* | *contextItemList* , *contextItem*

*controlBlock*

*whileBlock* | *repeatBlock*

*controlDir*

*controlIf* | *controlBlock*

*controlElseif*

**.ELSEIF** *cExpr* *;;*

*directiveList*

⟦ *controlElseif* ⟧

*controlIf*

**.IF** *cExpr* *;;*

*directiveList*

⟦ *controlElseif* ⟧

⟦ **.ELSE** *;;*

*directiveList*⟧

**.ENDIF** *;;*

*coprocessor*

**.8087** | **.287** | **.387** | **.NO87**

*crefDir*

*crefOption* *;;*

*crefOption*

**.CREF**

| **.XCREF** ⟦ *idList* ⟧

| **.NOCREF** ⟦ *idList* ⟧

*cxzExpr*

*expr*

| **!** *expr*

| *expr* **==** *expr*

| *expr* **!=** *expr*

*dataDecl*

**DB** | **DW** | **DD** | **DF** | **DQ** | **DT** | *dataType* | *typeId*

*dataDir*

⟦ *id* ⟧ *dataItem* *;;*

*dataItem*

*dataDecl* *scalarInstList*

| *structTag* *structInstList*

| *typeId* *structInstList*

| *unionTag* *structInstList*

| *recordTag* *recordInstList*

*dataType*

**BYTE** | **SBYTE** | **WORD** | **SWORD** | **DWORD** | **SDWORD** | **FWORD** | **QWORD** | **SQWORD** | **TBYTE** | **OWORD** | **REAL4** | **REAL8** | **REAL10** | **MMWORD** | **XMMWORD** | **YMMWORD**

*decdigit*

**0** | **1** | **2** | **3** | **4** | **5** | **6** | **7** | **8** | **9**

*decNumber*

*decdigit*

| *decNumber* *decdigit*

*delimiter*

Any character except *whiteSpaceCharacter*

*digits*

*decdigit*

| *digits* *decdigit*

| *digits* *hexdigit*

*directive*

*generalDir* | *segmentDef*

*directiveList*

*directive* | *directiveList* *directive*

*distance*

*nearfar* | **NEAR16** | **NEAR32** | **FAR16** | **FAR32**

*e01*

*e01* *orOp* *e02* | *e02*

*e02*

*e02* **AND** *e03* | *e03*

*e03*

**NOT** *e04* | *e04*

*e04*

*e04* *relOp* *e05* | *e05*

*e05*

*e05* *addOp* *e06* | *e06*

*e06*

*e06* *mulOp* *e07* | *e06* *shiftOp* *e07* | *e07*

*e07*

*e07* *addOp* *e08* | *e08*

*e08*

**HIGH** *e09*
| **LOW** *e09*
| **HIGHWORD** *e09*
| **LOWWORD** *e09*
| *e09*

*e09*

**OFFSET** *e10*
| **SEG** *e10*
| **LROFFSET** *e10*
| **TYPE** *e10*
| **THIS** *e10*
| *e09* **PTR** *e10*
| *e09* **:** *e10*
| *e10*

*e10*

*e10* **.** *e11*
| *e10* ⟦ *expr* ⟧
| *e11*

*e11*

**(** *expr* **)**
| ⟦ *expr* ⟧
| **WIDTH** *id*
| **MASK** *id*
| **SIZE** *sizeArg*
| **SIZEOF** *sizeArg*
| **LENGTH** *id*
| **LENGTHOF** *id*

| *recordConst*
| *string*
| *constant*
| *type*
| *id*
| **$**
| *segmentRegister*
| *register*
| **ST**
| **ST** **(** *expr* **)**

*echoDir*
**ECHO** *arbitraryText* *;;*
**%OUT** *arbitraryText* *;;*

*elseifBlock*
*elseifStatement* *;;*
*directiveList*
⟦ *elseifBlock* ⟧

*elseifStatement*
**ELSEIF** *constExpr*
| **ELSEIFE** *constExpr*
| **ELSEIFB** *textItem*
| **ELSEIFNB** *textItem*
| **ELSEIFDEF** *id*
| **ELSEIFNDEF** *id*
| **ELSEIFDIF** *textItem* **,** *textItem*
| **ELSEIFDIFI** *textItem* **,** *textItem*
| **ELSEIFIDN** *textItem* **,** *textItem*
| **ELSEIFIDNI** *textItem* **,** *textItem*
| **ELSEIF1**
| **ELSEIF2**

*endDir*
**END** ⟦ *immExpr* ⟧ *;;*

*endpDir*
*procId* **ENDP** *;;*

*endsDir*

*id* **ENDS** *;;*

*equDir*

*textMacroId* **EQU** *equType* *;;*

*equType*

*immExpr* | *textLiteral*

*errorDir*

*errorOpt* *;;*

*errorOpt*

**.ERR** ⟦ *textItem* ⟧
| **.ERRE** *constExpr* ⟦ *optText* ⟧
| **.ERRNZ** *constExpr* ⟦ *optText* ⟧
| **.ERRB** *textItem* ⟦ *optText* ⟧
| **.ERRNB** *textItem* ⟦ *optText* ⟧
| **.ERRDEF** *id* ⟦ *optText* ⟧
| **.ERRNDEF** *id* ⟦ *optText* ⟧
| **.ERRDIF** *textItem* **,** *textItem* ⟦ *optText* ⟧
| **.ERRDIFI** *textItem* **,** *textItem* ⟦ *optText* ⟧
| **.ERRIDN** *textItem* **,** *textItem* ⟦ *optText* ⟧
| **.ERRIDNI** *textItem* **,** *textItem* ⟦ *optText* ⟧
| **.ERR1** ⟦ *textItem* ⟧
| **.ERR2** ⟦ *textItem* ⟧

*exitDir*

**.EXIT** ⟦ *expr* ⟧ *;;*

*exitmDir*

**:** **EXITM** | **EXITM** *textItem*

*exponent*

**E** ⟦ *sign* ⟧ *decNumber*

*expr*

**SHORT** *e05*
| **.TYPE** *e01*
| **OPATTR** *e01*
| *e01*

*exprList*

*expr* | *exprList* **,** *expr*

*externDef*

⟦ *langType* ⟧ *id* ⟦ **(** *altId* **)** ⟧ **:** *externType*

*externDir*

*externKey* *externList* *;;*

*externKey*

**EXTRN** | **EXTERN** | **EXTERNDEF**

*externList*

*externDef* | *externList* **,** ⟦ *;;* ⟧ *externDef*

*externType*

**ABS** | *qualifiedType*

*fieldAlign*

*constExpr*

*fieldInit*

⟦ *initValue* ⟧ | *structInstance*

*fieldInitList*

*fieldInit* | *fieldInitList* **,** ⟦ *;;* ⟧ *fieldInit*

*fileChar*

*delimiter*

*fileCharList*

*fileChar* | *fileCharList* *fileChar*

*fileSpec*

*fileCharList* | *textLiteral*

*flagName*

**ZERO?** | **CARRY?** | **OVERFLOW?** | **SIGN?** | **PARITY?**

*floatNumber*

⟦ *sign* ⟧ *decNumber* **.** ⟦ *decNumber* ⟧ ⟦ *exponent* ⟧

| *digits* **R**

| *digits* **r**

*forcDir*

**FORC** | **IRPC**

*forDir*

**FOR** | **IRP**

*forParm*

*id* ⟦ **:** *forParmType* ⟧

*forParmType*

**REQ** | **=** *textLiteral*

*fpuRegister*

**ST** *expr*

*frameExpr*

**SEG** *id*

| **DGROUP** **:** *id*

| *segmentRegister* **:** *id*

| *id*

*generalDir*

*modelDir* | *segOrderDir* | *nameDir*

| *includeLibDir* | *commentDir*

| *groupDir* | *assumeDir*

| *structDir* | *recordDir* | *typedefDir*

| *externDir* | *publicDir* | *commDir* | *protoTypeDir*

| *equDir* | *=Dir* | *textDir*

| *contextDir* | *optionDir* | *processorDir*

| *radixDir*

| *titleDir* | *pageDir* | *listDir*

| *crefDir* | *echoDir*

| *ifDir* | *errorDir* | *includeDir*

| *macroDir* | *macroCall* | *macroRepeat* | *purgeDir*

| *macroWhile* | *macroFor* | *macroForc*

| *aliasDir*

*gpRegister*

**AX** | **EAX** | **CX** | **ECX** | **DX** | **EDX** | **BX** | **EBX**

| **DI** | **EDI** | **SI** | **ESI** | **BP** | **EBP** | **SP** | **ESP**
| **R8W** | **R8D** | **R9W** | **R9D** | **R12D** | **R13W** | **R13D** | **R14W** | **R14D**

*groupDir*

*groupId* **GROUP** *segIdList*

*groupId*

*id*

*hexdigit*

**a** | **b** | **c** | **d** | **e** | **f**
| **A** | **B** | **C** | **D** | **E** | **F**

*id*

*alpha*
| *id* *alpha*
| *id* *decdigit*

Maximum length is 247 characters.

*idList*

*id* | *idList* **,** *id*

*ifDir*

*ifStatement* *;;*
*directiveList*
⟦ *elseifBlock* ⟧
⟦ **ELSE** *;;*
*directiveList* ⟧ *;;*

*ifStatement*

**IF** *constExpr*
| **IFE** *constExpr*
| **IFB** *textItem*
| **IFNB** *textItem*
| **IFDEF** *id*
| **IFNDEF** *id*
| **IFDIF** *textItem* **,** *textItem*
| **IFDIFI** *textItem* **,** *textItem*
| **IFIDN** *textItem* **,** *textItem*
| **IFIDNI** *textItem* **,** *textItem*

| **IF1**
| **IF2**

*immExpr*
*expr*

*includeDir*
**INCLUDE** *fileSpec* *;;*

*includeLibDir*
**INCLUDELIB** *fileSpec* *;;*

*initValue*
*immExpr*
| *string*
| **?**
| *constExpr* **DUP** **(** *scalarInstList* **)**
| *floatNumber*
| *bcdConst*

*inSegDir*
⟦ *labelDef* ⟧ *inSegmentDir*

*inSegDirList*
*inSegDir* | *inSegDirList* *inSegDir*

*inSegmentDir*
*instruction*
| *dataDir*
| *controlDir*
| *startupDir*
| *exitDir*
| *offsetDir*
| *labelDir*
| *procDir* ⟦ *localDirList* ⟧ ⟦ *inSegDirList* ⟧ *endpDir*
| *invokeDir*
| *generalDir*

*instrPrefix*
**REP** | **REPE** | **REPZ** | **REPNE** | **REPNZ** | **LOCK**

*instruction*

⟦ *instrPrefix* ⟧ *asmInstruction*

*invokeArg*

*register* **::** *register*

| *expr*

| **ADDR** *expr*

*invokeDir*

**INVOKE** *expr* ⟦ **,** ⟦ *;;* ⟧ *invokeList* ⟧ *;;*

*invokeList*

*invokeArg* | *invokeList* **,** ⟦ *;;* ⟧ *invokeArg*

*keyword*

Any reserved word.

*keywordList*

*keyword* | *keyword* *keywordList*

*labelDef*

*id* **:** | *id* **::** | **@@:**

*labelDir*

*id* **LABEL** *qualifiedType* *;;*

*langType*

**C** | **PASCAL** | **FORTRAN** | **BASIC** | **SYSCALL** | **STDCALL**

*listDir*

*listOption* *;;*

*listOption*

**.LIST**

| **.NOLIST**

| **.XLIST**

| **.LISTALL**

| **.LISTIF**

| **.LFCOND**

| **.NOLISTIF**

| **.SFCOND**

| **.TFCOND**

| `.LISTMACROALL` | `.LALL`
| `.NOLISTMACRO` | `.SALL`
| `.LISTMACRO` | `.XALL`

*localDef*
`LOCAL` *idList* `;;`

*localDir*
`LOCAL` *parmList* `;;`

*localDirList*
*localDir* | *localDirList* *localDir*

*localList*
*localDef* | *localList* *localDef*

*macroArg*
`%` *constExpr*
| `%` *textMacroId*
| `%` *macroFuncId* `(` *macroArgList* `)`
| *string*
| *arbitraryText*
| `<` *arbitraryText* `>`

*macroArgList*
*macroArg* | *macroArgList* `,` *macroArg*

*macroBody*
`[[` *localList* `]]` *macroStmtList*

*macroCall*
*id* *macroArgList* `;;`
| *id* `(` *macroArgList* `)`

*macroDir*
*id* `MACRO` `[[` *macroParmList* `]]` `;;`
*macroBody*
`ENDM` `;;`

*macroFor*
*forDir* *forParm* `,` `<` *macroArgList* `>` `;;`

*macroBody*

**ENDM** *;;*

*macroForc*

*forcDir* *id* **,** *textLiteral* *;;*

*macroBody*

**ENDM** *;;*

*macroFuncId*

*id*

*macroId*

*macroProcId* | *macroFuncId*

*macroIdList*

*macroId* | *macroIdList* **,** *macroId*

*macroLabel*

*id*

*macroParm*

*id* ⟦ **:** *parmType* ⟧

*macroParmList*

*macroParm* | *macroParmList* **,** ⟦ *;;* ⟧ *macroParm*

*macroProcId*

*id*

*macroRepeat*

*repeatDir* *constExpr* *;;*

*macroBody*

**ENDM** *;;*

*macroStmt*

*directive*

| *exitmDir*

| **:** *macroLabel*

| **GOTO** *macroLabel*

*macroStmtList*

*macroStmt* *;;*

| *macroStmtList* *macroStmt* *;;*

*macroWhile*

**WHILE** *constExpr* *;;*

*macroBody*

**ENDM** *;;*

*mapType*

**ALL** | **NONE** | **NOTPUBLIC**

*memOption*

**TINY** | **SMALL** | **MEDIUM** | **COMPACT** | **LARGE** | **HUGE** | **FLAT**

*mnemonic*

Instruction name.

*modelDir*

**.MODEL** *memOption* [[ **,** *modelOptlist* ]] *;;*

*modelOpt*

*langType* | *stackOption*

*modelOptlist*

*modelOpt* | *modelOptlist* **,** *modelOpt*

*module*

[[ *directiveList* ]] *endDir*

*mulOp*

***** | **/** | **MOD**

*nameDir*

**NAME** *id* *;;*

*nearfar*

**NEAR** | **FAR**

*nestedStruct*

*structHdr* [[ *id* ]] *;;*

*structBody*

**ENDS** *;;*

*offsetDir*

*offsetDirType* **;;**

*offsetDirType*

**EVEN**

| **ORG** *immExpr*

| **ALIGN** ⟦ *constExpr* ⟧

*offsetType*

**GROUP** | **SEGMENT** | **FLAT**

*oldRecordFieldList*

⟦ *constExpr* ⟧ | *oldRecordFieldList* **,** ⟦ *constExpr* ⟧

*optionDir*

**OPTION** *optionList* **;;**

*optionItem*

**CASEMAP** **:** *mapType*

| **DOTNAME** | **NODOTNAME**

| **EMULATOR** | **NOEMULATOR**

| **EPILOGUE** **:** *macroId*

| **EXPR16** | **EXPR32**

| **LANGUAGE** **:** *langType*

| **LJMP** | **NOLJMP**

| **M510** | **NOM510**

| **NOKEYWORD** **:** **<** *keywordList* **>**

| **NOSIGNEXTEND**

| **OFFSET** **:** *offsetType*

| **OLDMACROS** | **NOOLDMACROS**

| **OLDSTRUCTS** | **NOOLDSTRUCTS**

| **PROC** **:** *oVisibility*

| **PROLOGUE** **:** *macroId*

| **READONLY** | **NOREADONLY**

| **SCOPED** | **NOSCOPED**

| **SEGMENT** **:** *segSize*

| **SETIF2** **:** *bool*

*optionList*

*optionItem* | *optionList* **,** ⟦ **;;** ⟧ *optionItem*

*optText*

, *textItem*

*orOp*

**OR** | **XOR**

*oVisibility*

**PUBLIC** | **PRIVATE** | **EXPORT**

*pageDir*

**PAGE** ⟦ *pageExpr* ⟧ *;;*

*pageExpr*

**+** | ⟦ *pageLength* ⟧ ⟦ **,** *pageWidth* ⟧

*pageLength*

*constExpr*

*pageWidth*

*constExpr*

*parm*

*parmId* ⟦ **:** *qualifiedType* ⟧

| *parmId* ⟦ *constExpr* ⟧ ⟦ **:** *qualifiedType* ⟧

*parmId*

*id*

*parmList*

*parm* | *parmList* **,** ⟦ *;;* ⟧ *parm*

*parmType*

**REQ** | **=** *textLiteral* | **VARARG**

*pOptions*

⟦ *distance* ⟧ ⟦ *langType* ⟧ ⟦ *oVisibility* ⟧

*primary*

*expr* *binaryOp* *expr* | *flagName* | *expr*

*procDir*

*procId* **PROC** ⟦ *pOptions* ⟧ ⟦ **<** *macroArgList* **>** ⟧

⟦ *usesRegs* ⟧ ⟦ *procParmList* ⟧

*processor*

| **.386** | **.386p** | **.486** | **.486P**

| **.586** | **.586P** | **.686** | **.686P** | **.387**

*processorDir*

*processor* *;;*

| *coprocessor* *;;*

*procId*

*id*

*procItem*

*instrPrefix* | *dataDir* | *labelDir* | *offsetDir* | *generalDir*

*procParmList*

⟦ **,** ⟦ *;;* ⟧ *parmList* ⟧

⟦ **,** ⟦ *;;* ⟧ *parmId* **:VARARG** ⟧

*protoArg*

⟦ *id* ⟧ **:** *qualifiedType*

*protoArgList*

⟦ **,** ⟦ *;;* ⟧ *protoList* ⟧

⟦ **,** ⟦ *;;* ⟧ ⟦ *id* ⟧ **:VARARG** ⟧

*protoList*

*protoArg*

| *protoList* **,** ⟦ *;;* ⟧ *protoArg*

*protoSpec*

⟦ *distance* ⟧ ⟦ *langType* ⟧ ⟦ *protoArgList* ⟧

| *typeId*

*protoTypeDir*

*id* **PROTO** *protoSpec*

*pubDef*

⟦ *langType* ⟧ *id*

*publicDir*

**PUBLIC** *pubList* *;;*

*pubList*

*pubDef* | *pubList* **,** ⟦ *;;* ⟧ *pubDef*

*purgeDir*

**PURGE** *macroIdList*

*qualifiedType*

*type*

| ⟦ *distance* ⟧ **PTR** ⟦ *qualifiedType* ⟧

*qualifier*

*qualifiedType* | **PROTO** *protoSpec*

*quote*

**"** | **'**

*qwordRegister*

**RAX** | **RCX** | **RDX** | **RBX** | **RSP** | **RBP** | **RSI** | **RDI**

| **R8** | **R9** | **R10** | **R11** | **R12** | **R13** | **R14** | **R15**

*radixDir*

**.RADIX** *constExpr* *;;*

*radixOverride*

**h** | **o** | **q** | **t** | **y**

| **H** | **O** | **Q** | **T** | **Y**

*recordConst*

*recordTag* **{** *oldRecordFieldList* **}**

| *recordTag* **<** *oldRecordFieldList* **>**

*recordDir*

*recordTag* **RECORD** *bitDefList* *;;*

*recordFieldList*

⟦ *constExpr* ⟧ | *recordFieldList* **,** ⟦ *;;* ⟧ ⟦ *constExpr* ⟧

*recordInstance*

**{** ⟦ *;;* ⟧ *recordFieldList* ⟦ *;;* ⟧ **}**

| **<** *oldRecordFieldList* **>**

| *constExpr* **DUP** **(** *recordInstance* **)**

*recordInstList*

*recordInstance* | *recordInstList* , ⟦ *;;* ⟧ *recordInstance*

*recordTag*

*id*

*register*

*specialRegister* | *gpRegister* | *byteRegister* | *qwordRegister* | *fpuRegister* | *SIMDRegister* | *segmentRegister*

*regList*

*register* | *regList* *register*

*relOp*

**EQ** | **NE** | **LT** | **LE** | **GT** | **GE**

*repeatBlock*

**.REPEAT** *;;*

*blockStatements* *;;* *untilDir* *;;*

*repeatDir*

**REPEAT** | **REPT**

*scalarInstList*

*initValue*

| *scalarInstList* , ⟦ *;;* ⟧ *initValue*

*segAlign*

**BYTE** | **WORD** | **DWORD** | **PARA** | **PAGE**

*segAttrib*

**PUBLIC** | **STACK** | **COMMON** | **MEMORY** | **AT** *constExpr* | **PRIVATE**

*segDir*

**.CODE** ⟦ *segId* ⟧

| **.DATA**

| **.DATA?**

| **.CONST**

| **.FARDATA** ⟦ *segId* ⟧

| **.FARDATA?** ⟦ *segId* ⟧

| **.STACK** ⟦ *constExpr* ⟧

*segId*

*id*

*segIdList*

*segId*

| *segIdList* , *segId*

*segmentDef*

*segmentDir* ⟦ *inSegDirList* ⟧ *endsDir* | *simpleSegDir* ⟦ *inSegDirList* ⟧ ⟦ *endsDir* ⟧

*segmentDir*

*segId* **SEGMENT** ⟦ *segOptionList* ⟧ *;;*

*segmentRegister*

**CS** | **DS** | **ES** | **FS** | **GS** | **SS**

*segOption*

*segAlign*

| *segRO*

| *segAttrib*

| *segSize*

| *className*

*segOptionList*

*segOption* | *segOptionList* *segOption*

*segOrderDir*

**.ALPHA** | **.SEQ** | **.DOSSEG** | **DOSSEG**

*segRO*

**READONLY**

*segSize*

**USE16** | **USE32** | **FLAT**

*shiftOp*

**SHR** | **SHL**

*sign*

**+** | **-**

*simdRegister*

**MM0** | **MM1** | **MM2** | **MM3** | **MM4** | **MM5** | **MM6** | **MM7**

| *xmmRegister*
| **YMM0** | **YMM1** | **YMM2** | **YMM3** | **YMM4** | **YMM5** | **YMM6** | **YMM7**
| **YMM8** | **YMM9** | **YMM10** | **YMM11** | **YMM12** | **YMM13** | **YMM14** | **YMM15**

*simpleExpr*
**(** *cExpr* **)** | *primary*

*simpleSegDir*
*segDir* *;;*

*sizeArg*
*id* | *type* | *e10*

*specialChars*
**:** | **.** | **[** | **]** | **(** | **)** | **<** | **>** | **{** | **}**
| **+** | **-** | **/** | ***** | **&** | **%** | **!**
| **'** | **** | **=** | **;** | **,** | **"**
| *whiteSpaceCharacter*
| *endOfLine*

*specialRegister*
**CR0** | **CR2** | **CR3**
| **DR0** | **DR1** | **DR2** | **DR3** | **DR6** | **DR7**
| **TR3** | **TR4** | **TR5** | **TR6** | **TR7**

*stackOption*
**NEARSTACK** | **FARSTACK**

*startupDir*
**.STARTUP** *;;*

*stext*
*stringChar* | *stext* *stringChar*

*string*
*quote* ⟦ *stext* ⟧ *quote*

*stringChar*
*quote* *quote* | Any character except quote.

*structBody*
*structItem* *;;*

| *structBody* *structItem* *;;*

*structDir*

*structTag* *structHdr* ⟦ *fieldAlign* ⟧
⟦ **,** **NONUNIQUE** ⟧ *;;*
*structBody*
*structTag* **ENDS** *;;*

*structHdr*

**STRUC** | **STRUCT** | **UNION**

*structInstance*

**<** ⟦ *fieldInitList* ⟧ **>**
| **{** ⟦ *;;* ⟧ ⟦ *fieldInitList* ⟧ ⟦ *;;* ⟧ **}**
| *constExpr* **DUP** ( *structInstList* )

*structInstList*

*structInstance* | *structInstList* **,** ⟦ *;;* ⟧ *structInstance*

*structItem*

*dataDir*
| *generalDir*
| *offsetDir*
| *nestedStruct*

*structTag*

*id*

*term*

*simpleExpr* | **!** *simpleExpr*

*text*

*textLiteral* | *text* *character* | **!** *character* *text* | *character* | **!** *character*

*textDir*

*id* *textMacroDir* *;;*

*textItem*

*textLiteral* | *textMacroId* | **%** *constExpr*

*textLen*

*constExpr*

*textList*

*textItem* | *textList* , ⟦ *;;* ⟧ *textItem*

*textLiteral*

**<** *text* **>** *;;*

*textMacroDir*

**CATSTR** ⟦ *textList* ⟧

| **TEXTEQU** ⟦ *textList* ⟧

| **SIZESTR** *textItem*

| **SUBSTR** *textItem* , *textStart* ⟦ , *textLen* ⟧

| **INSTR** ⟦ *textStart* , ⟧ *textItem* , *textItem*

*textMacroId*

*id*

*textStart*

*constExpr*

*titleDir*

*titleType* *arbitraryText* *;;*

*titleType*

**TITLE** | **SUBTITLE** | **SUBTTL**

*type*

*structTag*

| *unionTag*

| *recordTag*

| *distance*

| *dataType*

| *typeId*

*typedefDir*

*typeId* **TYPEDEF** *qualifier*

*typeId*

*id*

*unionTag*

*id*

*untilDir*

**.UNTIL** *cExpr* *;;*

**.UNTILCXZ** [[ *cxzExpr* ]] *;;*

*usesRegs*

**USES** *regList*

*whileBlock*

**.WHILE** *cExpr* *;;*

*blockStatements* *;;*

**.ENDW**

*whiteSpaceCharacter*

ASCII 8, 9, 11–13, 26, 32

*xmmRegister*

**XMM0** | **XMM1** | **XMM2** | **XMM3** | **XMM4** | **XMM5** | **XMM6** | **XMM7**

| **XMM8** | **XMM9** | **XMM10** | **XMM11** | **XMM12** | **XMM13** | **XMM14** | **XMM15**